

Document Title	Layered Software Architecture
Document Owner	AUTOSAR
Document Responsibility	AUTOSAR
Document Identification No	53
Document Status	published
Part of AUTOSAR Standard	Classic Platform
Part of Standard Release	R25-11

Document Change History			
Date	Release	Changed by	Change Description
2025-11-27	R25-11	AUTOSAR Release Management	➤ Added information about VDP ➤ Added information about Mirror ➤ Updated slides about libraries, DDS ➤ Removed TTCan ➤ Removed FIs and Eep ➤ Removed LdCom ➤ Minor changes
2024-11-27	R24-11	AUTOSAR Release Management	➤ Added L-SDU Router ➤ Incorporated J1939Fscp transformer into comm stack extensions ➤ Incorporated Partitioning examples ➤ Incorporated migration of BSWModuleList to BSWGeneral ➤ Added I2C in Comm Drivers ➤ Improve documentation about error handling and usage ➤ Cleanup remains of partition restart ➤ Added changes in memory manipulation library: Copy, Set, Move, Compare ➤ Remove E2EPW support ➤ Added changes to error handling

Document Change History			
Date	Release	Changed by	Change Description
2023-11-23	R23-11	AUTOSAR Release Management	➤ Added information about charging management (ChrgM) and firewall ➤ Editorial changes
2022-11-24	R22-11	AUTOSAR Release Management	➤ Incorporated new concepts for Vehicle-2-X Data Manager, MACsec, CAN XL, DDS, Secured Time Synchronization, Vehicle-2-X Support for China ➤ Editorial changes
2021-11-25	R21-11	AUTOSAR Release Management	➤ Incorporated draft concept for new Memory Driver and Memory Access
2020-11-30	R20-11	AUTOSAR Release Management	➤ Removed Pretended Networking ➤ Added caveats for E2E Protection Wrapper ➤ Layer Interaction Matrix: Allow Crypto Driver to access Memory Services ➤ Incorporated new concepts for Intrusion Detection System Manager, CP Software Clusters
2019-11-28	R19-11	AUTOSAR Release Management	➤ Incorporated new concepts for Atomic multicore safe operations, Signal-service-translation, NV data handling enhancement ➤ Changed Document Status from Final to published

Document Change History			
Date	Release	Changed by	Change Description
2018-10-31	4.4.0	AUTOSAR Release Management	➤ Adopting LIN Slave Support, LinNm removed ➤ New Concepts: Key Management, 1st draft of MCAL Multicore Distribution ➤ Editorial changes
2017-12-08	4.3.1	AUTOSAR Release Management	➤ Editorial changes
2016-11-30	4.3.0	AUTOSAR Release Management	➤ Incorporated new 4.3 concepts for Crypto Stack, Vehicle-2-X Communication, SOME/IP Transport Protocol, DLT rework ➤ Removed obsolete Dbg module ➤ Editorial changes
2015-07-31	4.2.2	AUTOSAR Release Management	➤ Editorial changes

Document Change History			
Date	Release	Changed by	Change Description
2014-10-31	4.2.1	AUTOSAR Release Management	➤ Incorporated new 4.2 concepts for: Switch Configuration; Sender-Receiver-Serialization; CAN-FD; Large-Data-COM; E2E-Extension; Global Time Synchronization; Support for Post-build ECU-Configuration; Secure-Onboard-Communication; ASIL/QM-Protection ➤ Introduction of new error classification ➤ Editorial changes
2014-03-31	4.1.3	AUTOSAR Release Management	➤ Editorial changes
2013-03-15	4.1.1	AUTOSAR Administration	➤ Clarification of partial network support for CAN/LIN slave. ➤ New Ethernet stack extensions ➤ Added Crypto Service Manager to System Services ➤ Revised presentation of J1939 and added new J1939 modules ➤ Added new energy management concepts: "Pretended Networking", "ECU Degradation" ➤ Added new modules: "Output Compare Unit Driver" and "Time Service" ➤ Changed handling of Production Errors ➤ Fixed various typography and layout issues
2011-12-22	4.0.3	AUTOSAR Administration	➤ Added a note for the R3-compatibility FlexRay Transport Layer FrArTp on slide "ki890". ➤ Added an overview chapter for energy management and partial networking ➤ Corrected examples regarding DEM symbol generation ➤ Fixed minor typography issues ➤ Clarification of term AUTOSAR-ECU on slide "94jt1" ➤ Corrected CDD access description for EcuM on slide "11123"

Document Change History			
Date	Release	Changed by	Change Description
2009-12-18	4.0.1	AUTOSAR Administration	➤ Added a note regarding support for System Basis Chips on slide "94juq" ➤ Clarification of DBG and DLT text on slide "3edfg" ➤ Corrected DBG description on slide "11231"
2010-02-02	3.1.4	AUTOSAR Administration	➤ The document has been newly structured. There are now 3 main parts: ■ Architecture ■ Configuration ■ Integration and Runtime Aspects ➤ The whole content has been updated to reflect the content of the R 4.0 specifications. ➤ Topics which have been newly introduced or heavily extended in release 4.0 have been added. E.g.,. Multi-Core Systems, Partitioning, Mode Management, Error Handling, Reporting and Diagnostic, Debugging, Measurement and Calibration, Functional Safety etc ➤ Legal disclaimer revised
2008-08-13	3.1.1	AUTOSAR Administration	➤ Legal disclaimer revised
2007-12-21	3.0.1	AUTOSAR Administration	➤ Updates based on new wakeup/startup concepts ➤ Detailed explanation for post-build time configuration ➤ "Slimming" of LIN stack description ➤ ICC2 figure ➤ Document meta information extended ➤ Small layout adaptations made

Document Change History			
Date	Release	Changed by	Change Description
2007-01-24	2.1.15	AUTOSAR Administration	➤ ICC clustering added. ➤ Document contents harmonized ➤ Legal disclaimer revised ➤ Release Notes added ➤ “Advice for users” revised ➤ “Revision Information” added
2006-11-28	2.1.1	AUTOSAR Administration	Rework Of: ➤ Error Handling ➤ Scheduling Mechanisms ➤ More updates according to architectural decisions in R2.0
2006-01-02	1.0.1	AUTOSAR Administration	➤ Correct version released
2005-05-31	1.0.0	AUTOSAR Administration	➤ Initial release

Disclaimer

Disclaimer

This work (specification and/or software implementation) and the material contained in it, as released by AUTOSAR, is for the purpose of information only. AUTOSAR and the companies that have contributed to it shall not be liable for any use of the work.

The material contained in this work is protected by copyright and other types of intellectual property rights. The commercial exploitation of the material contained in this work requires a license to such intellectual property rights.

This work may be utilized or reproduced without any modification, in any form or by any means, for informational purposes only. For any other purpose, no part of the work may be utilized or reproduced, in any form or by any means, without permission in writing from the publisher.

The work has been developed for automotive applications only. It has neither been developed, nor tested for non-automotive applications.

The word AUTOSAR and the AUTOSAR logo are registered trademarks.

Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置

3. 集成与运行时方面

Introduction

目的与输入

本文档的目的

分层软件架构描述了AUTOSAR的软件架构：

它采用自上而下的方法描述了AUTOSAR软件的分层结构，将基础软件模块映射到软件层，并展示了它们之间的关系。

本文档不包含要求，仅供参考。所举示例并非在所有方面都力求完备。

本文档侧重于概念分层软件架构的静态视图：它并未规定包含详细静态与动态接口描述的结构化软件架构（设计），■ 这些信息已包含在基础软件模块自身的规范中。

输入

本文档基于AUTOSAR的规范和需求文档。

Introduction

范围与可扩展性

AUTOSAR的应用范围

AUTOSAR专用于汽车电子控制单元（ECU）。此类ECU具有以下特性：

- 与硬件（传感器和执行器）的强交互作用，
- 连接到车辆网络，如CAN、LIN、FlexRay或以太网，
- 微控制器（通常为16或32位）在计算能力和内存资源方面有限（与企业级解决方案相比），

实时系统与

程序执行从内部或外部闪存。

注意：在AUTOSAR的定义中，一个ECU指一个微控制器加上外设及相应的软件/配置。机械设计不在AUTOSAR的范围内。这意味着，如果一个外壳内布置了多个微控制器，那么每个微控制器都需要单独描述为一个AUTOSAR-ECU实例。

AUTOSAR可扩展性

TAUTOSAR软件架构是一种通用方法：

- 标准模块可在保持合规性的同时扩展功能，■ 但其配置仍需在自动基础软件配置流程中予以考虑！
- 非标准模块可作为复杂驱动程序集成至基于AUTOSAR的系统中。

无法添加更多层。

Architecture – Overview of Software Layers

俯视图

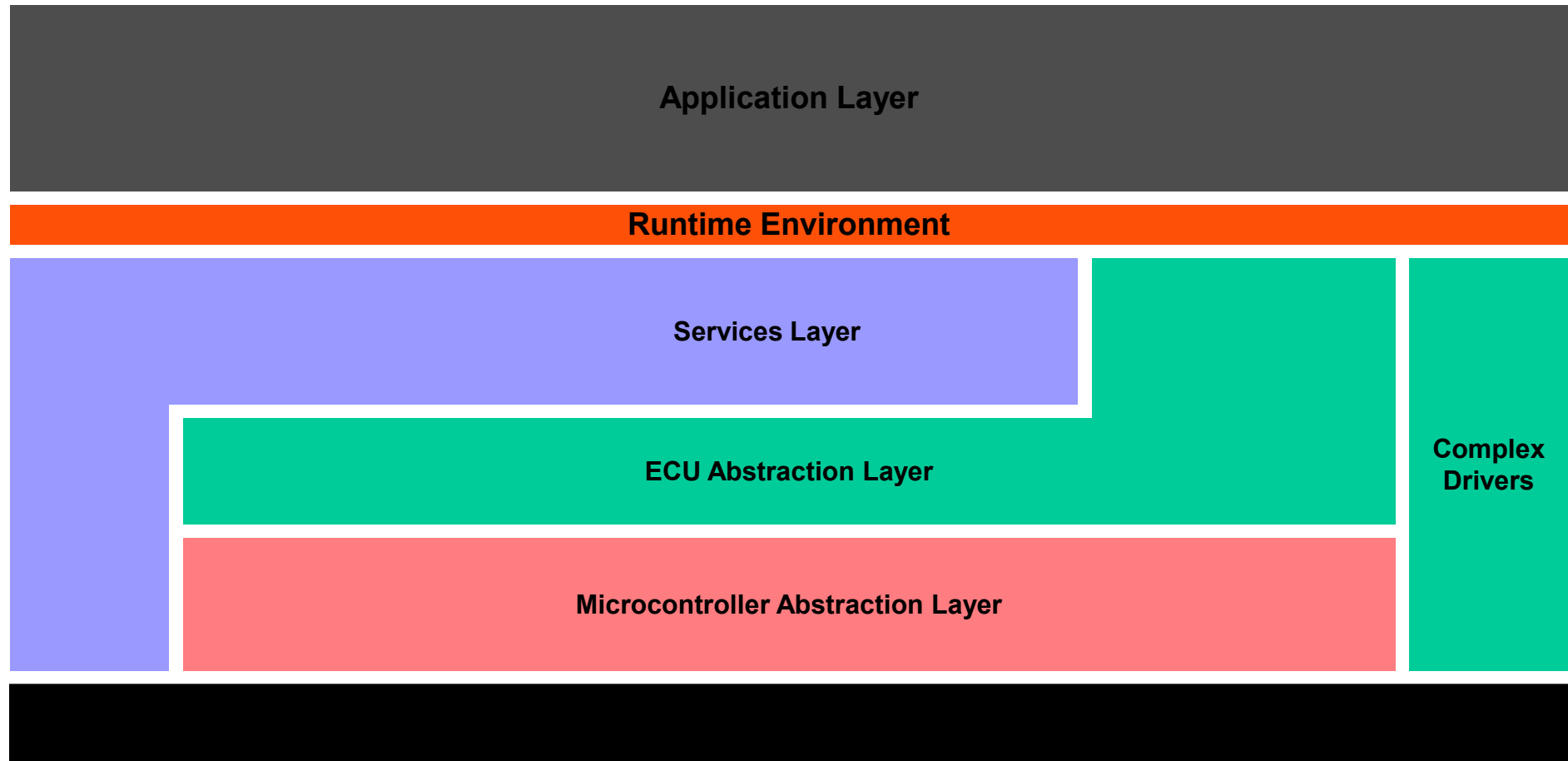
AUTOSAR架构在最高抽象层次上区分了三个软件层：运行在微控制器上的应用层、运行时环境层和基础软件层。



Architecture – Overview of Software Layers

粗略视图

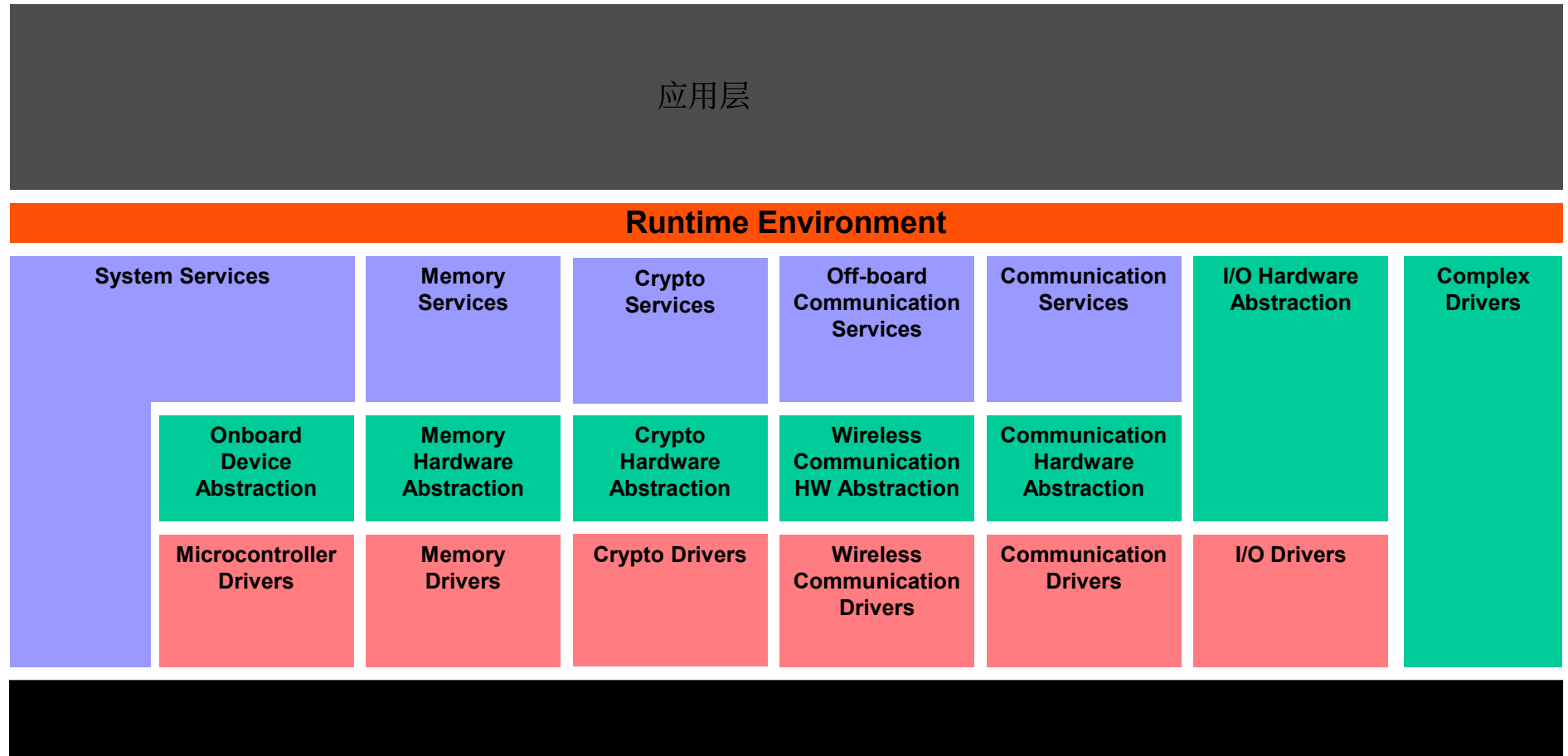
AUTOSAR基础软件进一步划分为以下层次：服务层、ECU抽象层、微控制器抽象层以及复杂驱动层。



Architecture – Overview of Software Layers

详细视图

基础软件层进一步划分为功能组。服务的示例包括系统、内存和通信服务。



Architecture – Overview of Software Layers

微控制器抽象层

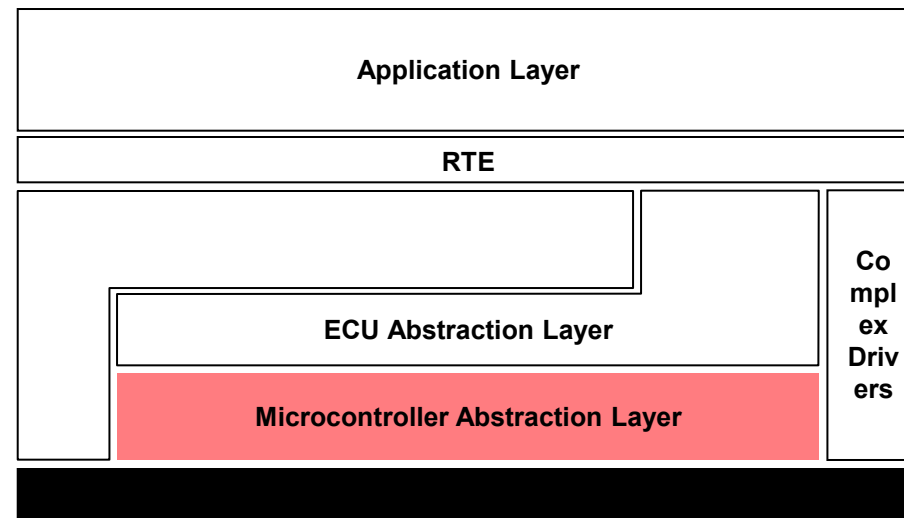
微控制器抽象层是基础软件的最低软件层。它包含内部驱动程序，这些是可直接访问微控制器及内部外设的软件模块。

任务

使高层软件独立于 μ C

属性

实现： μ C 依赖的上层接口：标准化且
 μ C 独立



Architecture – Overview of Software Layers

ECU抽象层

ECU抽象层与微控制器抽象层的驱动程序接口。
。它还包含外部设备的驱动程序。

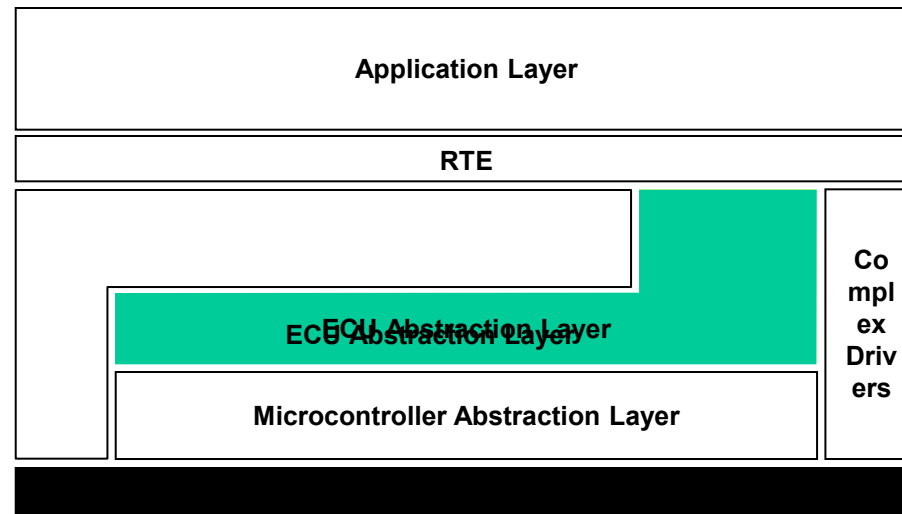
它提供了一个API，用于访问外围设备和器件，
无论其位置（微控制器内部/外部）以及它们与
微控制器的连接方式（端口引脚、接口类型）如
何。

任务

使高层软件独立于ECU硬件布局

属性

实现：μC独立，ECU硬件依赖 上层接口μC与EC
U硬件独立



Architecture – Overview of Software Layers

复杂驱动因素

复杂驱动层从硬件延伸至运行时环境（RTE）

。

任务

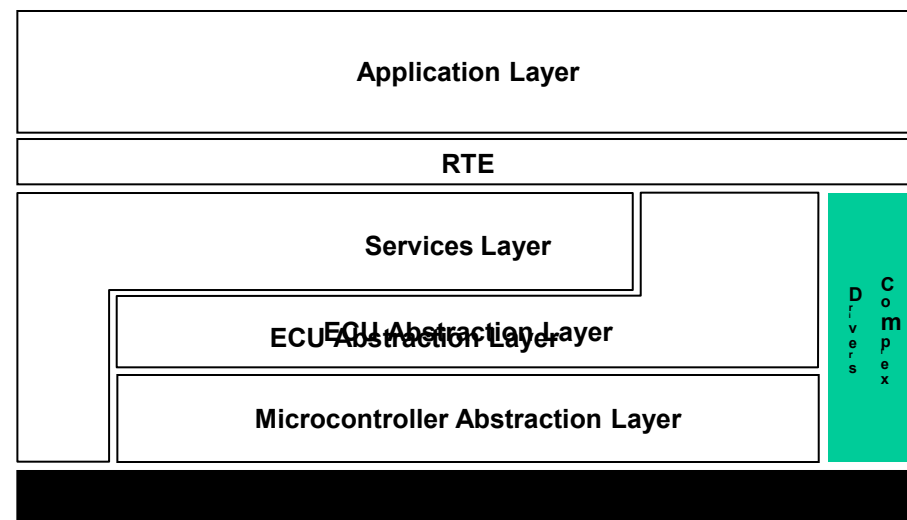
提供集成特殊功能的可能性，例如设备驱动程序：

未在AUTOSAR中明确规定的部分，具有极高时序约束的情况，或用于迁移目的等。

属性

实现：可能依赖于应用、 μ C和ECU硬件

上层接口：可能依赖于应用、 μ C和ECU硬件

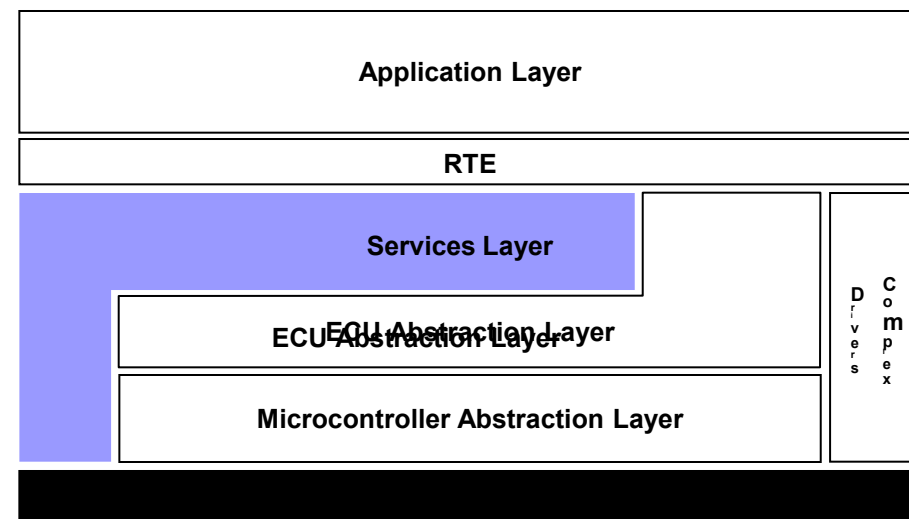


Architecture – Overview of Software Layers

服务层

服务层是基础软件的最高层，其重要性同样适用于应用软件：ECU抽象层负责处理对I/O信号的访问，而服务层则提供：

操作系统功能 车辆网络通信与管理服务 存储服务（NVRAM管理） 诊断服务（包括UDS通信、错误存储与故障处理） ECU状态管理、模式管理 逻辑与时间程序流监控（WdgManager）



任务

为应用程序、RTE和基础软件模块提供基本服务

。

属性

实现：主要与 μ C和ECU硬件无关 上层接口：与 μ C和ECU硬件无关

Architecture – Overview of Software Layers

AUTOSAR运行时环境 (RTE)

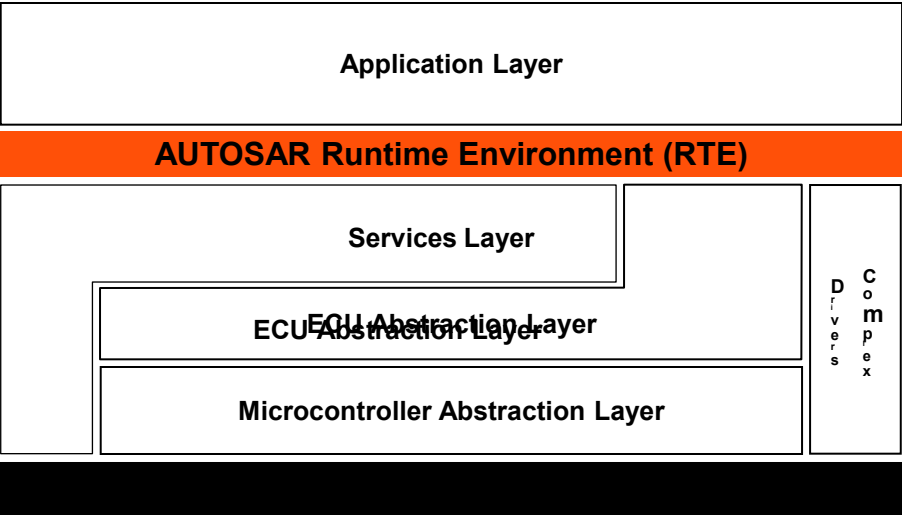
RTE是为应用软件（AUTOSAR软件组件和/或AUTOSAR传感器/执行器组件）提供通信服务的层。

在RTE之上，软件架构风格从“分层式”转变为“组件式”。

AUTOSAR软件组件通过RTE与其他组件（ECU间和/或ECU内）和/或服务进行通信。

任务
使AUTOSAR软件组件独立于特定ECU的映射。

属性
实现方式：ECU与应用特定（为每个ECU单独生成） 上层
接口：完全独立于ECU



Architecture – Overview of Software Layers

服务类型介绍

基本如此 软件可以细分为以下几种类型 服务：

输入/输出（I/O）对传感器、执行器和ECU车载外设的标准化访问

内存 对内部/外部存储器（非易失性存储器）的标准化访问

加密标准化访问加密原语，包括内部/外部硬件加速器

沟通

标准化访问：车辆网络系统、ECU车载通信系统及ECU内部软件

板外通信

标准化接入：车联万物通信、车载无线网络系统、ECU车外通信系统

系统

提供可标准化（操作系统、定时器、错误存储器）和ECU特定（ECU状态管理、看门狗管理器）的服务及库函数

Architecture – Introduction to Basic Software Module Types

驱动程序（内部）

驱动程序包含控制和访问内部或外部设备的功能。

内部设备位于微控制器内部。内部设备的例子有：

- 内部EEPROM

- 内部CAN控制器

- 内部ADC

内部设备的驱动程序称为内部驱动程序，位于微控制器抽象层。

Architecture – Introduction to Basic Software Module Types

驱动程序（外部）

外部设备位于微控制器之外的ECU硬件上。外部设备的例子有：

外部EEPROM

外部看门狗

外部闪存

外部设备的驱动程序称为外部驱动程序，位于ECU抽象层。它通过微控制器抽象层的驱动程序访问外部设备。

这种方式也支持集成在系统基础芯片（SBCs）中的组件，如收发器和看门狗，由AUTOSAR提供支持。

示例：一个带有SPI接口的外部EEPROM驱动程序通过SPI总线处理程序/驱动程序访问外部EEPROM。

异常：

用于内存映射外部设备（例如外部闪存）的驱动程序可以直接访问微控制器。这些外部驱动程序位于微控制器抽象层中，因为它们依赖于特定的微控制器。

接口

接口（接口模块）包含对架构上位于其下方的模块进行抽象的功能。例如，一个接口模块可以对特定设备的硬件实现进行抽象。它提供一个通用API来访问特定类型的设备，该API独立于该类型现有设备的数量，也独立于不同设备的硬件实现。

接口不会改变数据的内容。

通常，接口位于ECU抽象层中。

示例：CAN通信系统的接口提供了一个通用API，用于访问CAN通信网络，该接口独立于ECU内CAN控制器的数量，也独立于硬件实现方式（片上或片外）。

处理器是一种特定接口，用于控制一个或多个客户端对一个或多个驱动器的并发、多路和异步访问。即它执行缓冲、排队、仲裁和多路复用功能。

处理程序不会改变数据的内容。

处理程序功能通常集成在驱动程序或接口中（例如SPIHandlerDriver、ADC Driver）。

经理为多个客户提供特定服务。在所有纯粹的处理程序功能不足以从多个客户中抽象出来的情况下，都需要它。

除了处理程序功能外，管理器还可以评估、更改或调整数据的内容。

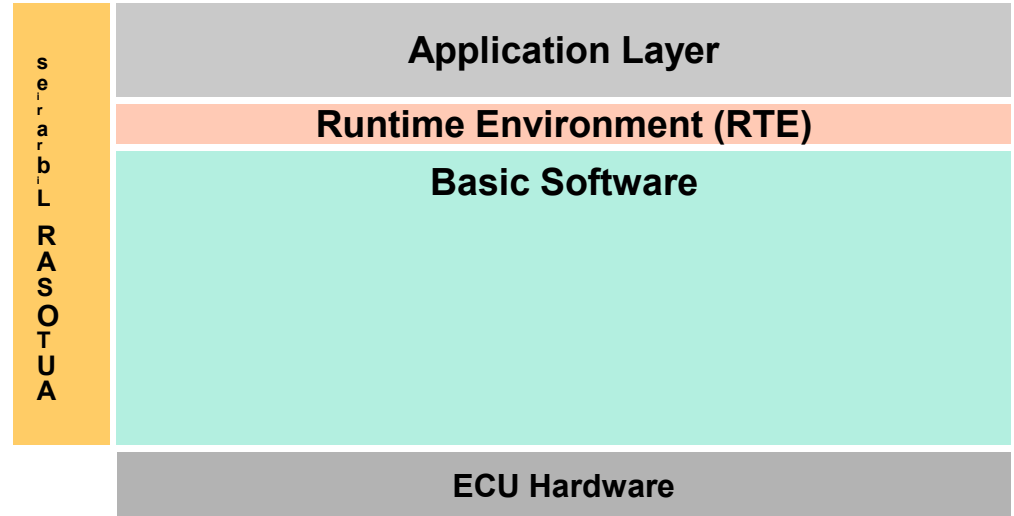
通常，管理者位于服务层

示例：NVRAM管理器管理对内部和/或外部存储设备（如闪存和EEPROM）的并发访问。它还执行分布式可靠数据存储、数据检查、提供默认值等功能。

库是用于相关目的函数集合

库:

可由BSW模块（包括RTE）、SW-C、库或集成代码调用 在调用者的上下文中运行，处于相同的保护环境
仅能调用库，不允许调用BSW模块 可重入 无内部状态 无需任何初始化 同步执行，即无等待点
可配置，但不建议使用配置



AUTOSAR中规定了以下库：

定点数学运算 (Mfx)， 浮点数学运算 (Mfl)， 定点数据插值 (Ifx)， 浮点数据插值 (IfI)， 扩展功能（例如64位计算、滤波等） (Efx)， 位处理 (Bfx)， E2E通信 (E2E)， CRC计算 (Crc)， 原子多核安全操作 (Bmc)， 内存标准函数 (Msf)

Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

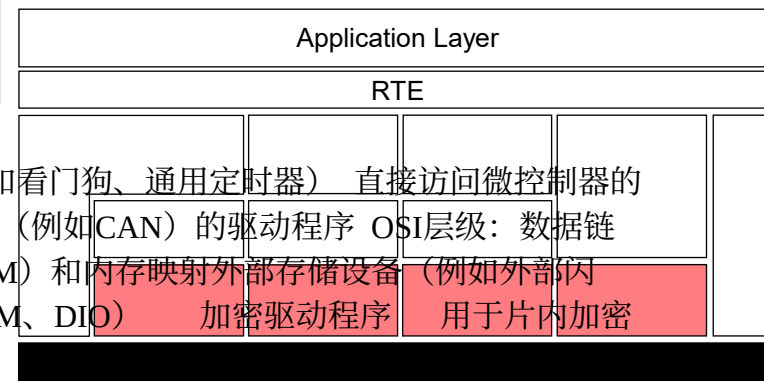
2. 配置

3. 集成与运行时方面

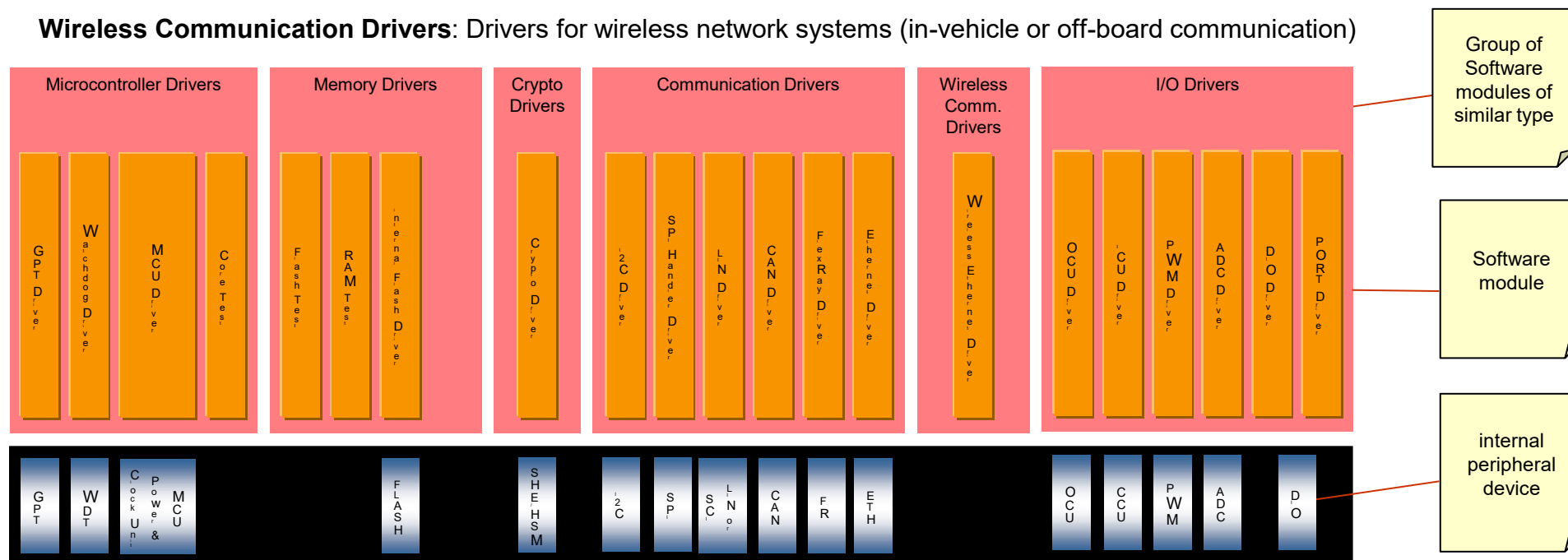
Architecture – Content of Software Layers

微控制器抽象层

微控制器抽象层包含以下模块组： 微控制器驱动程序 内部外设驱动程序（例如看门狗、通用定时器） 直接访问微控制器的功能（例如核心测试） 通信驱动程序 用于ECU板载通信（例如SPI）和车辆通信（例如CAN）的驱动程序 OSI层级：数据链路层的一部分 存储器驱动程序 用于片内存储设备（例如内部闪存、内部EEPROM）和内存映射外部存储设备（例如外部闪存）的驱动程序 I/O驱动程序： 用于模拟和数字I/O的驱动程序（例如ADC、PWM、DIO） 加密驱动程序 用于片内加密设备（如SHE或HSM）的驱动程序



Wireless Communication Drivers: Drivers for wireless network systems (in-vehicle or off-board communication)



Architecture – Content of Software Layers

微控制器抽象层：I2C 驱动程序

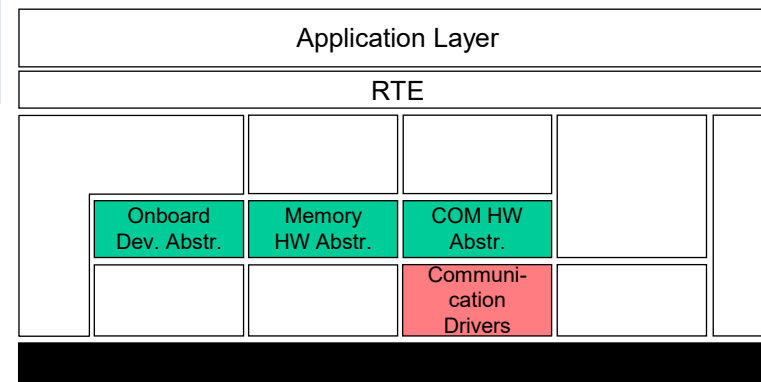
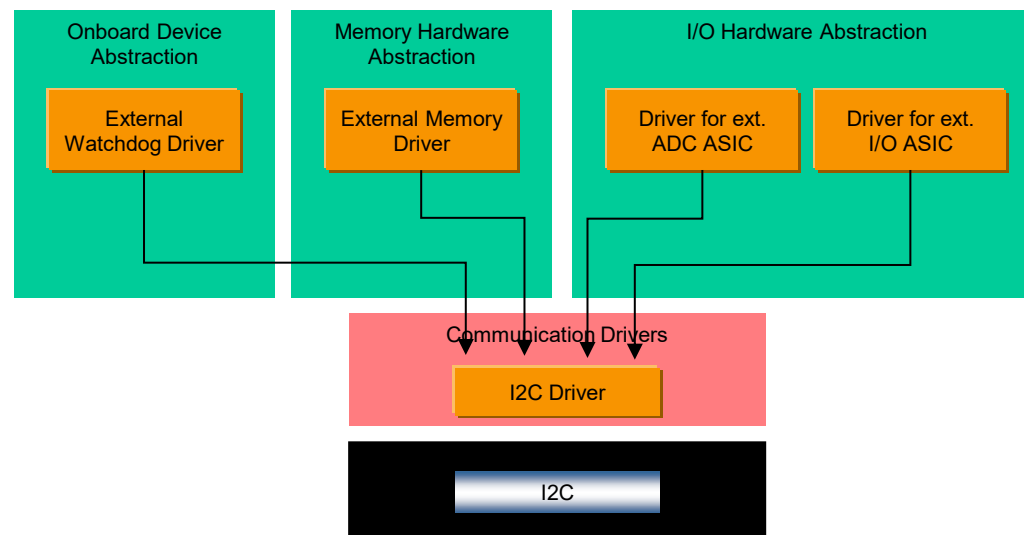
I2C驱动程序允许多个节点同时访问一个或多个I2C总线。

I²C（Inter-Integrated Circuit）是一种广泛应用于汽车传感器或执行器的2线串行数据总线。

示例：

- 温度传感器
- 三轴加速度传感器
- 气压传感器
- 电可擦可编程只读存储器
- ...

示例：

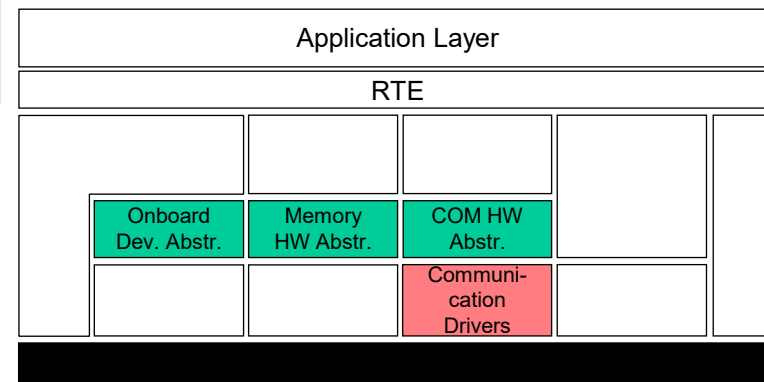


Architecture – Content of Software Layers

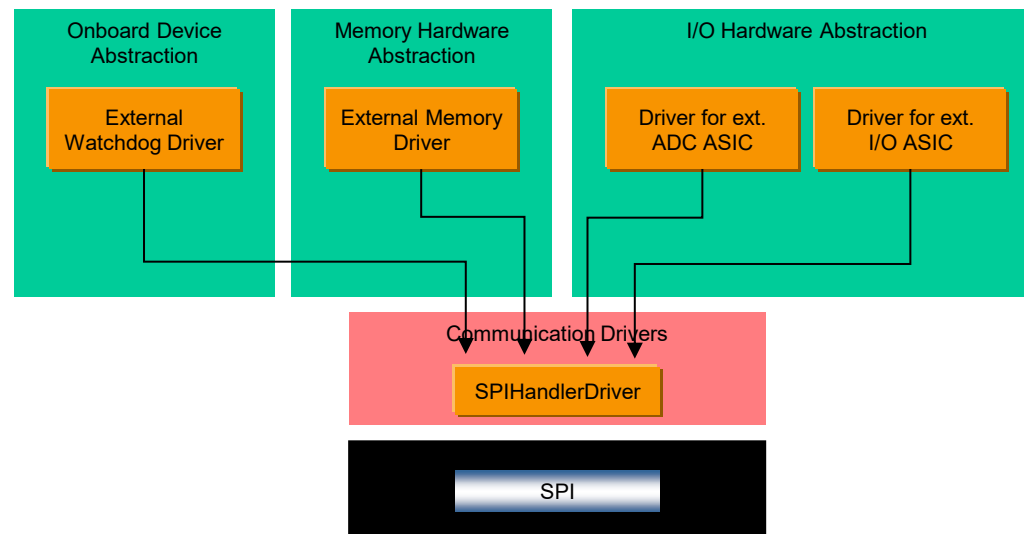
微控制器抽象层：SPIHandlerDriver

SPIHandlerDriver允许多个客户端并发访问一个或多个SPI总线。

为了抽象SPI微控制器中专门用于片选引脚的所有特性，这些引脚应直接由SPIHandlerDriver处理。这意味着这些引脚不应在DIO驱动器中可用。



示例：



Architecture – Content of Software Layers

复杂驱动因素

复杂驱动程序是在基础软件栈内实现非标准化功能的模块。

一个例子是使用特定的中断和/或复杂的微控制器外设（如PCP、TPU）直接访问μC，来实现复杂的传感器评估和执行器控制。

注入控制

电动阀门控制

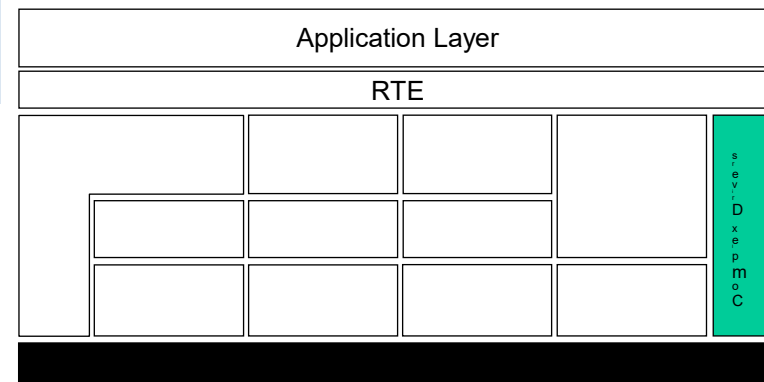
增量位置检测

任务：

满足处理复杂传感器和执行器的特殊功能和时序要求

属性：

实现：高度依赖于微控制器、ECU和应用程序 上层接口至软件组件：根据AUTOSAR规范指定与实现（AUTOSAR接口） 下层接口：对标准化接口的受限访问



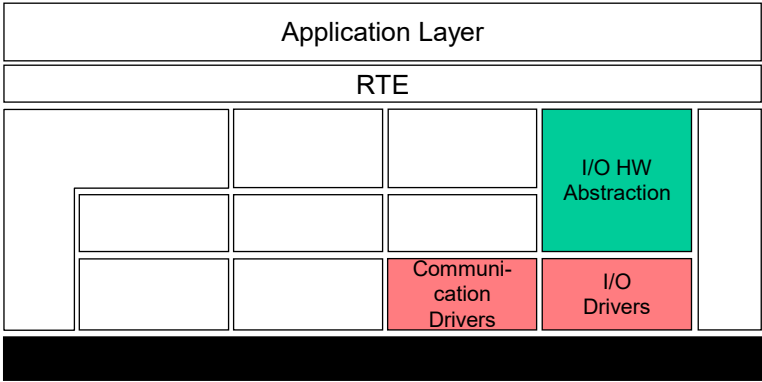
示例：



Architecture – Content of Software Layers

ECU抽象化: I/O Hardware Abstraction

I/O硬件抽象是一组模块，它从外围I/O设备（片上或板上）的**location**和**ECU hardware layout**（例如微控制器引脚连接和信号电平反转）中抽象出来。I/O硬件抽象不对传感器/执行器进行抽象！



不同的I/O设备可能通过I/O信号接口进行访问。

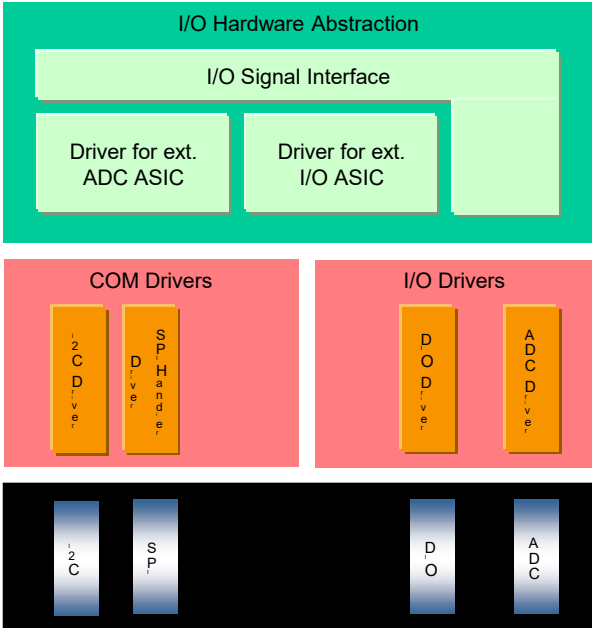
任务：

将I/O信号表示为它们连接到ECU硬件的方式（例如电流、电压、频率）。向更高层的软件层隐藏ECU硬件和布局属性。

属性：

实现：μC独立，ECU硬件依赖 上层接口μC与ECU硬件独立，依赖于指定信号类型，并根据AUTOSAR（AUTOSAR接口）实现

示例：



Architecture – Content of Software Layers

ECU抽象化: *Communication Hardware Abstraction*

通信硬件抽象是一组模块，它从通信控制器的**location**和**ECU hardware layout**中抽象出来。对于所有通信系统，都需要一个**specific**通信硬件抽象（例如LIN、CAN、FlexRay）。

示例：一个ECU拥有一个带有2个内部CAN通道的微控制器，以及一个带有4个CAN控制器的额外板载ASIC。该CAN-ASIC通过SPI与微控制器连接。

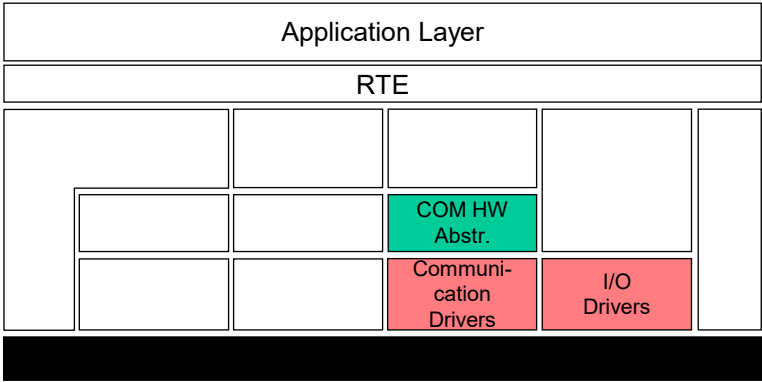
通信驱动程序通过总线特定接口（例如CAN接口）进行访问。

任务：

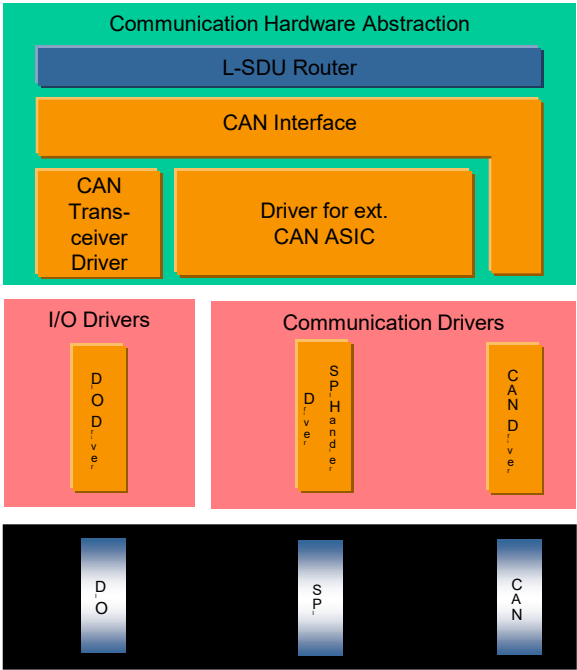
无论其位置（片上/板上）如何，都提供平等的机制来访问总线通道。

属性：

实现：μC独立，ECU硬件依赖及外部设备依赖
 上层接口：总线依赖，μC与ECU硬件独立



示例：



Architecture – Content of Software Layers

范围：内存硬件抽象

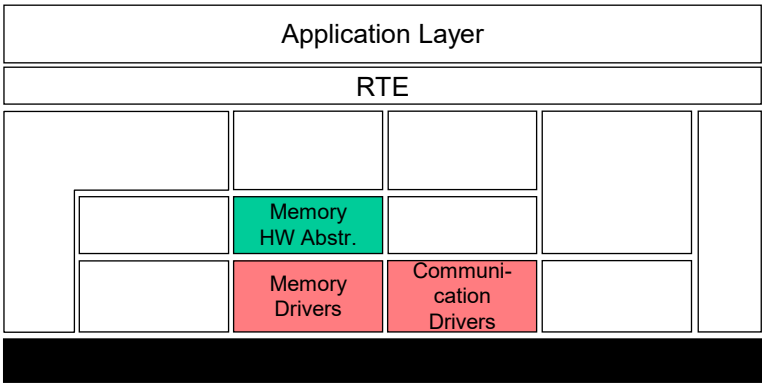
内存硬件抽象是一组模块，它从外围存储设备（片上或板上）的**location**和**ECU hardware layout**中抽象出来。

示例：片上EEPROM和外部EEPROM设备可通过同一机制访问。

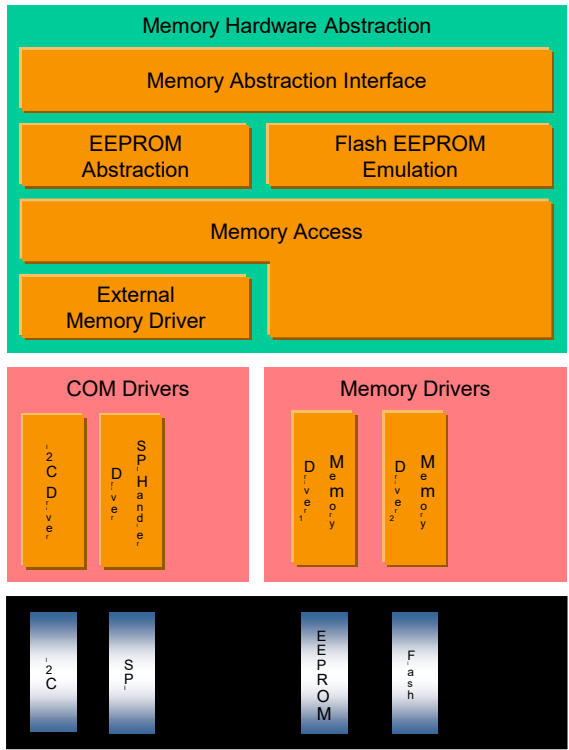
内存驱动程序通过内存特定的抽象/仿真模块（例如EEPROM抽象）进行访问。通过在Flash硬件单元之上仿真EEPROM抽象，实现了通过内存抽象接口对两种硬件类型的统一访问。

任务：
提供访问内部（片上）和外部（板上）存储设备以及存储器硬件类型（EEPROM、Flash）的平等机制。

属性：
实现：μC独立，外部设备依赖 上层接口μC、ECU硬件和存储设备独立



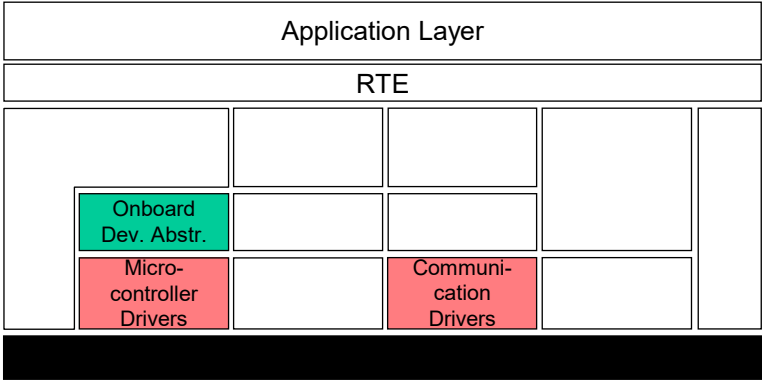
示例：



Architecture – Content of Software Layers

板载设备抽象

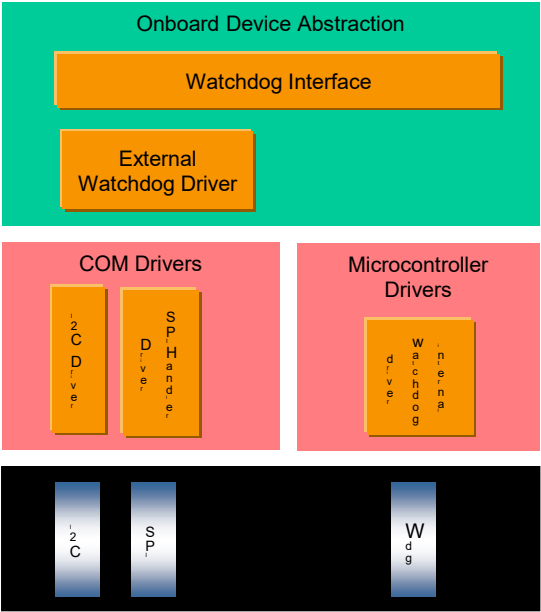
板载设备抽象层包含用于ECU板载设备的驱动程序，这些设备不能被视为传感器或执行器，例如内部或外部看门狗。这些驱动程序通过μC抽象层访问ECU板载设备。



任务：
来自ECU特定车载设备的摘要。

属性：
实现：μC独立，外部设备依赖 上层接口：μC独立，部分依赖ECU硬件

示例：



Architecture – Content of Software Layers

范围：加密硬件抽象

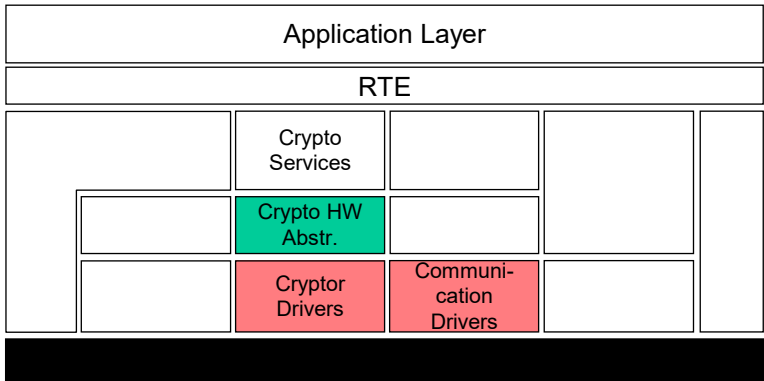
加密硬件抽象是一组模块，它从密码原语（基于内部或外部硬件或软件）的 **location** 中抽象出来。

示例：AES原语在SHE中实现或作为软件库提供

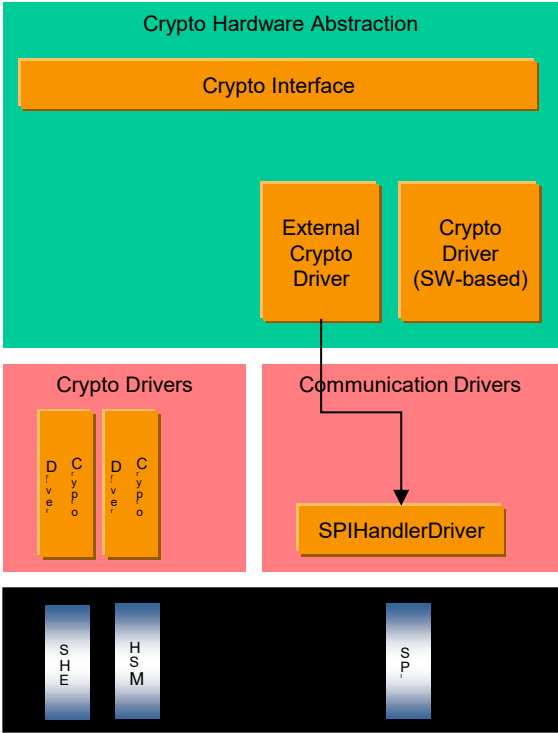
任务：
为访问内部（片上）和软件加密设备提供平等的机制。

属性：
实现：μC 独立

上层接口：独立于μC、ECU硬件和加密设备



示例：

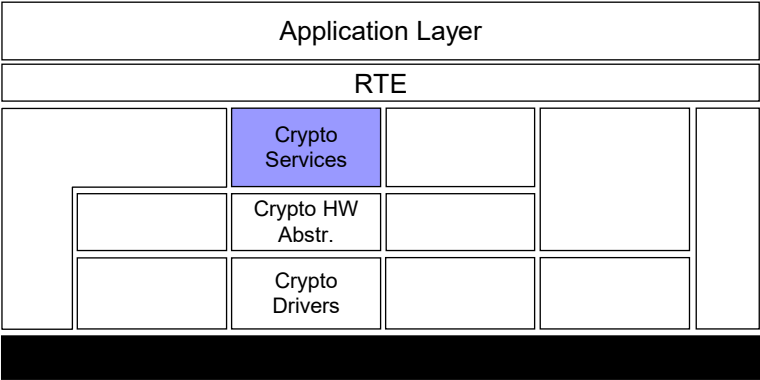


Architecture – Content of Software Layers

服务：加密服务

加密服务由三个模块组成

加密服务管理器负责管理加密作业 密钥管理器与密钥配置主控（位于NVM或加密驱动中）交互，并管理证书链的存储与验证 入侵检测系统管理器负责处理由BSW模块或SW-C报告的安全事件



任务：

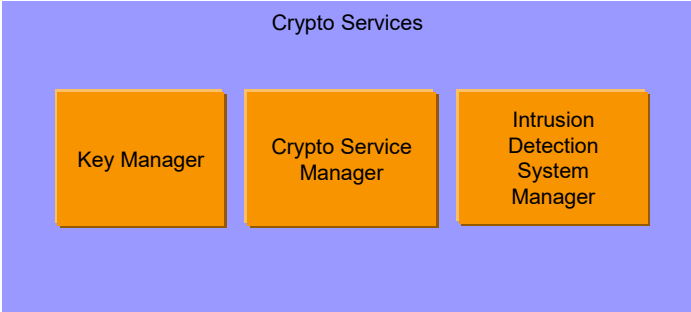
以统一的方式为应用程序提供加密原语、IDS服务和密钥存储。抽象于硬件设备和属性之外。

属性：

实现：μC与ECU硬件无关，高度可配置 上层接口μC与ECU硬件无关，依据AUTOSAR规范制定与实现

(AUTOSAR接口)

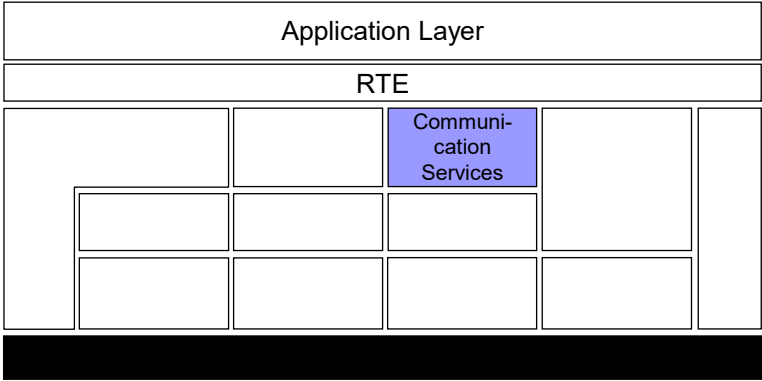
示例：



Architecture – Content of Software Layers

通信服务 - 通用

通信服务是一组用于车辆网络通信（CAN、LIN、FlexRay和以太网）的模块。它们通过通信硬件抽象层与通信驱动程序进行交互。



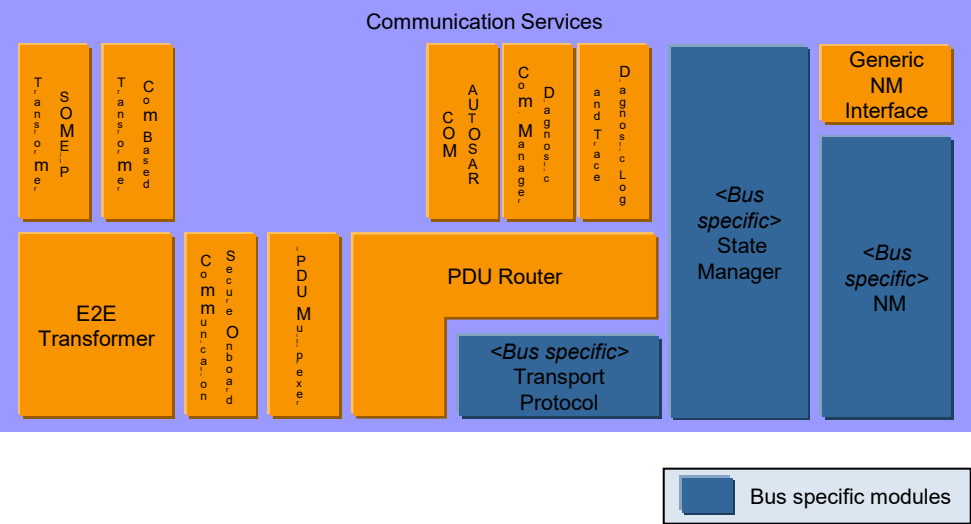
任务：
 为车辆网络通信提供统一接口。 为网络管理提供统一服务。
 为车辆网络提供统一接口。

诊断通信
 对应用程序隐藏协议和消息属性。

属性：
 实现：μC和ECU硬件独立，部分依赖于总线类型
 上层接口：独立于μC、ECU硬件和总线类型

相关车辆网络系统的通信服务将在以下页面中详细说明。

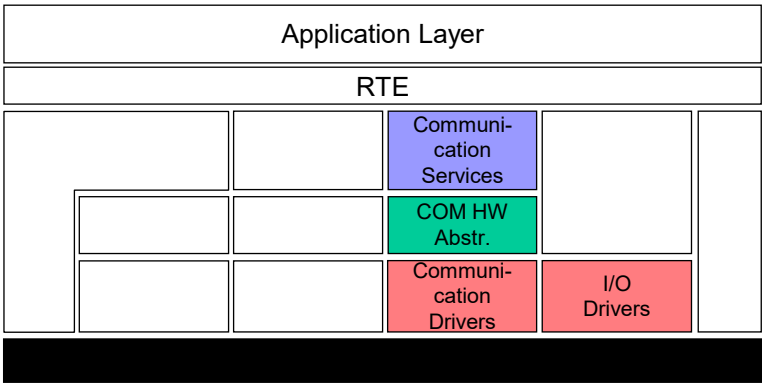
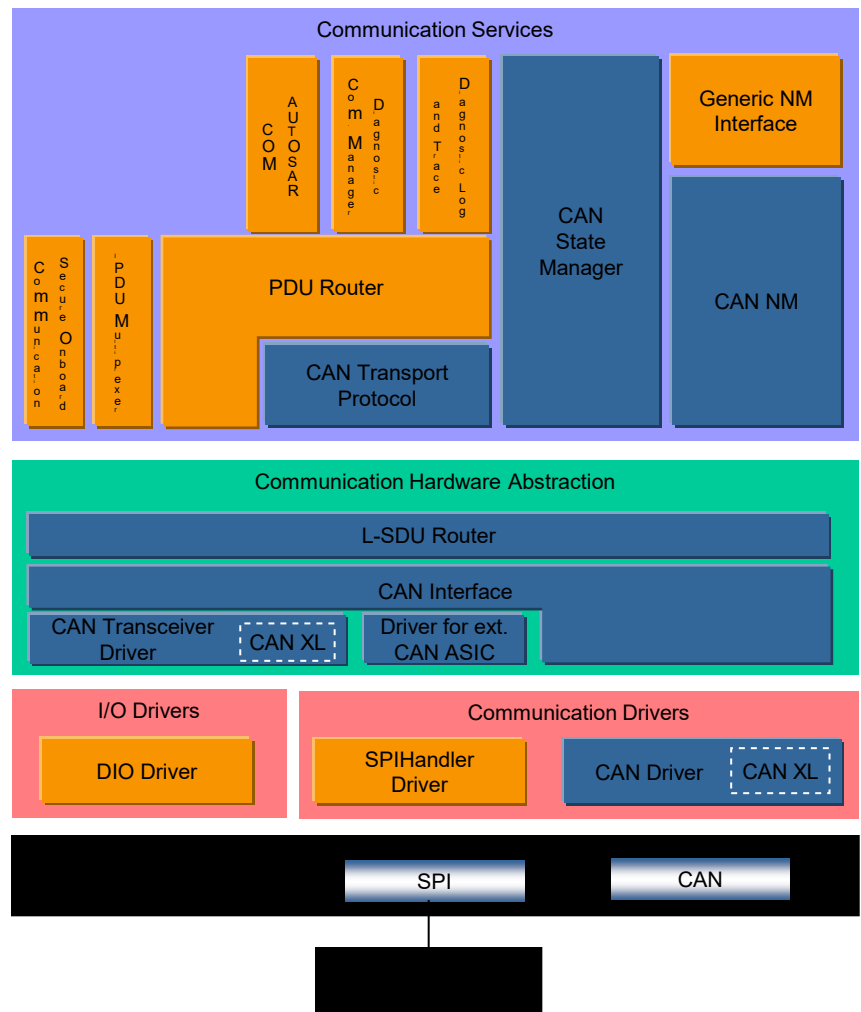
示例：



Architecture – Content of Software Layers

通信栈 – CAN

示例:



CAN通信服务是一组用于车辆网络通信的模块，与通信系统CAN配合使用。

任务:

为CAN网络提供统一的接口。向应用程序隐藏协议和消息属性。

CAN通信栈支持:

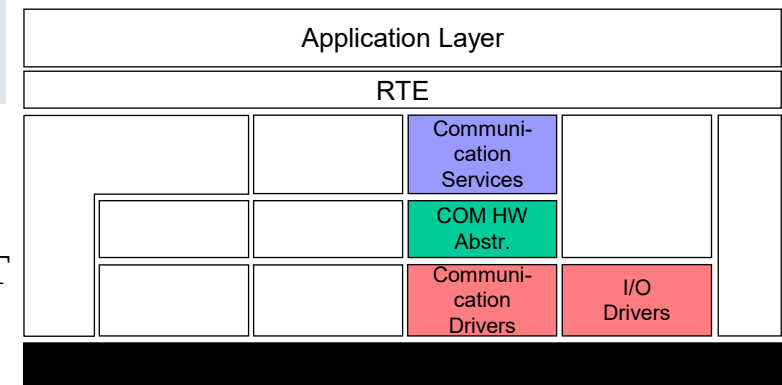
- 经典CAN通信 (CAN 2.0)
- CAN FD通信, 如果硬件支持
- CAN XL通信, 如果硬件支持

Architecture – Content of Software Layers

通信栈 – CAN

属性:

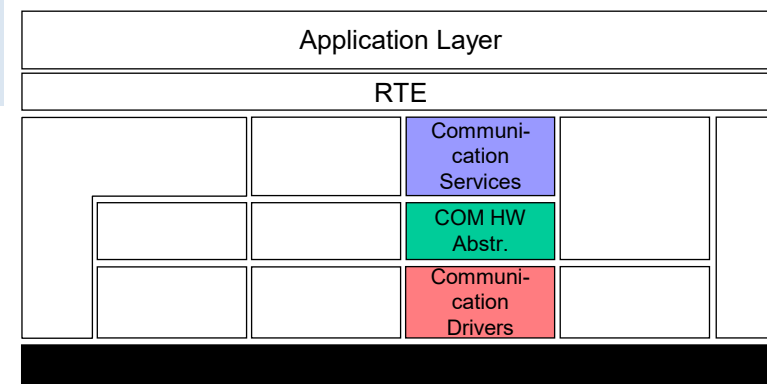
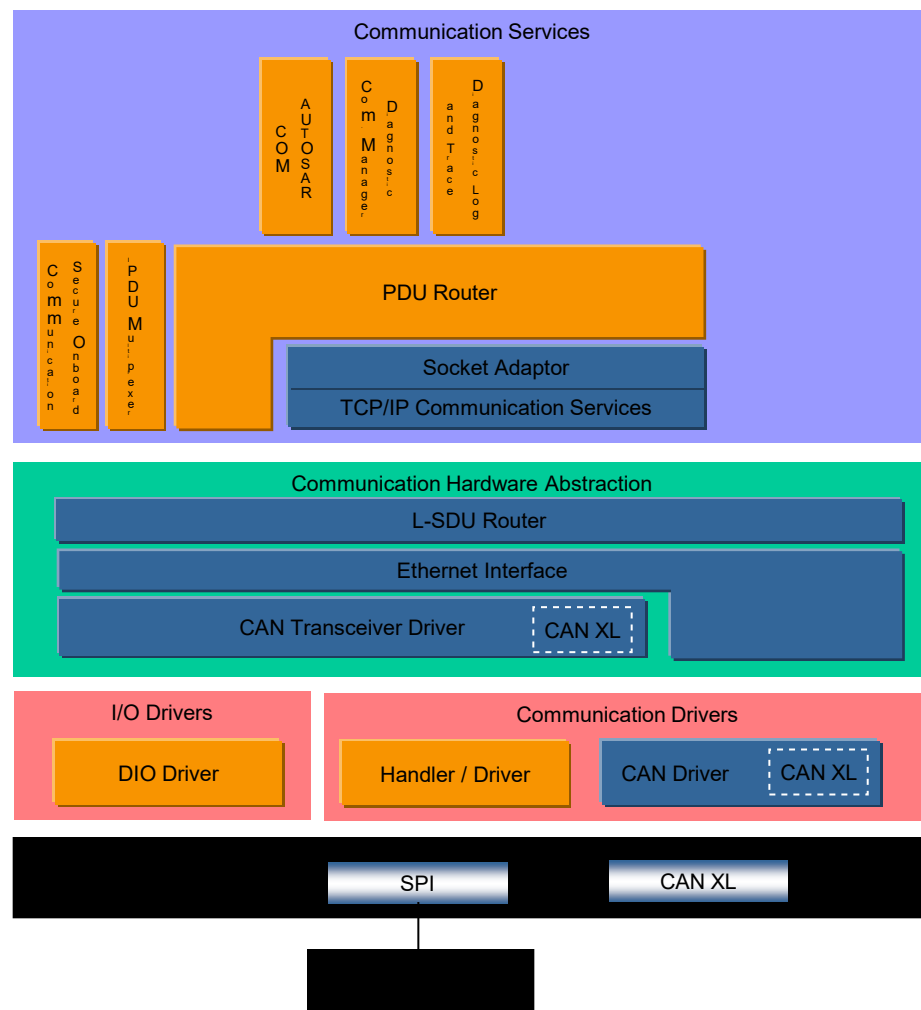
实现: 微控制器与ECU硬件无关, 部分依赖于CAN。 AUTOSAR COM、通用网络管理接口及诊断通信管理器在所有车辆网络系统中保持一致, 每个ECU仅存在一个实例。 通用网络管理接口仅包含调度器, 不集成其他功能。在网关ECU中, 该接口还可集成网络管理协调器功能, 用于同步多个不同网络 (同类型或不同类型) 的唤醒与关闭操作。 CAN网络管理专用于CAN网络, 每个CAN车辆网络系统将独立实例化。 通信系统特定的CAN状态管理器负责处理与通信系统相关的启动和关闭功能, 同时控制COM发送协议数据单元及监控信号超时的不同选项。



Architecture – Content of Software Layers

通信栈 – 以太网/CAN XL

示例:



CAN XL支持直接隧道传输IEEE 802.3以太网帧，以参与IP通信。

任务:

提供车辆范围内的通信，无论物理连接（CAN XL / 以太网）或通信范式（基于信号和服务的通信）如何，均使用统一的语义。

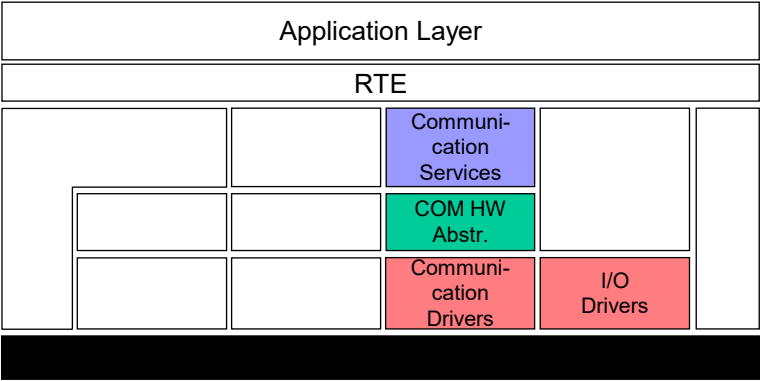
Architecture – Content of Software Layers

通信栈扩展 – CAN XL

属性:

CAN XL 是 CAN 的绝对超集，即支持 CAN XL 的 CAN 协议栈可同时服务于 CAN 和 CAN XL。

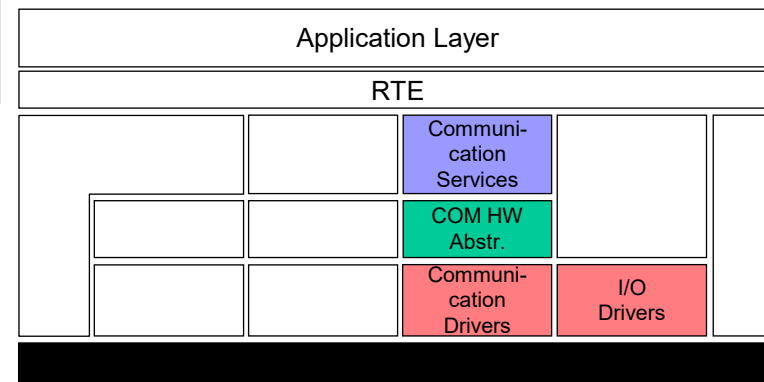
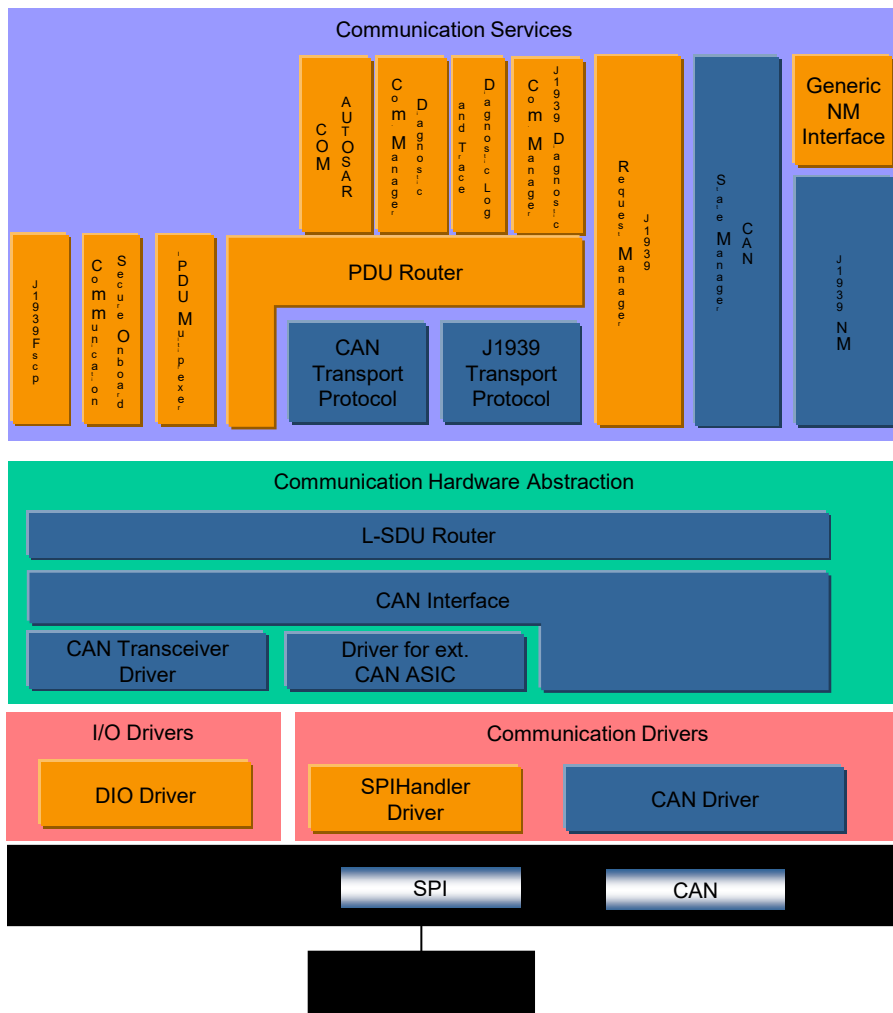
XL总线。 CanIf、CanTrcvDrv和CanDrv是唯一需要扩展以支持CAN XL通信的模块。 通信栈CAN的特性同样适用于具备CAN XL功能的CAN。



Architecture – Content of Software Layers

通信栈扩展 – J1939

示例:



J1939通信服务扩展了重型车辆网络通信中普通CAN通信栈的功能。

任务:

提供J1939所需的协议服务。在不必要的情况下，向应用程序隐藏协议和消息属性。

请注意:

CAN堆栈中有两个传输协议模块（CanTp和J1939Tp），可在不同通道上交替或并行使用：其使用方式如下：

- CanTp: ISO诊断（DCM），标准CAN总线上的大型PDU传输
- J1939Tp: J1939诊断，J1939驱动的CAN总线上的大型PDU传输

Architecture – Content of Software Layers

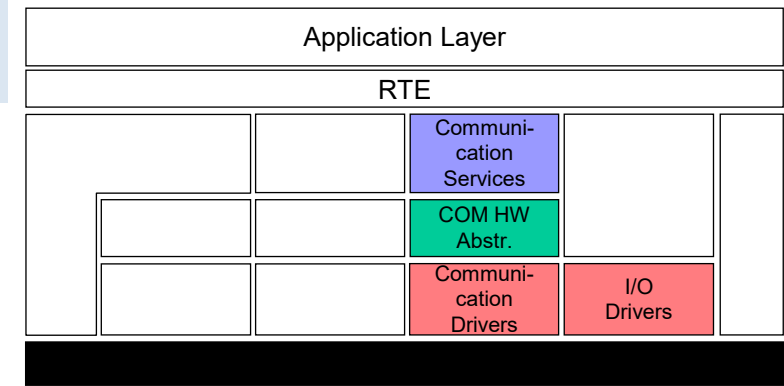
通信栈扩展 – J1939

属性:

实现: 基于CAN, 与 μ C和ECU硬件无关。

AUTOSAR COM、通用NM（网络管理）接口以及诊断通信管理器对所有车辆网络系统均相同, 且每个ECU中仅存在一个实例。支持配置时未知的动态帧标识符。J1939网络管理负责为每个ECU分配唯一地址, 但不支持休眠/唤醒处理及相关概念（如部分网络）。提供J1939诊断与请求处理功能。

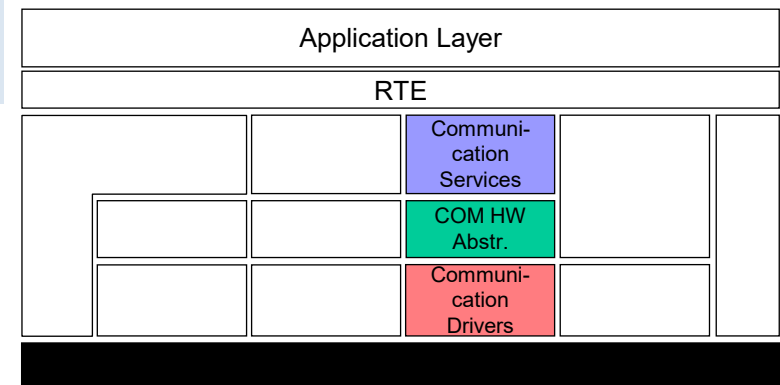
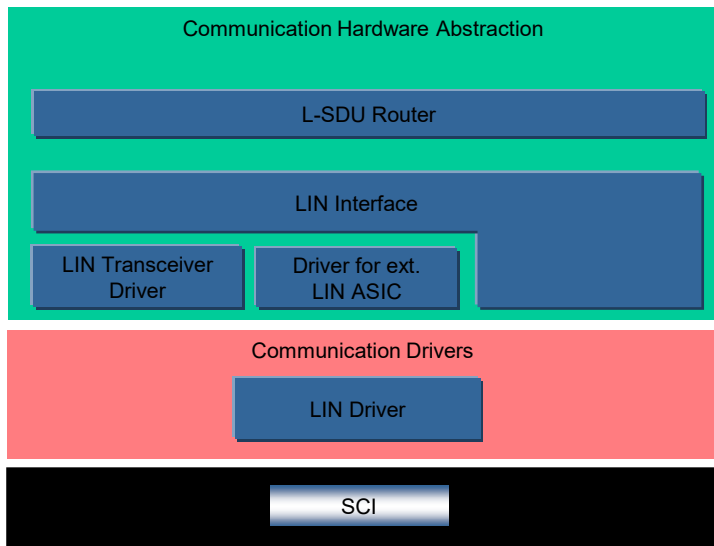
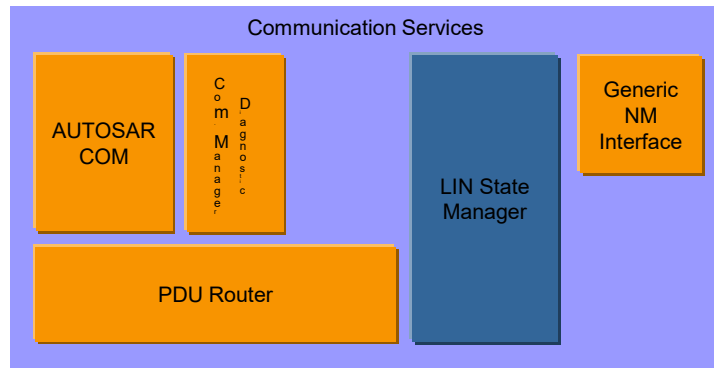
支持符合SAE J1939-76标准的功能安全, 包括初始化、安全数据组（SDG）操作、使用PROFILE_J1939_76 E2E转换器处理安全报头/数据消息, 以及生产者与消费者应用软件组件之间的错误管理。



Architecture – Content of Software Layers

通信栈 – LIN

示例:



LIN通信服务是一组用于车辆网络与LIN通信系统进行通信的模块。

任务:

为LIN网络提供统一的接口。向应用程序隐藏协议和消息属性。

属性:

LIN通信服务包含:

符合ISO 17987标准的通信协议栈，具备

- 调度表管理器，用于处理切换到其他调度表的请求（适用于LIN主节点）
- 不同LIN帧类型的通信处理
- 传输协议，用于诊断
- 唤醒与休眠接口

底层LIN驱动程序： ■ 实现LIN协议并访问特定硬件 ■ 同时支持基于简单UART和复杂帧的LIN硬件

Architecture – Content of Software Layers

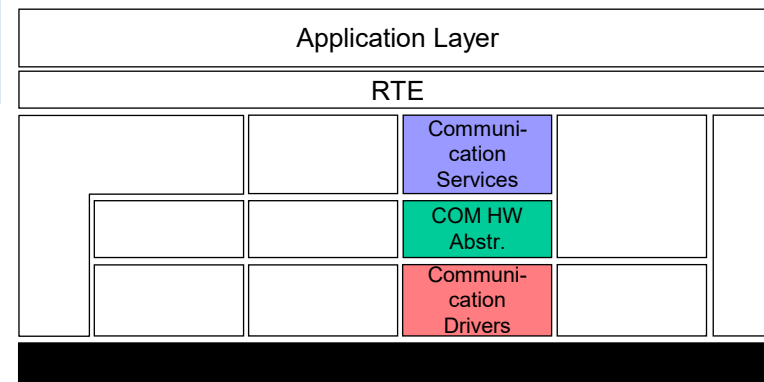
通信栈 – LIN

注意：LIN集成到AUTOSAR中：

LIN接口控制着WakeUp/Sleep API，并允许从节点保持总线唤醒状态（分布式方案）。

通信系统特定的LIN状态管理器负责处理与通信相关的启动和关闭功能。此外，它还控制来自通信管理器的通信模式请求。LIN状态管理器还通过接口COM控制I-PDU组。

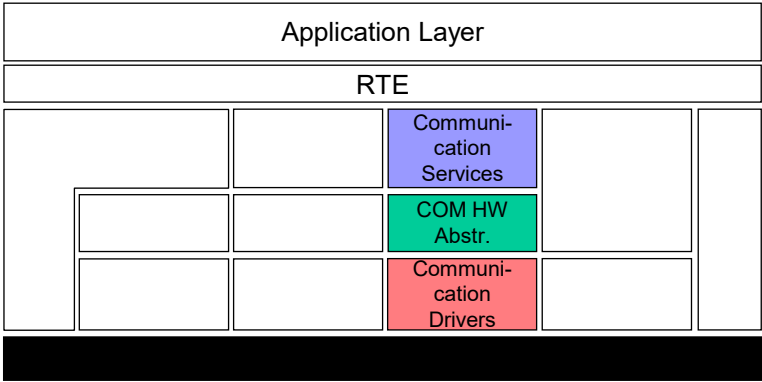
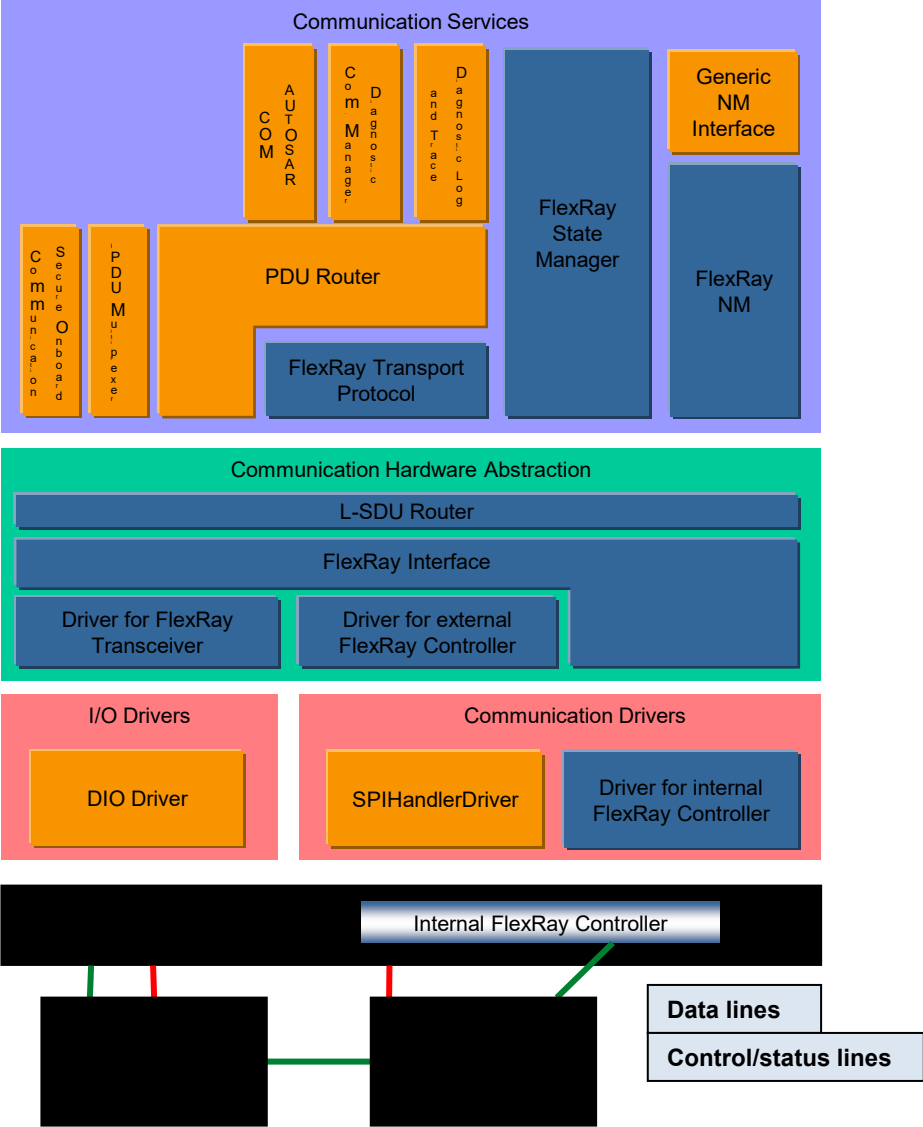
发送LIN帧时，LIN接口在需要数据（即即将发送LIN帧之前）的时刻向PDU路由器请求该帧（I-PDU）的数据。



Architecture – Content of Software Layers

通信栈 – FlexRay

示例:



FlexRay通信服务是一组用于车辆网络通信的模块，与FlexRay通信系统协同工作。

任务:

为FlexRay网络提供统一接口，向应用程序隐藏协议与消息属性。

请注意:

FlexRay堆栈中有两个传输协议模块，可以交替使用。

- FrTp: FlexRay ISO传输层
- FrArTp: FlexRay AUTOSAR传输层，为AUTOSAR R3.x提供总线兼容性

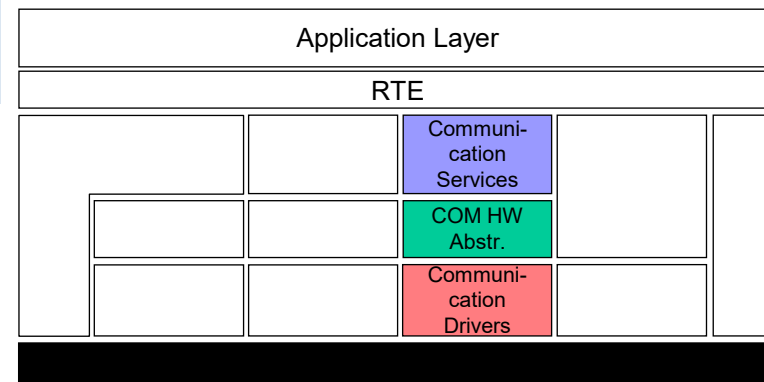
Architecture – Content of Software Layers

通信栈 – FlexRay

属性：

实现：微控制器与ECU硬件无关，部分依赖于FlexRay。AUTOSAR COM、通用网络管理接口及诊断通信管理器对所有车载网络系统均相同，且每个ECU仅存在一个实例。通用网络管理接口仅包含调度器，不包含其他功能。在网关ECU中，该接口由网络管理协调器替代，后者额外提供同步多个不同网络（相同或不同类型）以唤醒或关闭的功能。FlexRay网络管理专用于FlexRay网络，每个FlexRay车载网络系统均会实例化该模块。

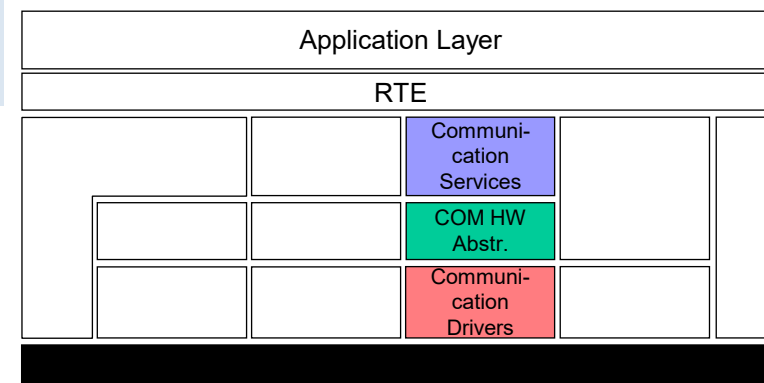
通信系统特定的FlexRay状态管理器负责处理依赖于通信系统的启动与关闭功能，同时控制COM发送协议数据单元及监控信号超时的不同选项。



The diagram illustrates the Network Management Architecture, showing the interaction between various components. The components are organized into layers and functional groups:

- Communication Services (Top Layer):**
 - Generic NM Interface:** The primary interface for network management.
 - UDP NM:** User Datagram Protocol Network Management.
 - Socket Adaptor:** Facilitates communication between the Generic NM Interface and the TCP/IP Communication Services.
 - TCP/IP Communication Services:** The underlying network protocol stack.
- Core Components:**
 - PDU Router:** The central component responsible for packet forwarding and routing.
 - COM and AUTOSAR:** Communication and Automotive Open System Architecture interfaces.
 - COM Manager and Diagnostic Log:** Components for managing communication and logging diagnostic data.
- External Interface:**
 - Ethernet State Manager:** Manages the state of the Ethernet network.

The diagram shows how the Generic NM Interface interacts with the UDP NM, Socket Adaptor, and Ethernet State Manager. The Socket Adaptor connects to the TCP/IP Communication Services, which in turn interacts with the PDU Router. The PDU Router is connected to the COM and AUTOSAR, COM Manager, and Diagnostic Log. The Ethernet State Manager is connected to the Generic NM Interface and the Ethernet State Manager.



任务：

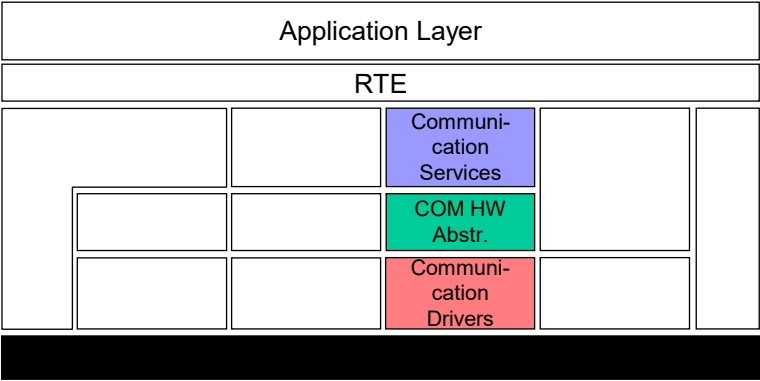
为TCP/IP网络提供统一的接口。向应用程序隐藏协议和消息属性。

Architecture – Content of Software Layers

通信栈 – TCP/IP

属性:

TcpIp模块实现了TCP/IP协议族（TCP、UDP、IPv4、IPv6、ARP、ICMP、DHCP）的主要协议，并通过以太网提供基于套接字的动态通信。套接字适配器模块（SoAd）是TcpIp模块唯一的上层模块。



Architecture – Content of Software Layers

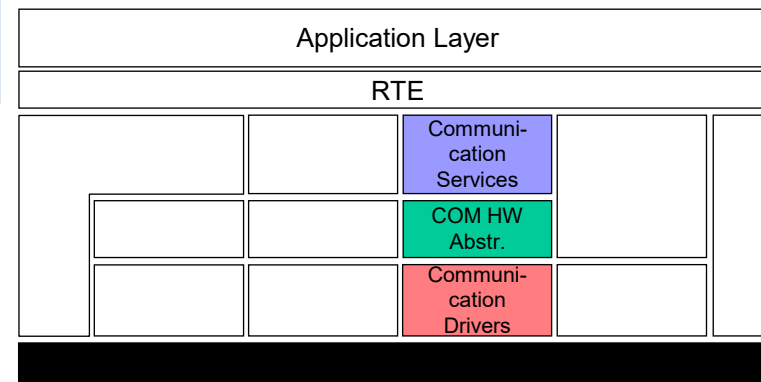
通信栈 – 防火墙

防火墙模块通过检查网络数据包并根据预定义的规则集进行过滤，保护AUTOSAR堆栈免受恶意消息的侵害。

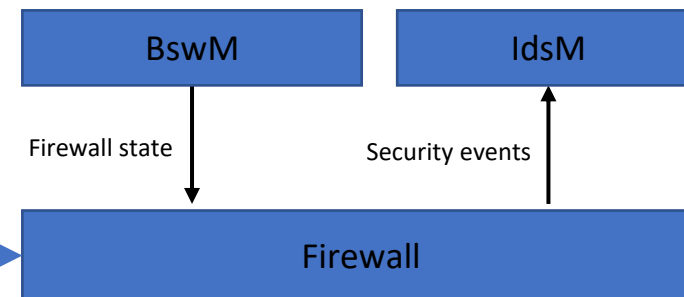
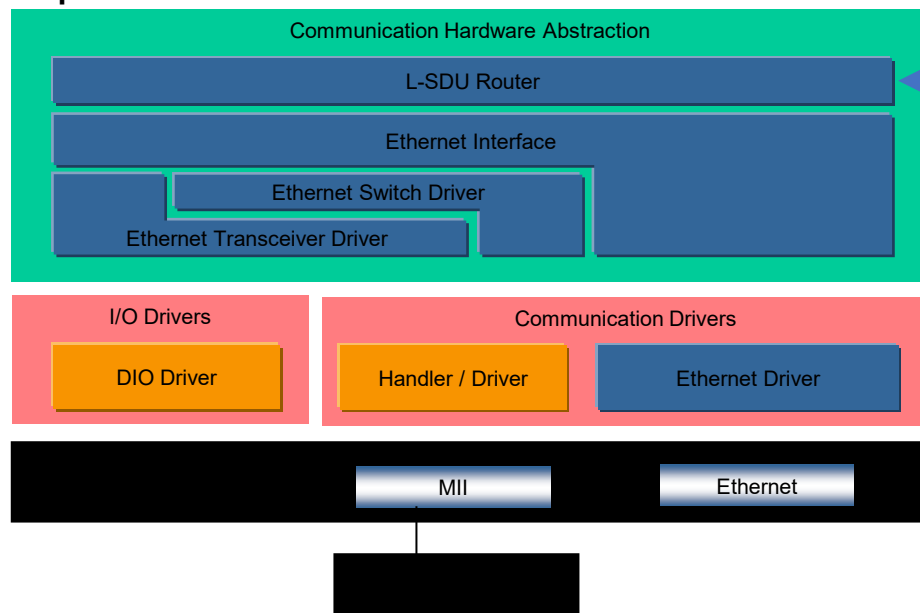
防火墙支持在三个不同级别进行网络数据包检查。

无状态数据包检查 有状态数据包检查 深度数据包检查

防火墙与IdsM模块相连，以便在出现意外网络数据包时提升安全事件。



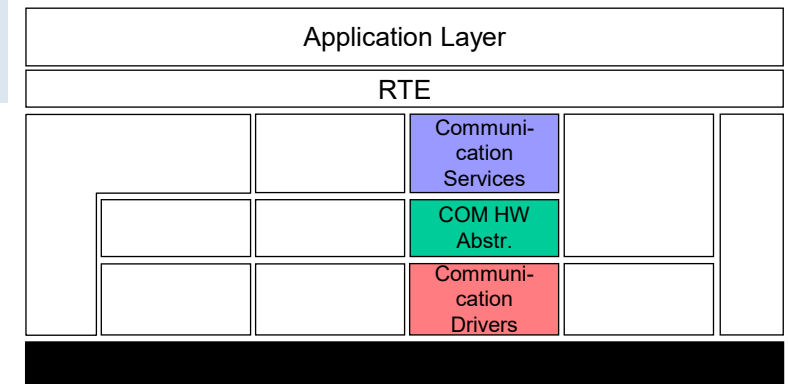
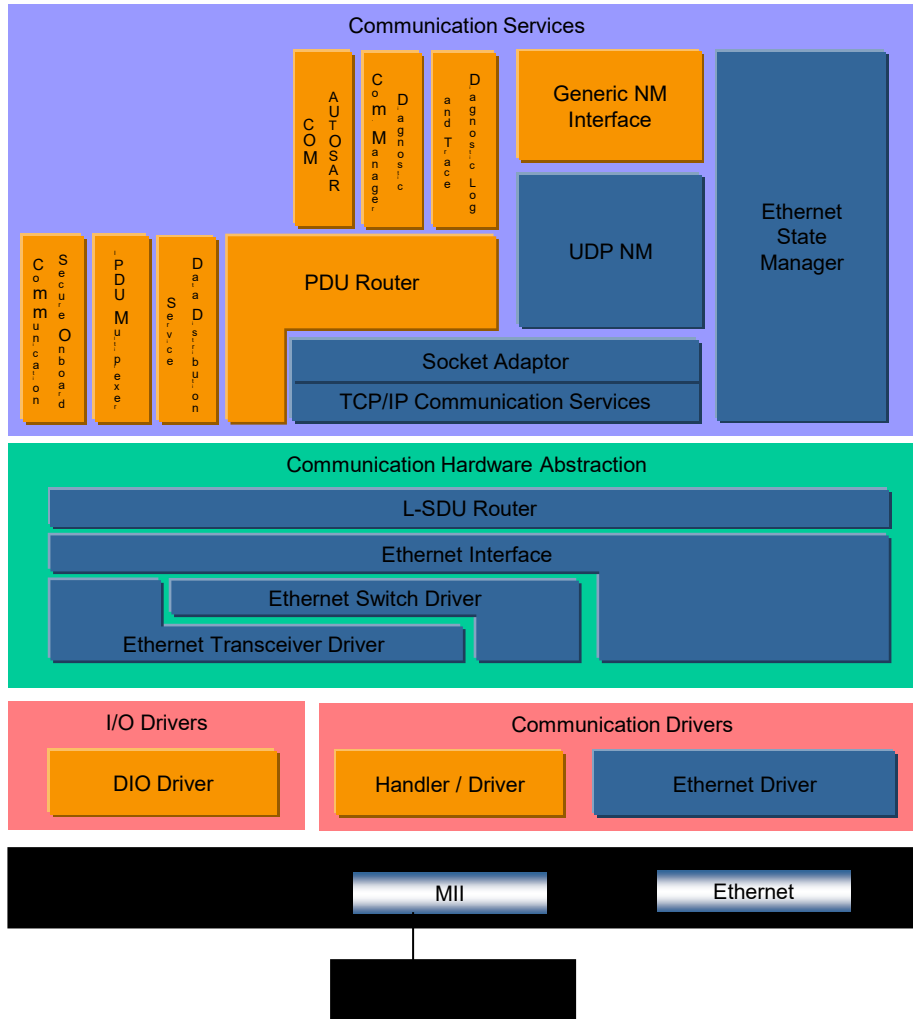
Example:



Architecture – Content of Software Layers

通信栈 – DDS

示例:



数据分发服务是一个面向数据和服务导向的车联网通信模块。

任务:

提供DDS标准接口。

DDS模块支持:

信号基础发布者/订阅者通信路径
服务质量处理 全静态配置

Architecture – Content of Software Layers

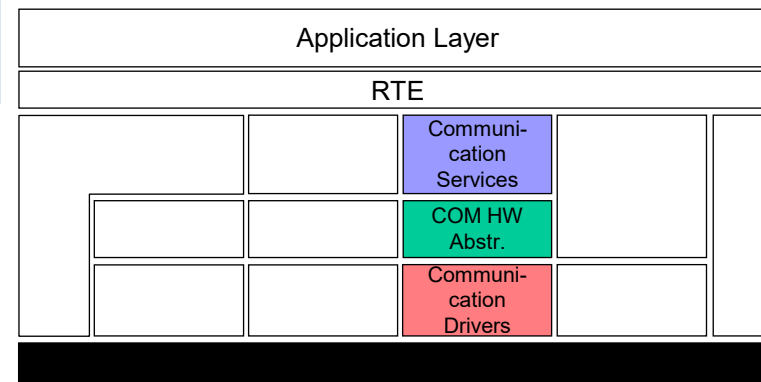
通信栈 – DDS

属性:

DDS模块支持对象管理组（OMG）DDS标准的核心特性，包括SPDP与SEDP发现协议、基于DDS-RPC的扩展型SOA架构，以及符合AUTOSAR AP规范的服务发现机制。

Socket Adaptor模块（SoAd）是唯一能够通过PDU路由器（PduR）处理DDS-PDU的模块。

DDS模块本身提供端到端特性和安全服务。



Architecture – Content of Software Layers

Communication Stack - ChrgM

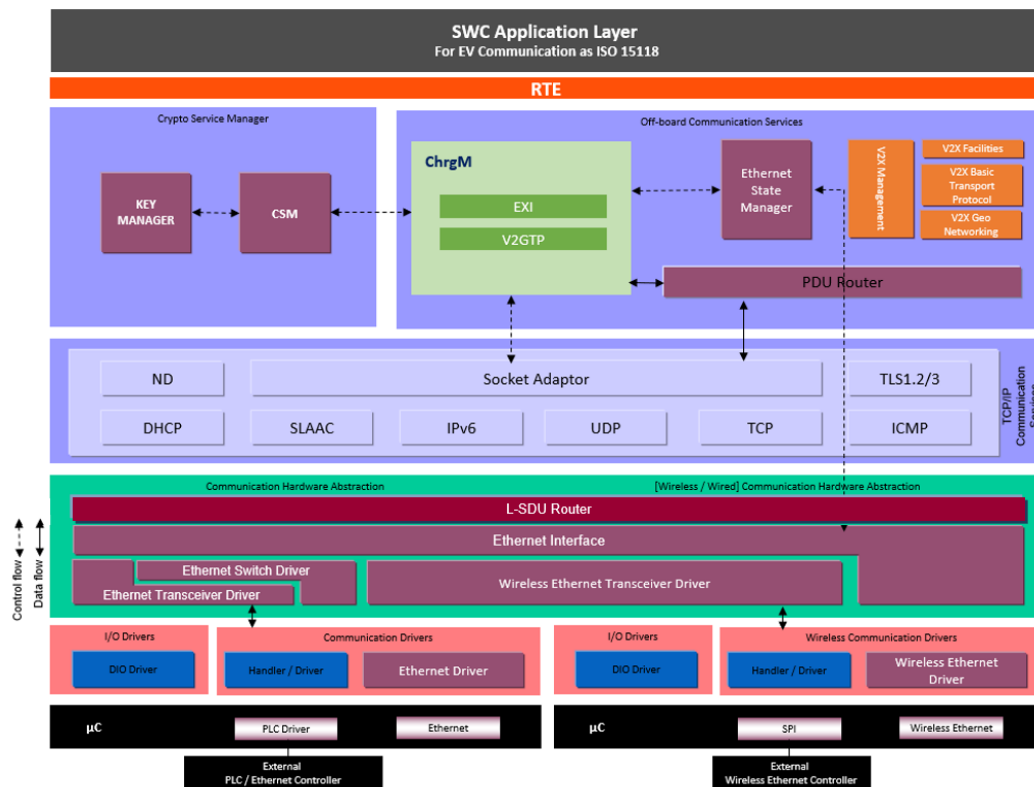
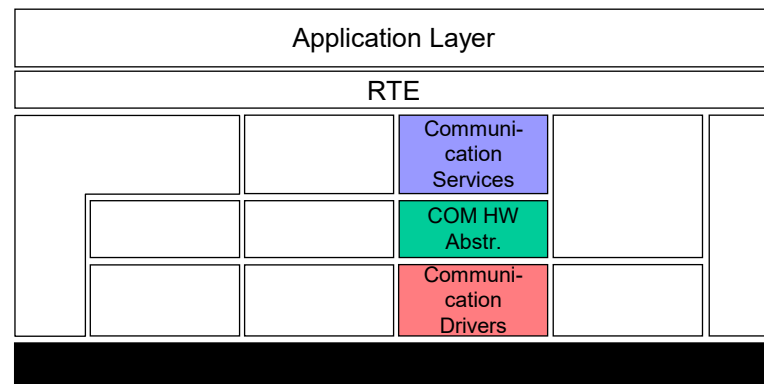
充电管理器（ChrgM）属于AUTOSAR分层架构的通信服务。

任务：

- ChrgM 根据 ISO 15118-2 标准控制电动汽车（EV）与电动汽车供电设备（EVSE）之间的充电过程。
- ChrgM 与不同的 BSW 模块（如 PduR、SoAd、Csm、KeyM、BswM）进行通信，以启用充电过程。
- 提供端口，可供实现充电过程应用部分的软件组件（SW-Cs）使用。
- 提供V2GTP通信协议
- 提供消息的EXI编码与解码
- ChrgM 为管理电动汽车（EV）与电动汽车供电设备（EVSE）之间的通信提供了错误处理机制和定时器。

ChrgM由两个子模块组成：V2GTP和EXI。

- V2GTP（车辆到电网标准协议）：此子模块将数据格式化为PDU。
- EXI（高效 XML 交换）：该子模块根据 W3C 1.0 建议将数据转换为字节流。

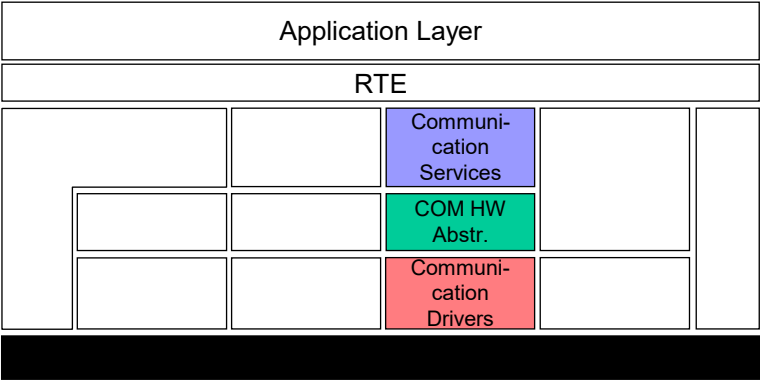


Architecture – Content of Software Layers

通信栈 – 概述

通用通信栈属性：

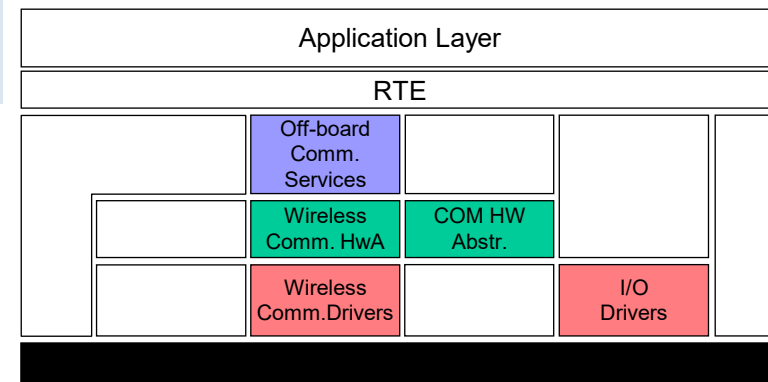
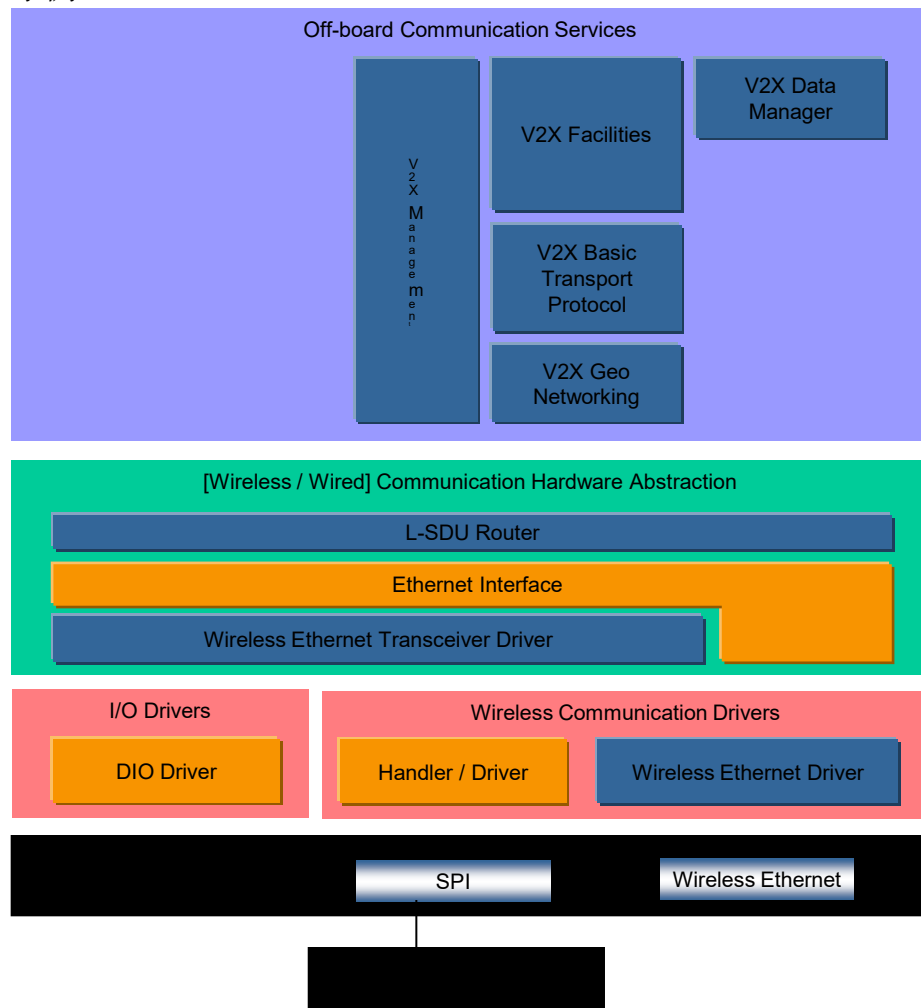
信号网关是AUTOSAR COM中用于路由信号的部分。
 基于PDU的网关是PDU路由器的一部分。 IPDU复用提供了添加信息以实现I-PDU复用的可能性（不同内容但总线ID相同）。
 多I-PDU到容器的映射提供了将多个I-PDU组合成一个更大的（容器）I-PDU的可能性，以便在单个（总线特定）帧中传输。
 上层接口：独立于μC、ECU硬件和网络类型。
 关于网关架构的细化，请参考“示例通信”。



Architecture – Content of Software Layers

车外通信堆栈 – 欧洲车辆对X通信

示例:



欧洲车辆对X通信服务

是一组用于通过自组织无线网络进行车联网通信的模块。

设施层：实现标准化V2X消息的接收与发送功能，构建面向车辆特定软件组件的接口
基础传输协议 = 第4层
地理网络 = 第3层（基于地理区域寻址，对应以太网帧具有独立以太类型标识）
V2X管理层：管理跨层功能（如动态拥塞控制、安全机制、定位与时间同步）
V2X数据管理器：管理V2X消息的接收与格式转换，并通过运行时环境发送至软件组件或经由SOME/IP协议传输

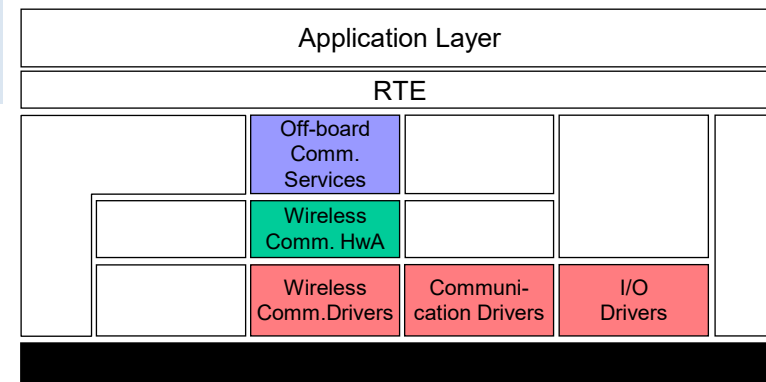
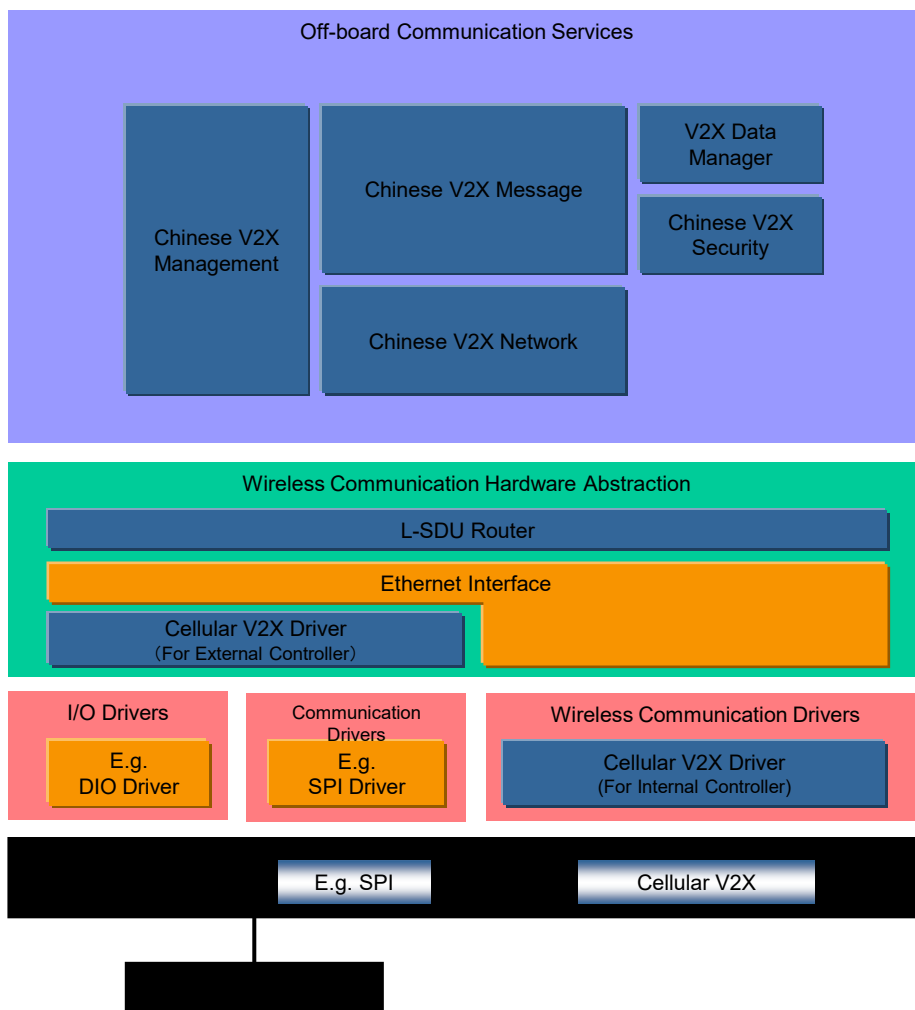
任务：

为无线以太网网络提供统一的接口。向应用程序隐藏协议和消息属性。

Architecture – Content of Software Layers

车外通信堆栈 – 中国车辆对万物通信

示例：



中国车辆-2-X通信服务是一组基于蜂窝V2X技术、遵循中国V2X标准的模块。

消息：实现标准化中国V2X消息的接收与发送功能，构建车辆专用软件组件（SW-C）的接口；实现与消息层相关的管理功能（发送频率、位置与时间、消息标识符） 安全：实现消息封装、解封装及假名管理功能 网络：消息接收与发送、二层ID设置等 管理：管理跨层功能（如专用服务通告等）

任务：

为基于蜂窝的V2X网络提供统一接口，向应用程序隐藏协议与消息属性。

Architecture – Content of Software Layers

服务：内存服务

内存服务由一个模块组成，即NVRAM管理器。它负责管理非易失性数据（从不同的内存驱动程序进行读写）。

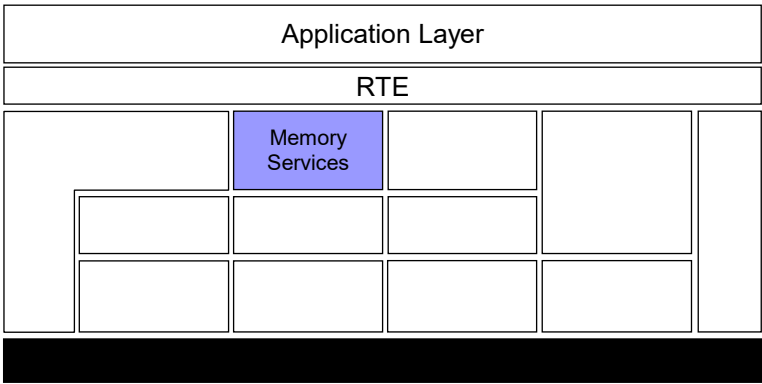
任务：以统一的方式向应用程序提供非易失性数据。抽象化内存位置和属性。提供非易失性数据管理机制，如保存、加载、校验和保护、验证、可靠存储等。

属性：

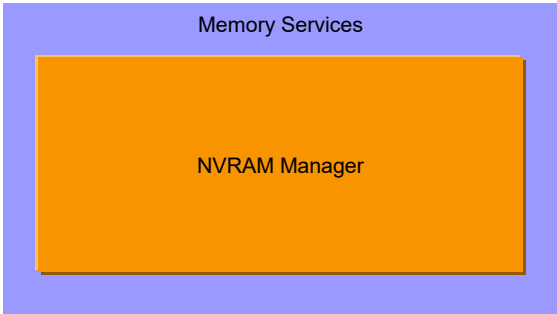
实现：μC与ECU硬件独立，高度可配置

上层接口：根据AUTOSAR规范独立指定和实现，与μC及ECU硬件无关

(AUTOSAR接口)



示例：



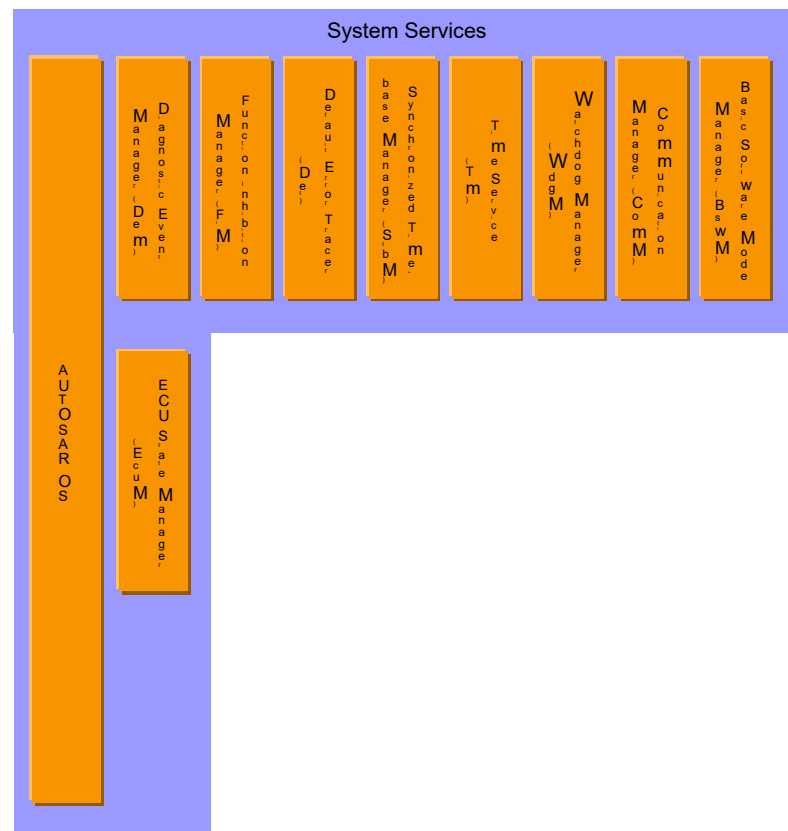
O

依赖于微控制器（如操作系统），并可能支持特殊的微控制器功能（如时间服务），部分依赖于ECU硬件和应用程序（如ECU状态管理器），或独立于硬件和微控制器。

为应用程序和基础软件模块提供基本服务

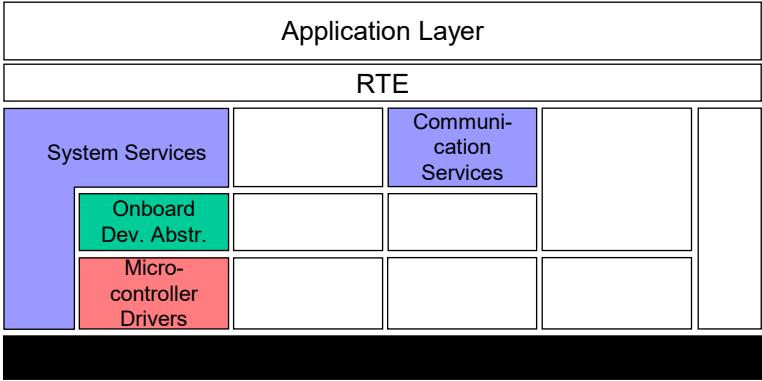
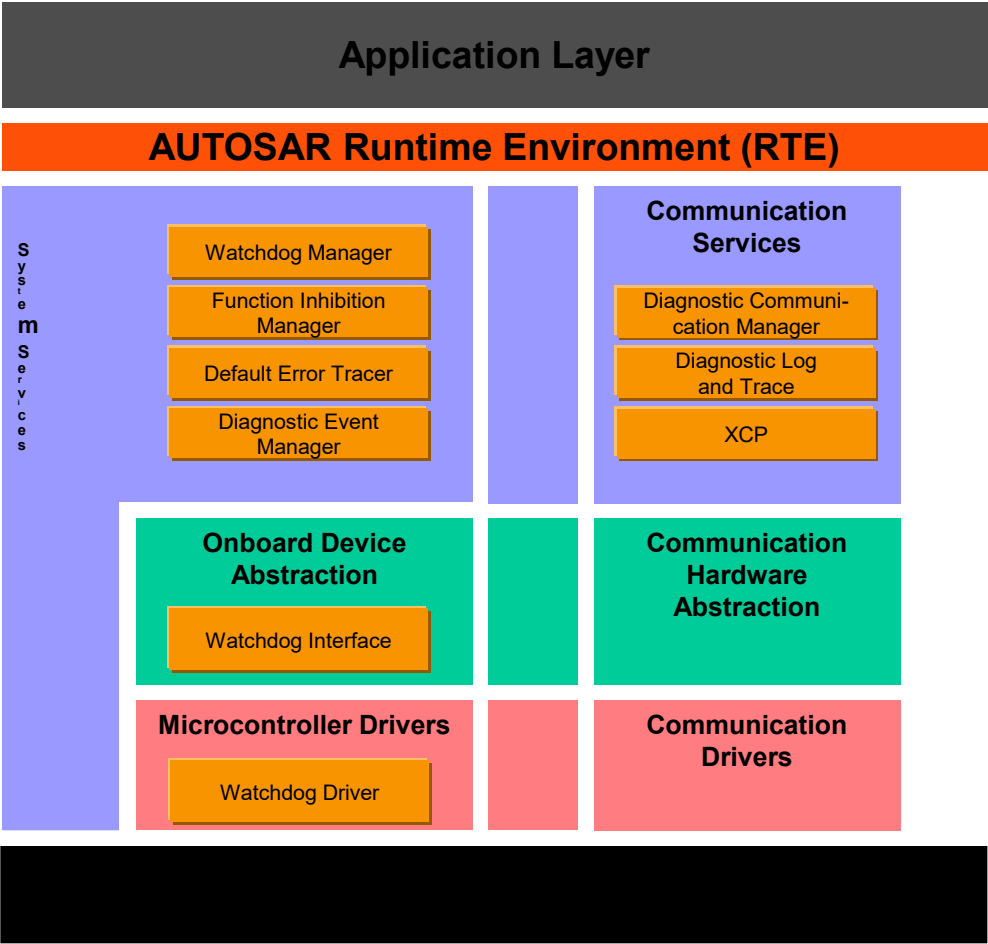
O

实现：部分采用微控制器，ECU硬件及应用特定的上层接口：独立于微控制器和ECU硬件



Architecture – Content of Software Layers

错误处理、报告与诊断



AUTOSAR中针对错误处理的不同方面设有专用模块。例如： 诊断事件管理器负责处理和存储诊断事件（错误）及相关的冻结帧数据。 诊断日志与跟踪模块支持应用程序的日志记录与跟踪功能，可收集用户定义的日志消息并进行转换。

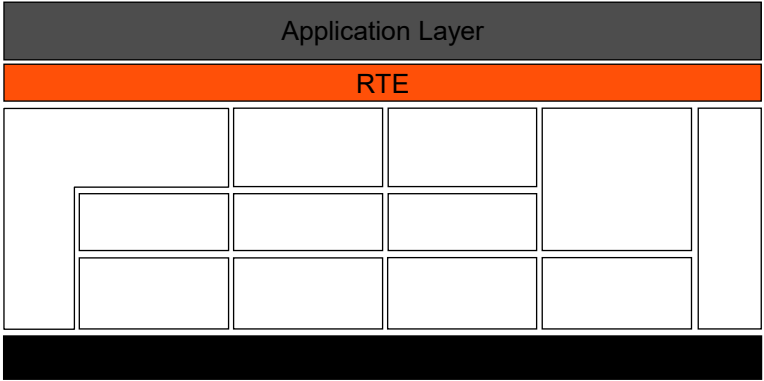
将它们转换为标准化格式。

所有在基础软件中检测到的开发错误都会报告给默认错误跟踪器。
诊断通信管理器为诊断服务提供了一个通用API。
等等。

T传感器/执行器 AUTOSAR 软件组件是一种用于传感器评估和执行器控制的特定类型 AUTOSAR 软件组件。尽管不属于 AUTOSAR 基础软件，但由于其与本地信号的紧密关联，此处仍对其进行说明。出于集成原因（标准化接口实现与接口描述），已决定将传感器/执行器软件组件置于 RTE 之上。因其与原始本地信号的强交互性，可重定位性受到限制。

任务：
提供对连接到ECU的硬件传感器和执行器具体物理属性的抽象。

属性：
实现：μC和ECU硬件独立，传感器和执行器相关



示例：

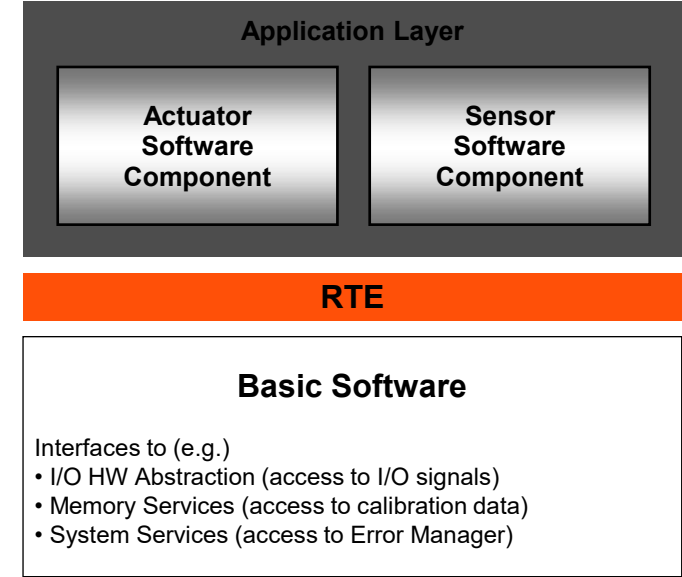


Table of contents

1. 架构

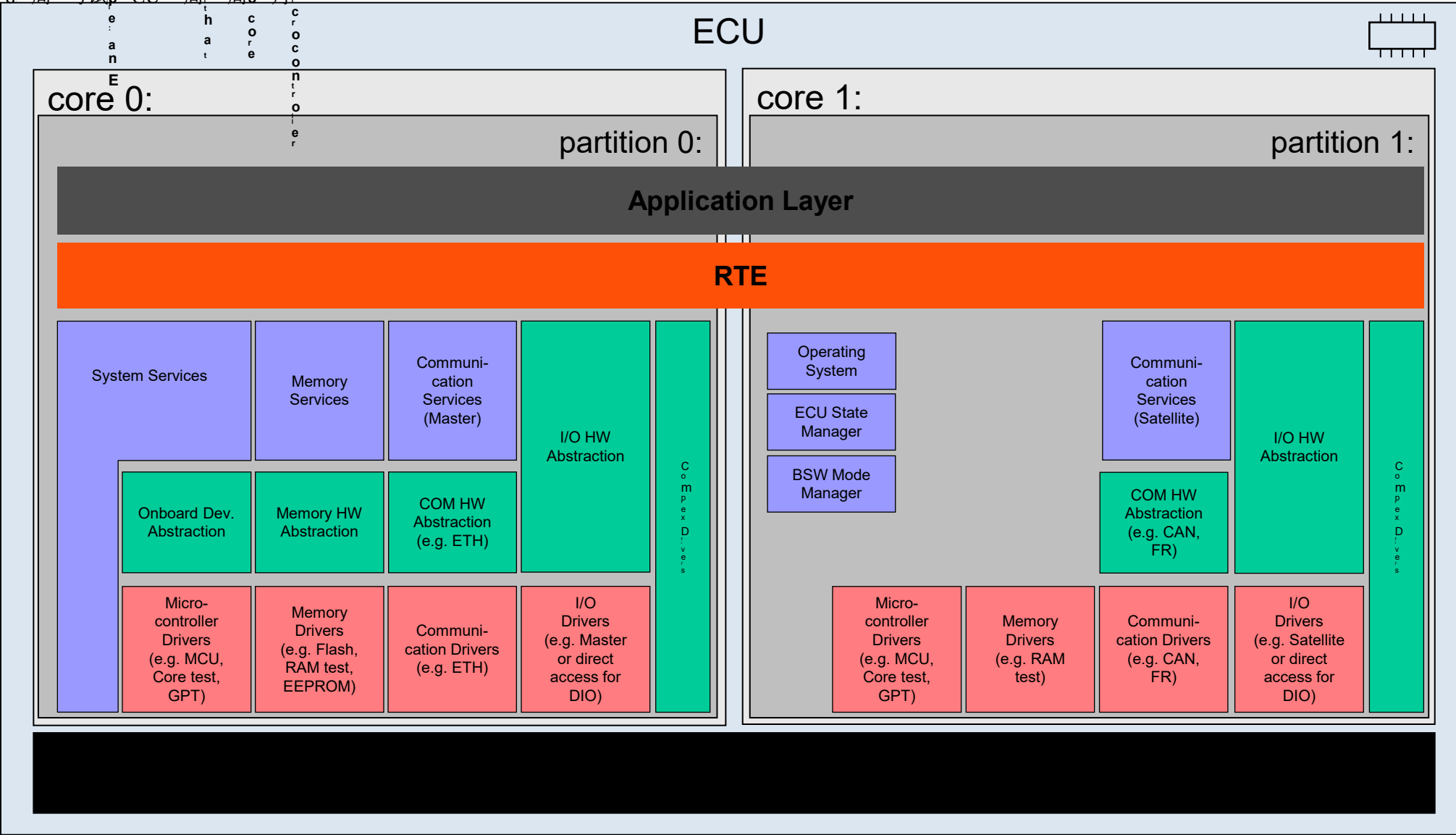
1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置

3. 集成与运行时方面

Architecture – Content of Software Layers

Example of a Layered Software Architecture for Multi-Core Microcontroller



Architecture – Content of Software Layers

分布式BSW模块的详细视图

页 d w 考试, CU w, w. m

BSW模块可以分布在多个分区和核心上。

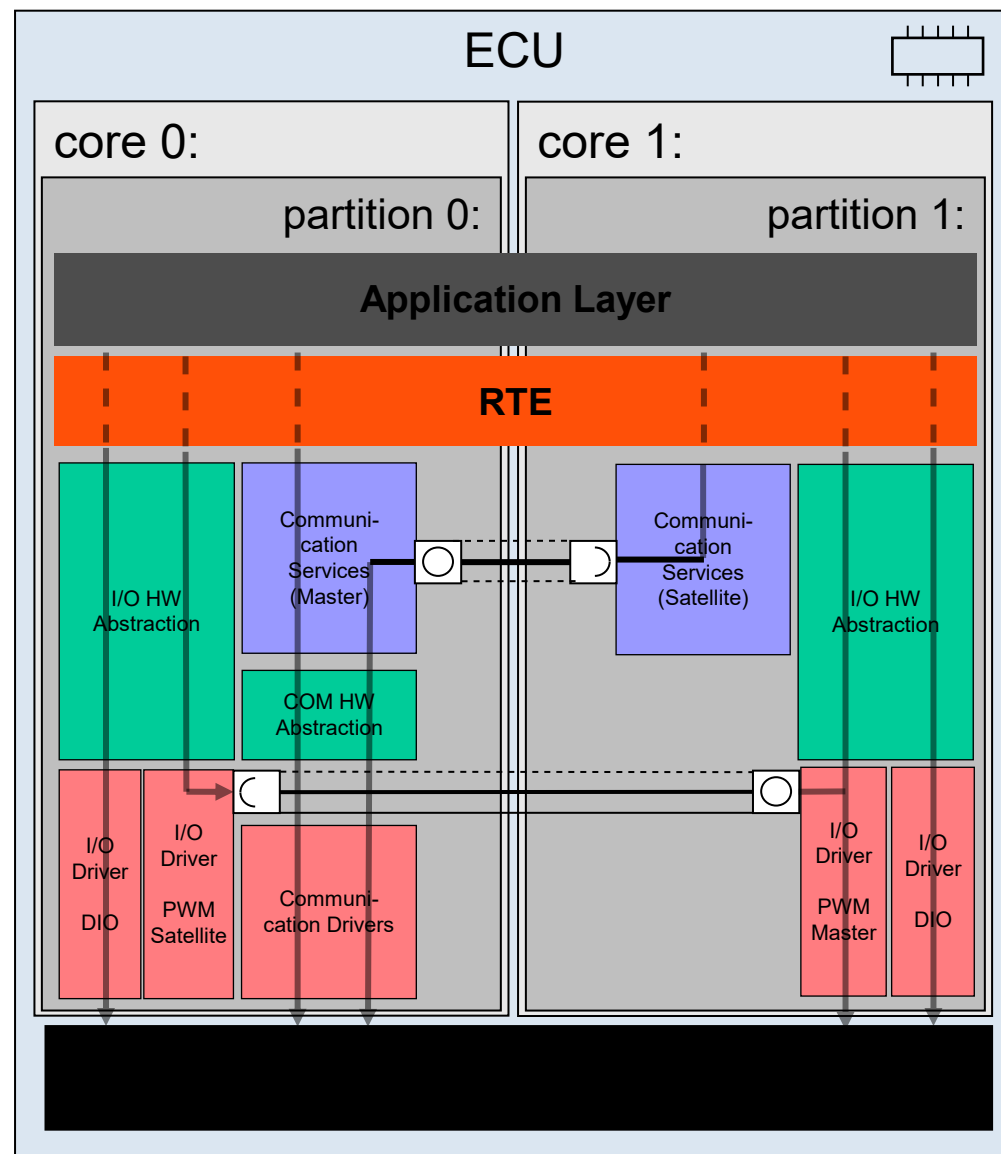
所有分区共享相同的代码。

模块可以在每个分区上完全一致，如图中I/O堆栈外的DIO驱动程序所示。

作为替代方案，他们可以使用核心依赖分支来实现不同的行为。Com服务和PWM驱动器采用主从通信方式，以处理来自相应从属设备对主设备的调用。

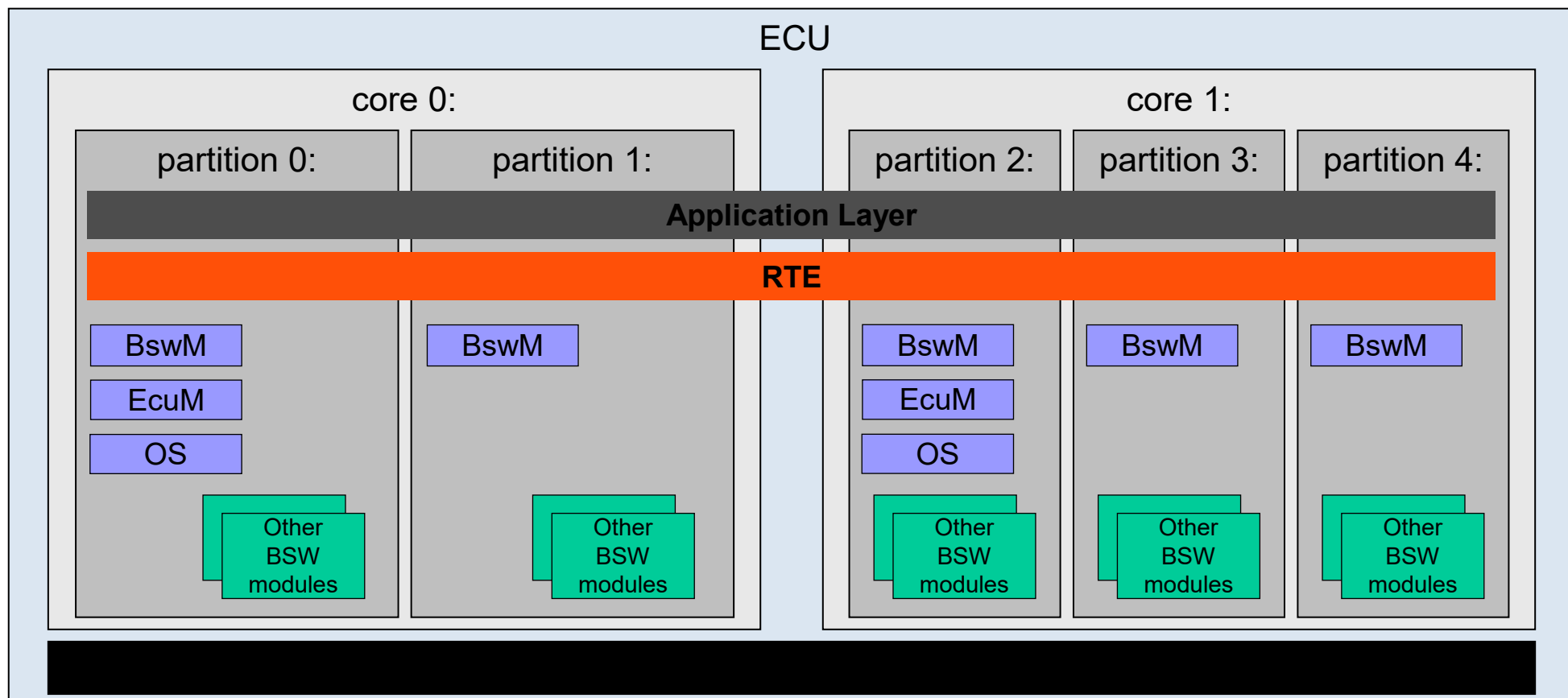
- 主节点与卫星节点之间的通信并未标准化。例如，它可以基于BSW调度器提供的功能，也可以基于共享内存实现。

箭头指示了哪些组件参与处理服务调用，这取决于分布方法以及调用的来源。



Architecture – Content of Software Layers

多分区BSW模块、OS、BswM和EcuM概述



运行BSW模块的每个分区中的基础软件模式管理器（BswM） ■ 所有这些分区都是可信的

每个核心一个EcuM（各自位于可信分区中）

通过引导加载程序启动的那个核心上的 EcuM 是主 EcuM ■，主 EcuM 会启动所有卫星 EcuM。

Architecture – Content of Software Layers

范围：多核系统服务

如图所示，IOC提供通信服务，可供需要在同一ECU上跨O S-Application边界进行通信的客户端访问。IOC是操作系统的一部分。

BSW模块可以在多个核心上执行，例如图中的ComM。负责执行服务的核心在运行时确定。

每个核心都运行一种ECU状态管理。

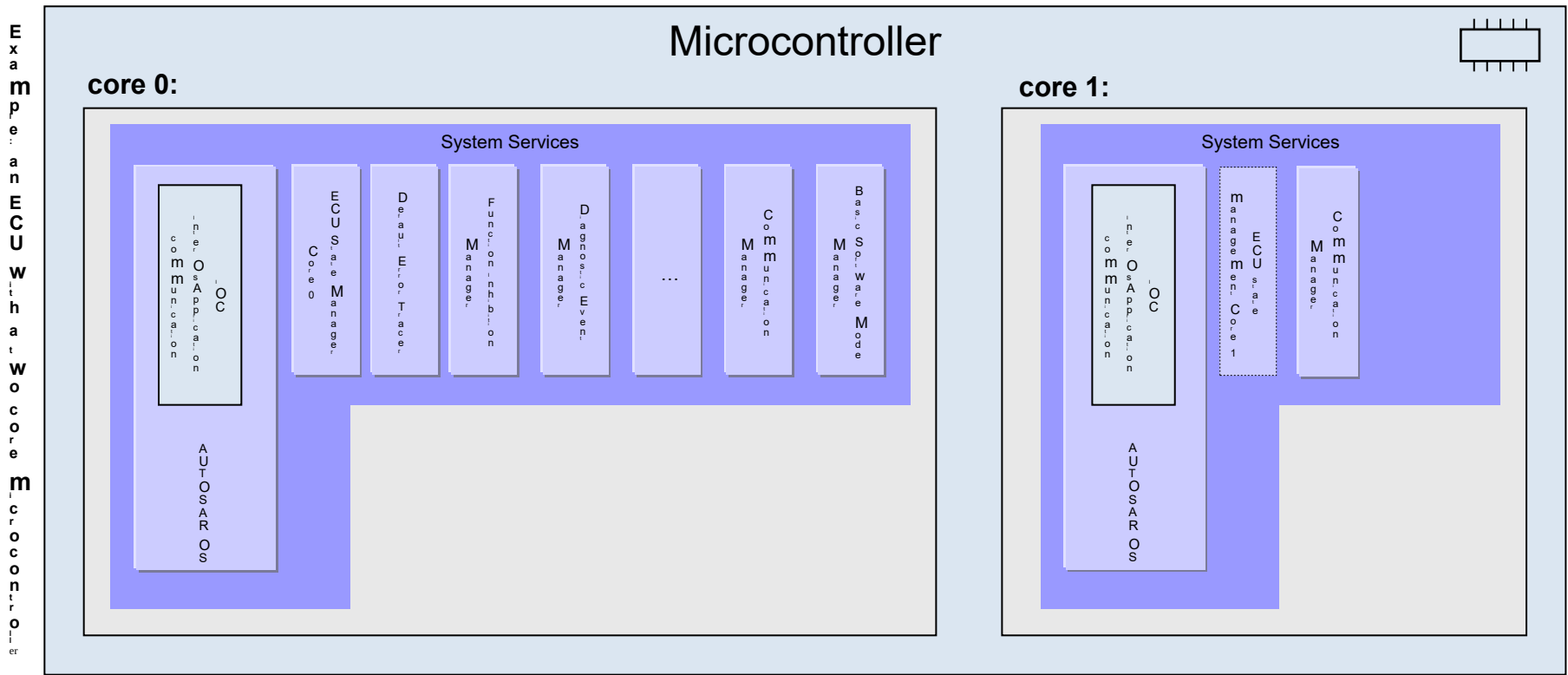
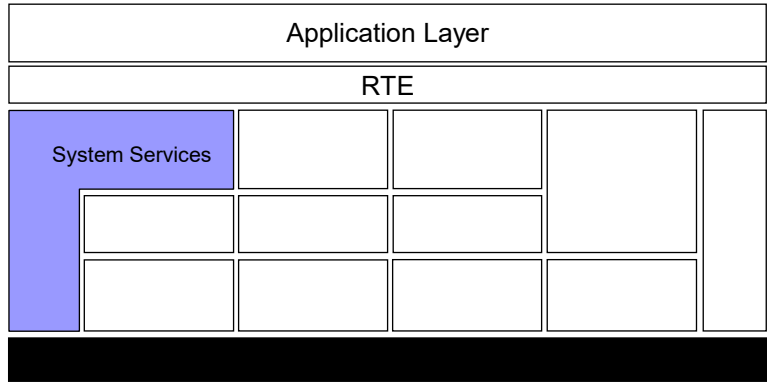


Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置

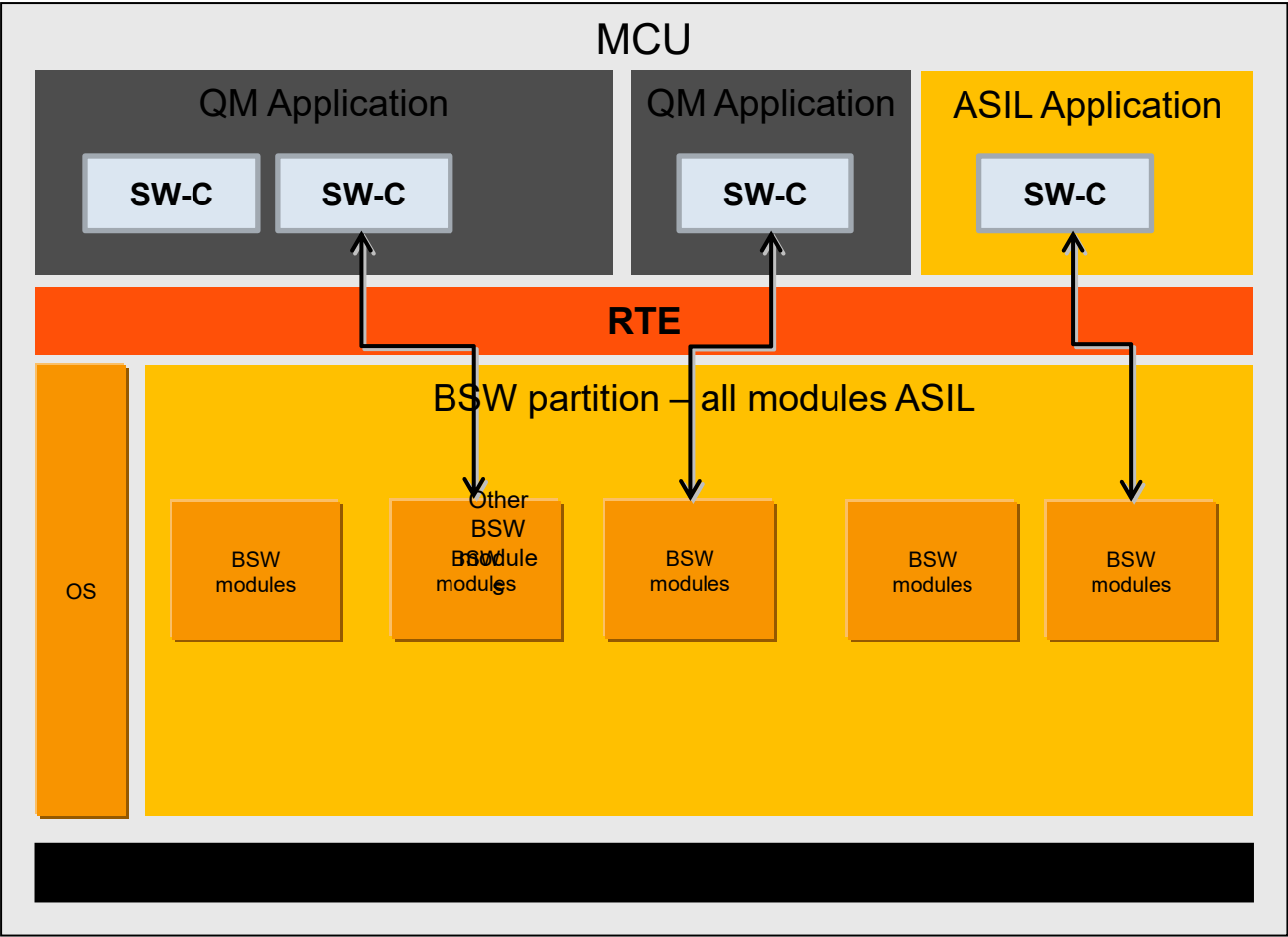
3. 集成与运行时方面

Architecture – Content of Software Layers

AUTOSAR安全处理概述

AUTOSAR提供了一种灵活的方法来支持安全相关的ECU。可采用两种方法：

1. 所有BSW模块均按照要求的ASIL等级进行开发。
2. 选定的模块按照ASIL等级进行开发。ASIL与非ASIL模块被分隔到不同的分区中（BSW分布）。



方法（1）的使用示例

注意：分区基于操作系统应用程序。操作系统应用程序的TRUSTED属性与ASIL/非ASIL无关。

Architecture – Content of Software Layers

面向安全系统的AUTOSAR基础软件层分布式架构

使用不同BSW分区的示例

- 看门狗栈被放置在一个独立的分区■中，ASIL和非ASIL的SW-C可以通过RTE■访问WdgM，其余BSW则放置在各自的分区中。

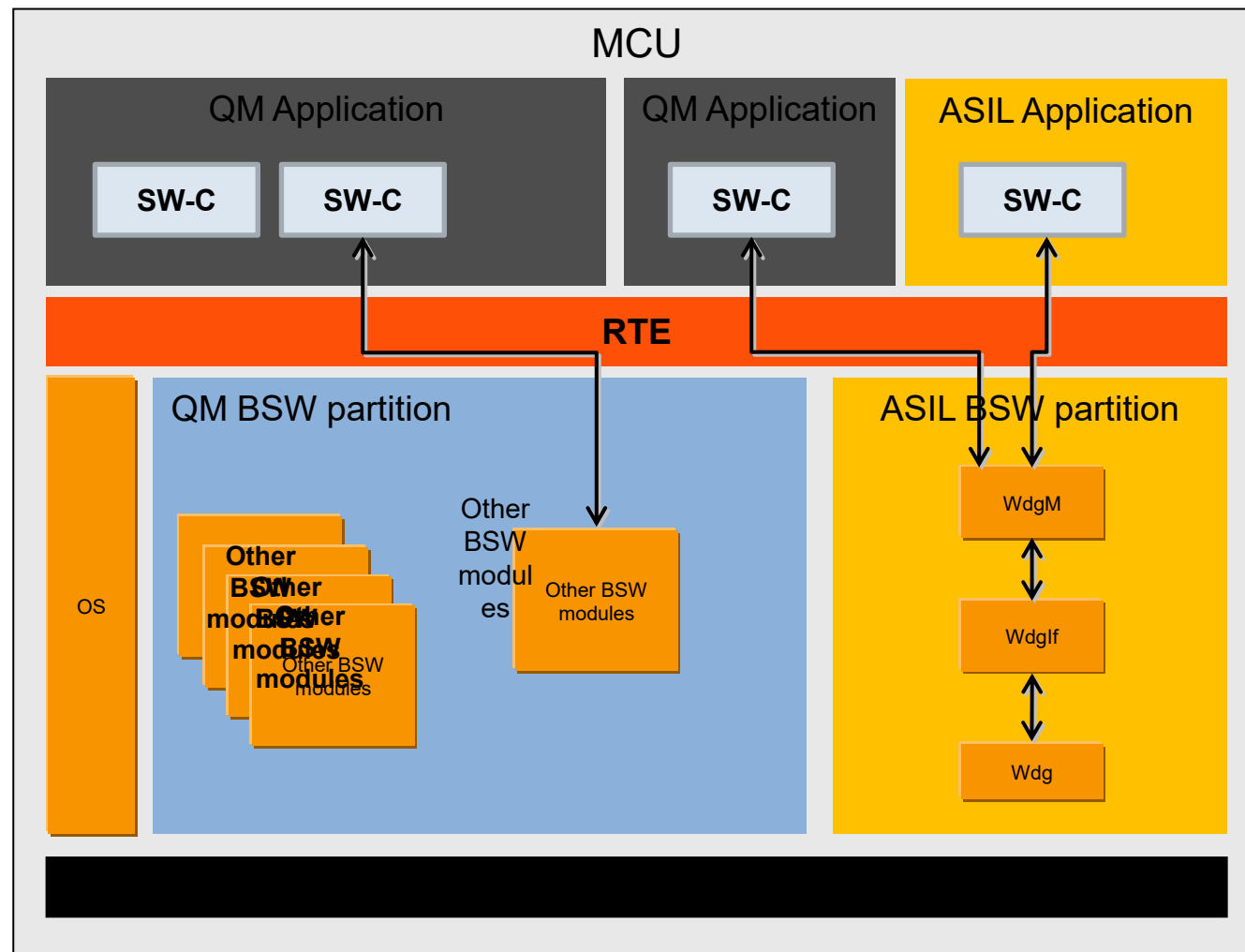


Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置 3. 集成与运行时方面

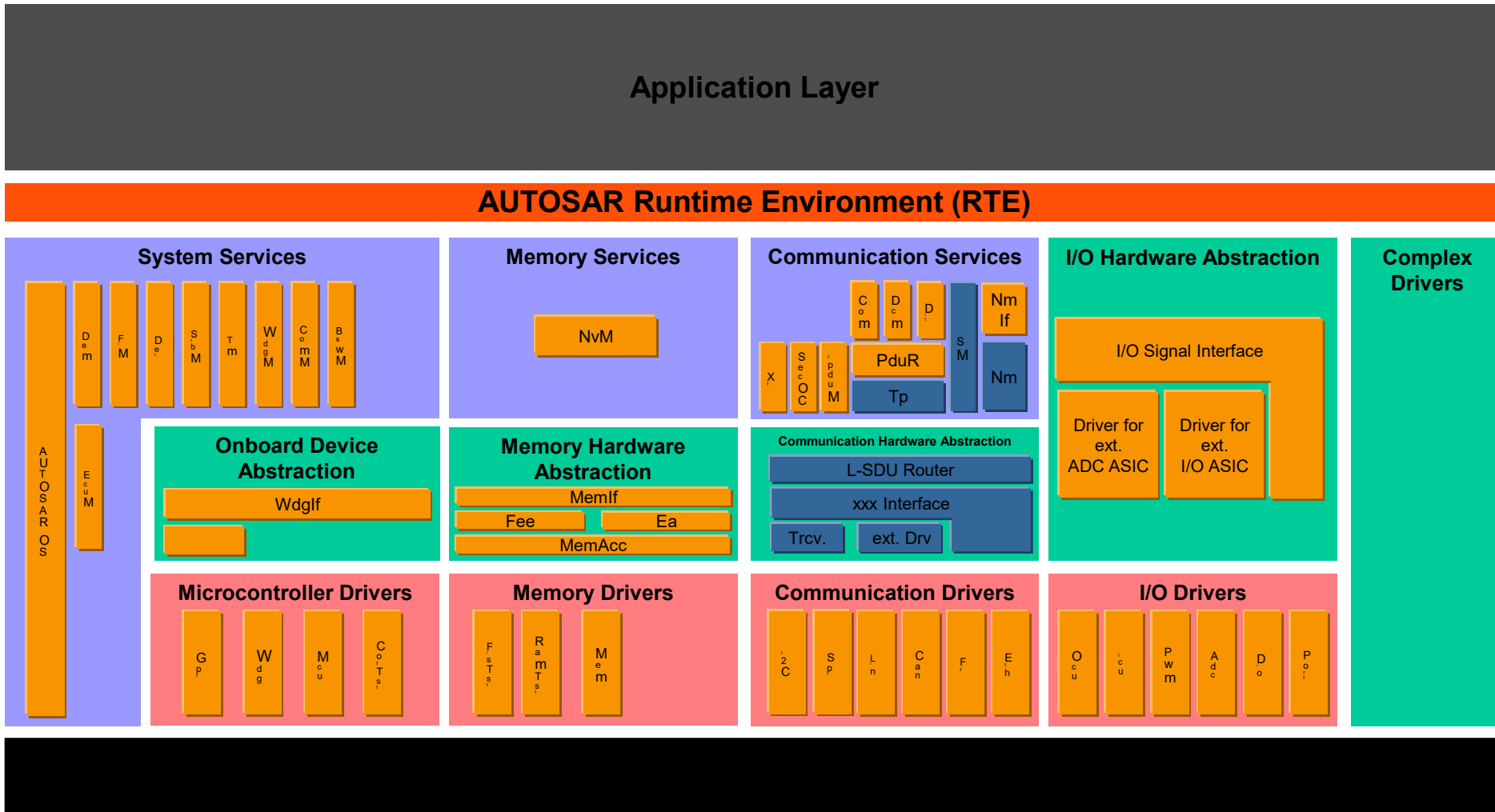
Architecture

概述

f 模块 – 实现一致性类 3

- ICC3

该图展示了基础软件模块到AUTOSAR层的映射关系



Not all modules are shown here

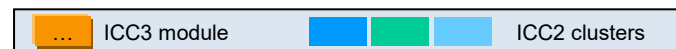
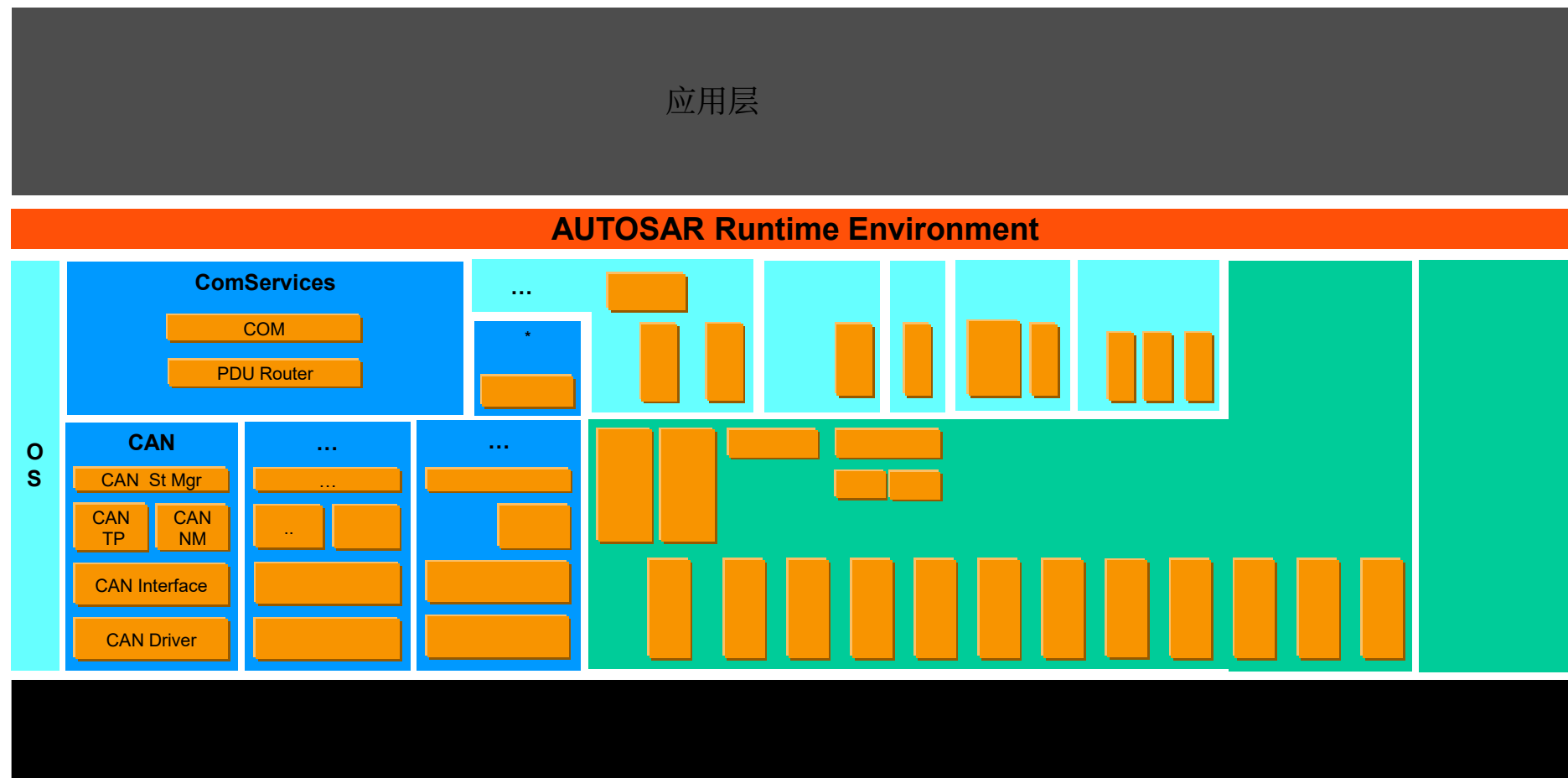
Architecture

概述

模块 – 实现一致性类别 –

ICC2

本文档中展示的聚类是项目迄今为止所定义的聚类。AUTOSAR目前并未在ICC2级别将聚类限制为特定集群，因为许多不同的约束和优化标准可能导致不同的ICC2聚类。可能存在多种不同的AUTOSAR ICC2聚类方案，针对这些方案，可以基于待定义的ICC2合规性方法声明合规性。



Architecture

概述

模块 – 实现一致性类别 –

ICC1

在符合ICC1标准的基础软件中，不需要模块或集群。该专有基础软件的内部结构未作具体规定。

Application Layer

AUTOSAR Runtime Environment

Proprietary software

Architecture

模块概述 – 实现一致性类别 – 对外行为

基础软件（包括RTE）若符合AUTOSAR标准（ICC1-3），其外部行为必须遵循ICC3模块规范的规定。

例如，对于以下方面的行为：总线，引导加载程序和应用程序。此外，ICC1/2配置在系统描述方面应与ICC3兼容。符合ICC3规范的行为

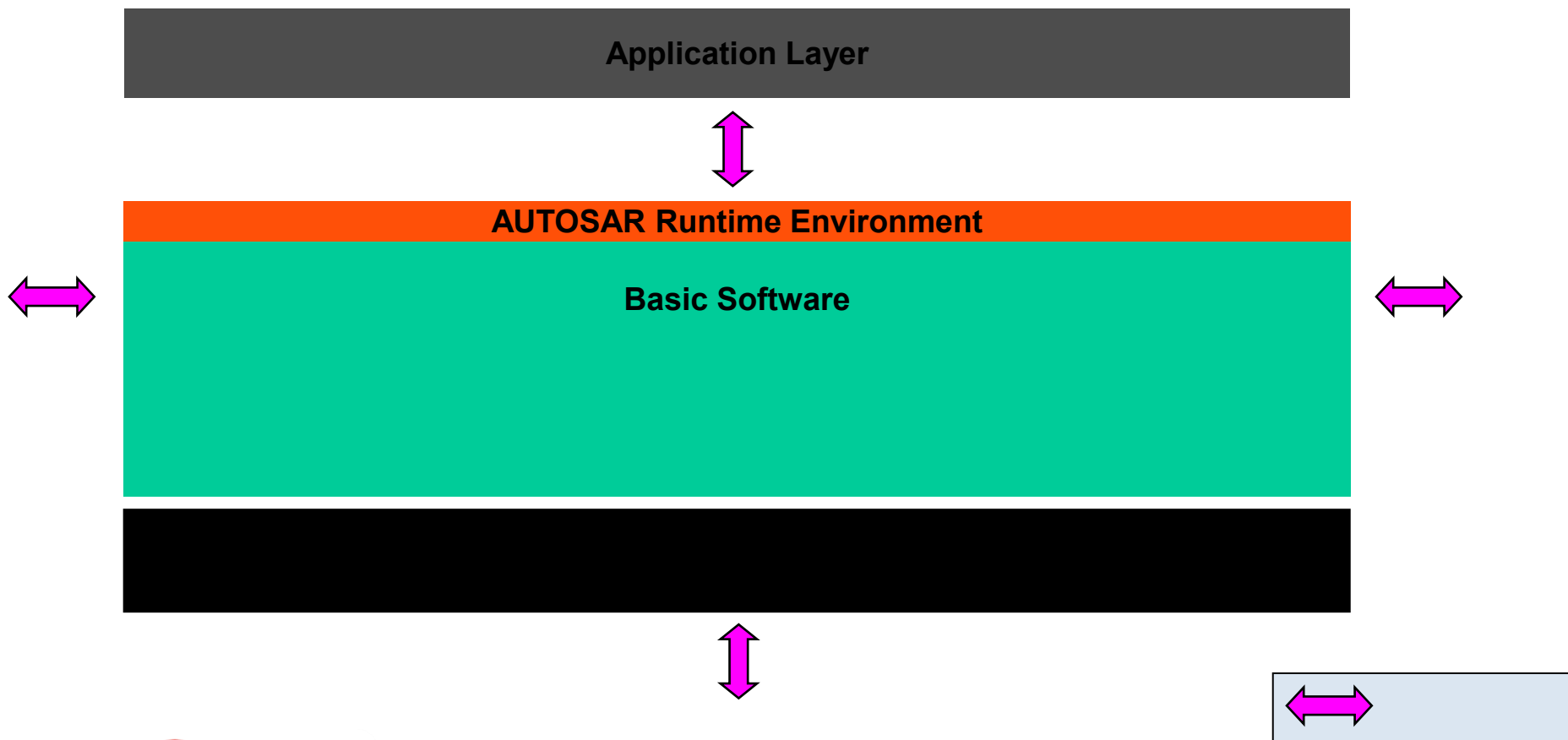


Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置 3. 集成与运行时方面

Interfaces

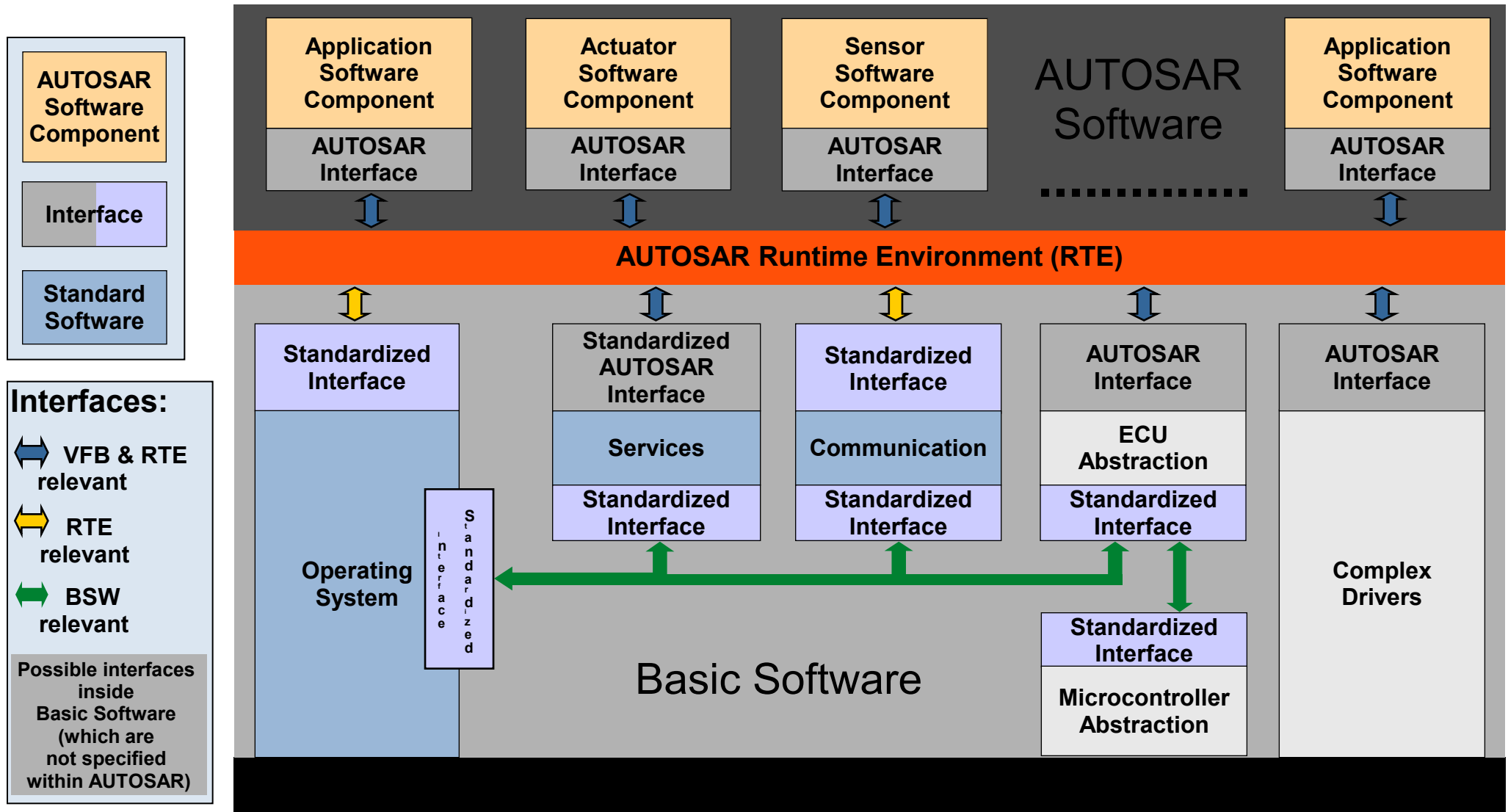
AUTOSAR中的接口类型

AUTOSAR Interface	An "AUTOSAR Interface" defines the information exchanged between software components and/or BSW modules. This description is independent of a specific programming language, ECU or network technology. AUTOSAR Interfaces are used in defining the ports of software-components and/or BSW modules. Through these ports software-components and/or BSW modules can communicate with each other (send or receive information or invoke services). AUTOSAR makes it possible to implement this communication between Software-Components and/or BSW modules either locally or via a network.
Standardized AUTOSAR Interface	A "Standardized AUTOSAR Interface" is an "AUTOSAR Interface" whose syntax and semantics are standardized in AUTOSAR. The "Standardized AUTOSAR Interfaces" are typically used to define AUTOSAR Services, which are standardized services provided by the AUTOSAR Basic Software to the application Software-Components.
Standardized Interface	A "Standardized Interface" is an API which is standardized within AUTOSAR without using the "AUTOSAR Interface" technique. These "Standardized Interfaces" are typically defined for a specific programming language (like "C"). Because of this, "standardized interfaces" are typically used between software-modules which are always on the same ECU. When software modules communicate through a "standardized interface", it is NOT possible any more to route the communication between the software-modules through a network.

5
u
4
9
d
e
g
a
p

Interfaces

组件与接口视图（简化版）



注意：该图在表示各层之间可能的交互方面并不完整。

Interfaces: General Rules

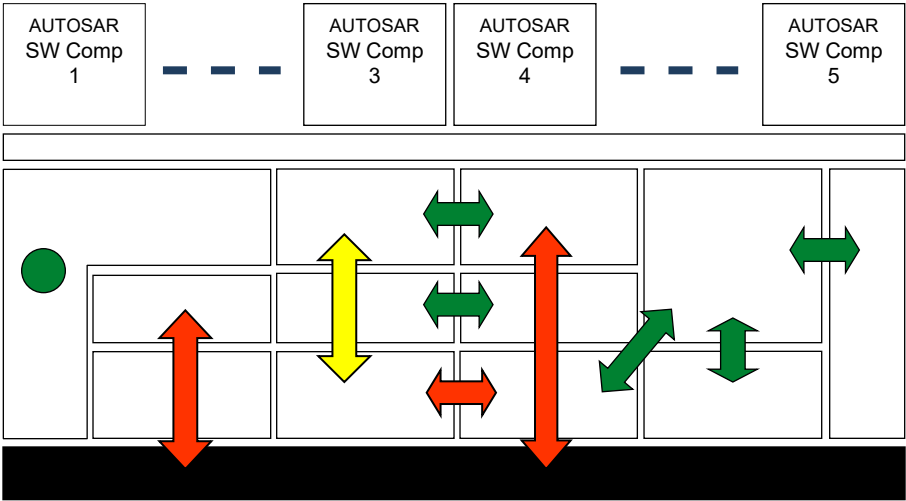
通用接口规则

水平接口

-  服务层：允许水平接口 示例：错误管理器使用NVR
AM管理器保存故障数据
-  ECU抽象层：允许水平接口
-  一个复杂的驱动程序可能会使用选定的其他BS
W模块
-  μ C抽象层：不允许水平接口。例外：出于性能原因
，允许可配置的通知。

垂直接口

-  一个层可以访问其下方SW层的所有接口。
-  应避免绕过任何软件层
-  不允许绕过两个或更多软件层
-  绕过 μ C抽象层是不允许的
-  一个模块可以访问另一层组的较低层模块（例如用于外部硬件的SPI）
-  所有层都可能与系统服务交互。





uses

- (gray background indicates “non-Basic Software” layers)

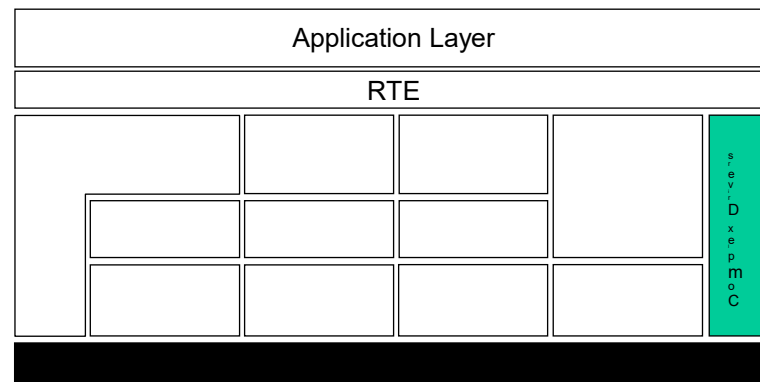
The diagram illustrates a 2D grid with 10 rows and 20 columns. The rows are color-coded: the first 4 rows are purple, the next 4 rows are green, and the last 2 rows are red. The columns are color-coded: the first 5 columns are purple, the next 5 columns are green, and the last 5 columns are red. A legend in the top-left corner shows a curved arrow pointing right, labeled "uses", indicating that the color of the row and column determines the color of the cell.

*: includes wired and wireless communication

Interfaces

与复杂驱动程序接口（1）

复杂驱动程序可能需要与分层软件架构中的其他模块对接，或者分层软件架构中的模块可能需要与复杂驱动程序对接。若存在此类情况，应遵循以下规则：



1. 接口 从分层软件架构的模块到Com

plex 驱动程序

这仅在复杂驱动程序提供一个可由访问的AUTOSAR模块通用配置的接口时才被允许。一个典型例子是PDU路由器：复杂驱动程序可以实现新总线系统的接口模块。这一点已在PDU路由器的配置中得到处理。

2. 从复杂驱动程序到分层软件架构模块的接口对接

同样，这只有在分层软件架构的相应模块提供了接口，并且准备好被复杂驱动程序访问时才被允许。通常这意味着

各接口被定义为可重入式。 若使用回调例程，其名称可配置。 不存在对模块状态进行管理的上层模块（并行访问可能改变状态而未被上层模块察觉）。

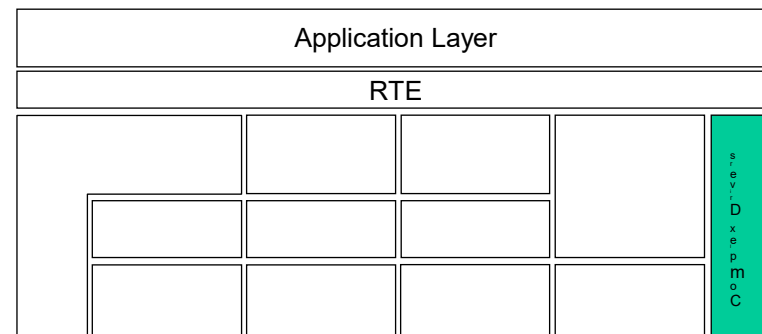
Interfaces

与复杂驱动程序接口 (2)

通常可以访问以下模块：

SPI驱动程序 GPT驱动程序 I/O驱动程序，但需注意可重入性通常仅存在于{v*}

分离组/通道等。通常不允许并行访问同一组/通道等。这需要在配置过程中加以注意。NVRAM管理器作为内存栈的独占访问点 看门狗管理器作为看门狗栈的独占访问点 PDU路由器作为通信栈中独立于总线与协议的独占访问点 总线专用接口模块作为通信栈中总线专用的独占访问点 NM接口模块作为网络管理栈的独占访问点 通信管理器（仅来自上层）与基础软件模式管理器作为状态管理的独占访问点 Det、Dem和Dlt 操作系统（前提是所使用的OS对象未被分层软件架构中的模块占用） 尽管如此，每个模块仍需检查其相应功能是否标记为可重入。例如，“初始化”函数通常不可重入，应仅由ECU状态管理器调用。



Interfaces

与复杂驱动程序接口 (3)

在多核架构的情况下，存在额外的规则：

BSW可以分布在多个核心上。负责执行对BSW服务调用的核心由其BswOperationInvokedEvent的任务映射决定。

仅允许模块内部通信跨越分区和核心边界，采用主/从实现方式。

因此，如果CDD需要访问BSW的标准化接口，它必须驻留在同一核心上。

如果CDD位于不同的核心上，它可以使用正常的端口机制来访问AUTOSAR接口和标准化的AUTOSAR接口。这会调用RTE，RTE利用操作系统的IOC机制将请求传输到另一个核心。

然而，如果CDD需要访问BSW的标准化接口且不驻留在同一核心上，

- 要么一个提供标准化接口的卫星可以在CDD所在的核心上运行，并将调用转发到另一个核心■；要么需要在另一个核心上实现CDD的存根部分，并且需要使用操作系统的IOC机制（类似于RTE所做的那样）来组织CDD本地的通信。

此外，在后一种情况下，CDD的初始化部分也需要驻留在不同核心的存根部分。

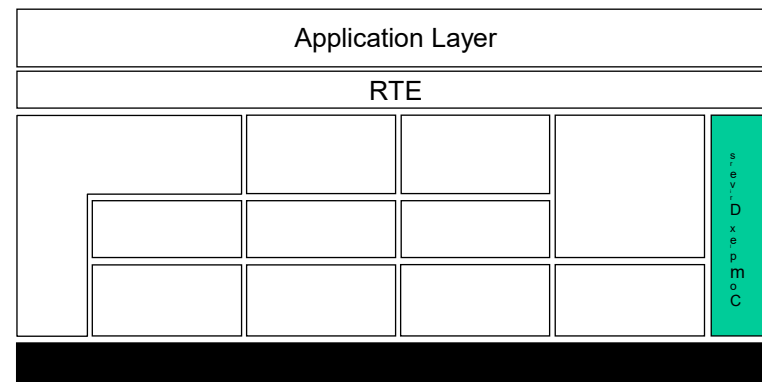


Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置 3. 集成与运行时方面

Interfaces: Interaction of Layers – Example “Memory”

引言

以下页面以“内存”为例进行说明：

可用的内存服务模块有哪些特性/区别？

软件各层之间如何交互？

软件界面看起来是怎样的？

ECU抽象层内部包含什么？

抽象层如何高效实现？

Background: Comparison between memory service modules and memory types

不同的服务模块（内存管理器）从所使用的非易失性（NV）内存中抽象出来，但硬件的特性会影响它们的设计以及访问的实现方式。

不同列出的模块的使用受到所用NV硬件属性的限制。

下表列出了模块及相关NV存储器的属性。

Module	Use cases, features	Supported NV memory properties	Example hardware
NvM	- Storage of module data (e.g. Error information, special configuration info, status information, diagnostic data, ...) - Supports many reader/writer (BSW and SW-C) in parallel. - Mostly read during start-up and written in shutdown, but intermediate reads/writes during normal operation are also supported - Typical data size per user is bytes to some KiB	- Direct (memory mapped) and indirect (e.g. via SPI) NV access - Serialized access (read-while-write-in-same-HW-segment may not work → NvM always buffer the data)	- Internal data flash (via Flash EEPROM emulation) - External EEPROM / data flash
BndM	- Storage of car specific data (Very rare) Writes via diagnostics, only in „controlled environment“ (e.g. repair shop) - Supports many readers (SW-C) in parallel - Users have direct access via pointer - Typical size many KiB	- Direct access of NV data (via pointer) is required - Parallel read of NV data is required	- Internal data flash - Internal code flash
FOTA (manager)	- Storage of model specific car data/code - Very few users, typically only one - Typical size in MiB - Write new data in the background e.g. over several driving cycles (interruptible and preemptable update procedure)	Read-While-Write (e.g. via memory abstraction/partitioning)	Internal and external code flash

Interfaces: Interaction of Layers – Example “Memory”

示例与初览

此示例展示了NVRAM管理器与看门狗管理器如何在假定的硬件配置下与驱动程序交互：

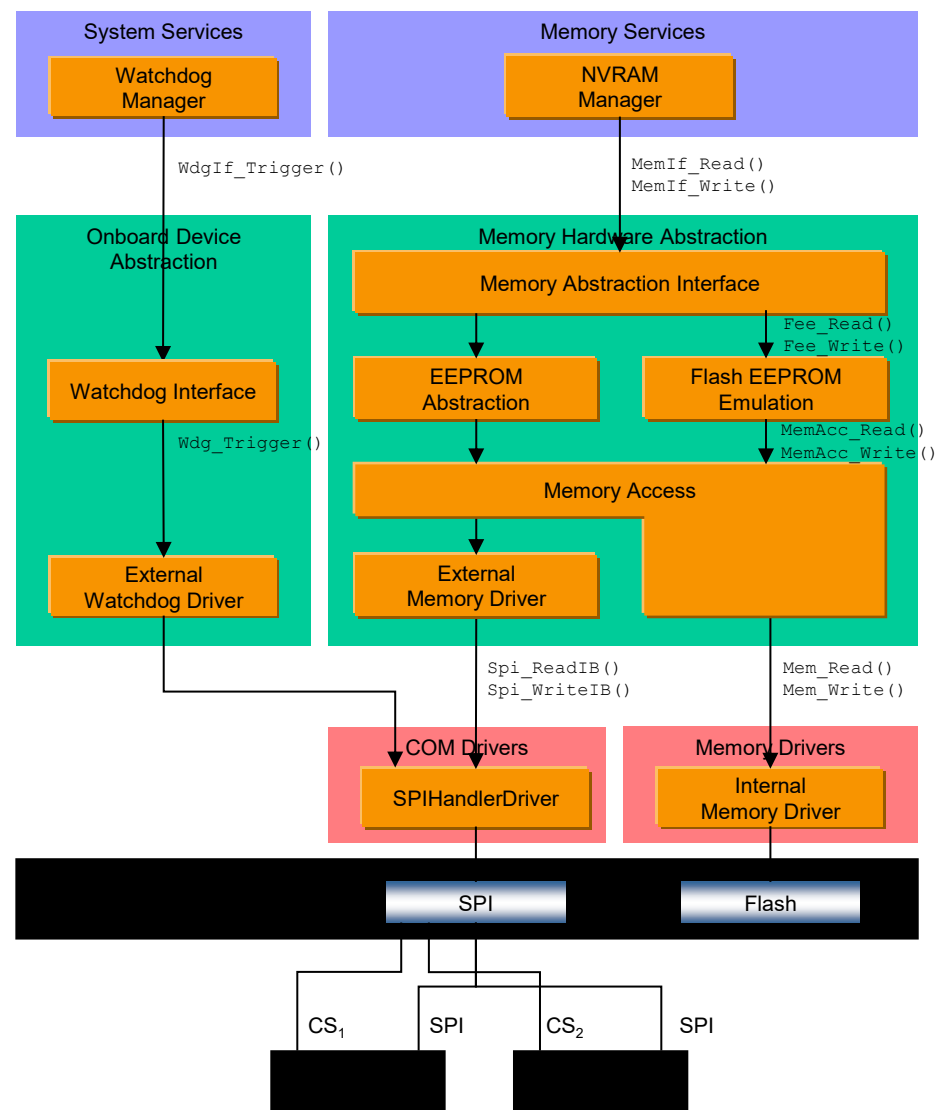
ECU硬件包括一个外部EEPROM和一个外部看门狗，它们通过相同的SPI接口连接到微控制器。

SPIHandlerDriver控制对SPI硬件的并发访问，必须赋予看门狗访问比EEPROM访问更高的优先级。

该微控制器还包含一个内部闪存，它与外部EEPROM并行使用。EEPROM抽象层和闪存EEPROM仿真提供的API在语义上是完全相同的。

内存抽象接口可以通过以下方式实现：

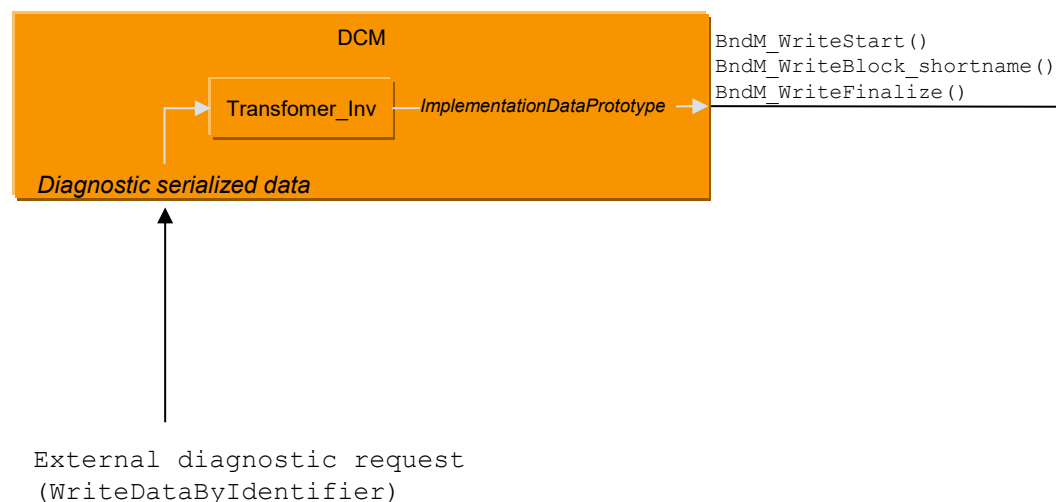
在运行时基于设备索引（内部/外部）进行路由 在运行时基于块索引进行路由（例如 > 0x01FF = 外部EEPROM） 在配置期间通过NVRAM管理器内带函数指针的ROM表进行路由（此时内存抽象接口仅“虚拟”存在）



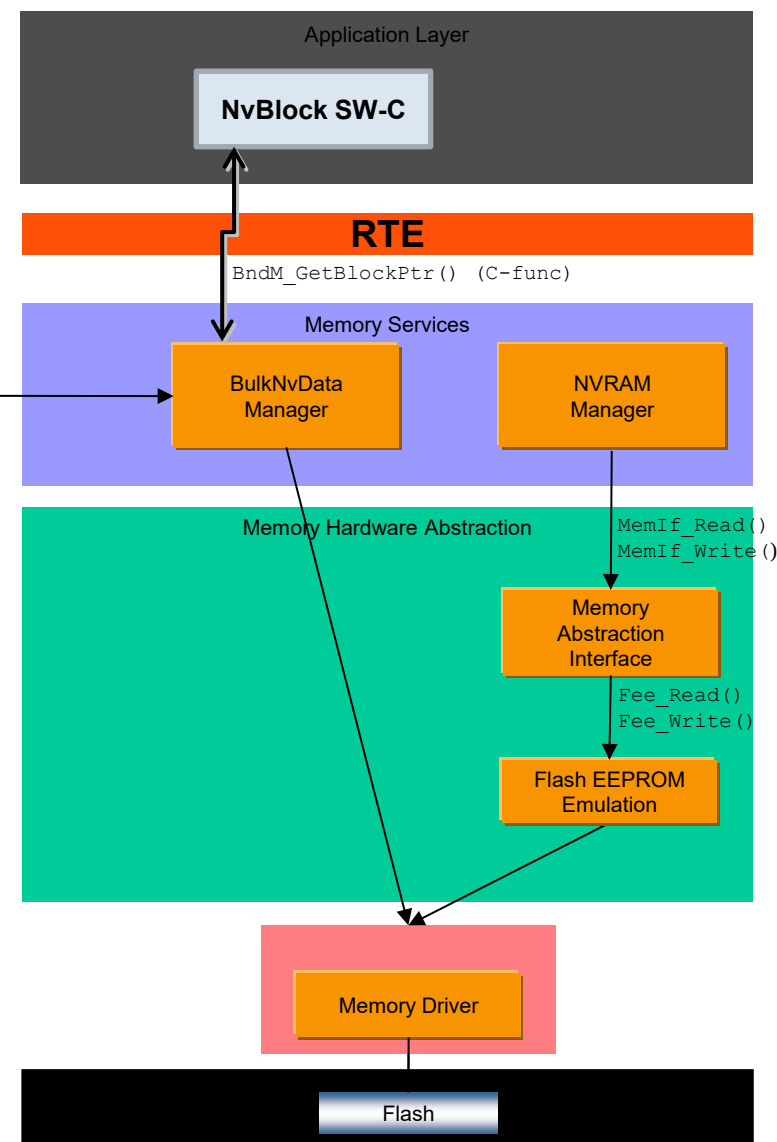
Interfaces: Interaction of Layers – Example “Memory”

批量NV数据管理器

Use-case Bulk NV Data Manager (BndM):
Persistent data which is very infrequently written
and additionally huge in size.



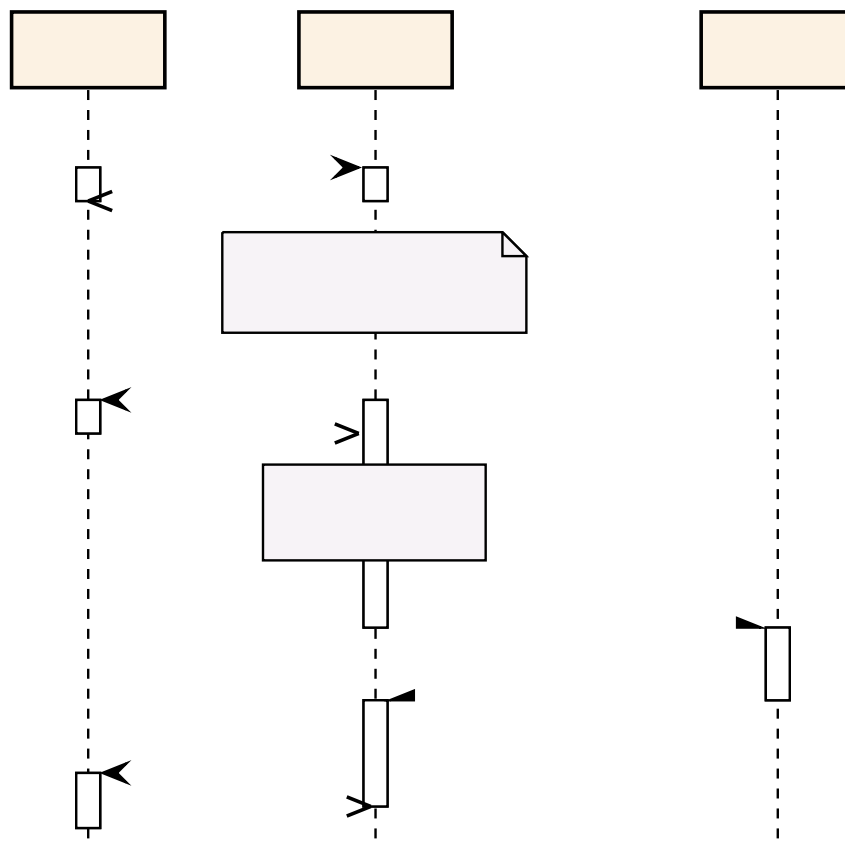
Use-case NVRAM Manager (NvM):
Persistent data which is high frequently updated
or small in its size



Interfaces: Interaction of Layers – Example “Memory”

NvM块压缩

用例：频繁写入大型数据块，仅涉及局部小范围更改 ■ 实际算法因供应商而异
(数据块分割、压缩、差异编码等)



Interfaces: Interaction of Layers – Example “Memory”

深入探究内存硬件抽象

架构描述

TNVRAM管理器通过内存抽象接口访问驱动程序。
。它使用设备索引来寻址不同的内存设备。

接口描述

T内存抽象接口可以具有以下接口（例如对于写入函数）：

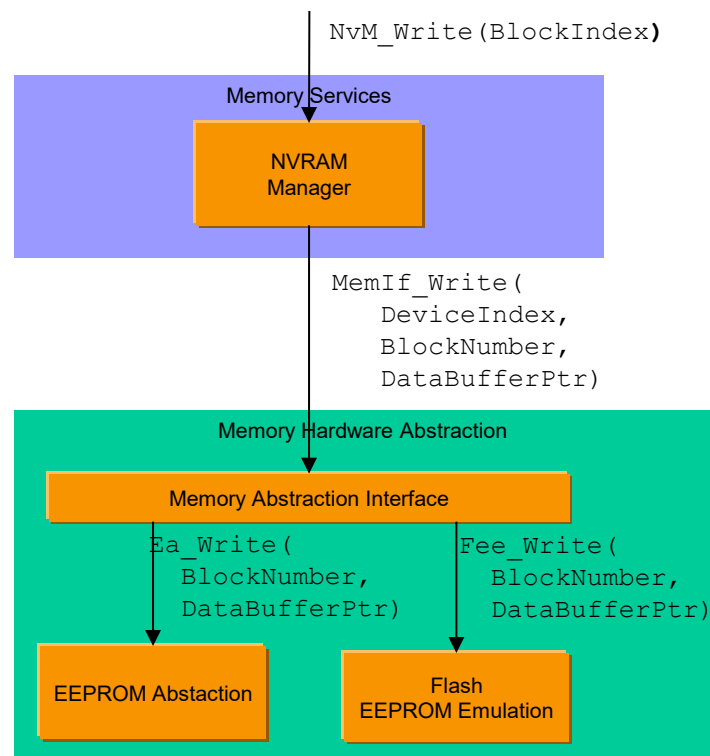
MemIf_Write

(uint8 设备索引, uint16 块编号, uint8 *数据缓冲区指针)

EEPROM抽象以及Flash EEPROM仿真可具备以下接口（例如针对写入函数）：

Std_ReturnType Ea_Write

(uint16 块编号, uint8 *数据缓冲区指针)



Interfaces: Interaction of Layers – Example “Memory”

内存抽象接口的实现

情况1: 仅使用一种NV设备类型

这是常见的使用场景。在这种情况下，若源代码可用，内存抽象可通过忽略DeviceIndex参数的简单宏来实现。

以下示例仅展示写入函数：

文件 MemIf.h:

```
#include "Ea.h"                /* 用于提供对EEPROM抽象层的访问 */  
#define MemIf_Write(DeviceIndex, BlockNumber, DataBufferPtr) \ Ea_Write(BlockNum  
ber, DataBufferPtr)
```

文件 MemIf.c:

不存在

结果:

运行时无需额外代码，NVRAM管理器可直接虚拟访问EEPROM抽象层或闪存模拟层。

Interfaces: Interaction of Layers – Example “Memory”

内存抽象接口的实现

情况2：使用两种或更多种不同类型的NV设备

在这种情况下，必须使用DeviceIndex来选择正确的NV设备。通过使用函数指针数组，实现也可以非常高效。

以下示例仅展示写入函数：

文件 MemIf.h：

```
extern const WriteFctPtrType WriteFctPtr[2];
```

```
#define MemIf_Write(DeviceIndex, BlockNumber, DataBufferPtr) \WriteFctPtr[DeviceIndex](BlockNumber, DataBufferPtr)
```

文件 MemIf.c：

```
#include "Ea.h"    /* 用于获取API函数地址 */ #include "Fee.h"    /* 用于获取API函数地址 */  
#include "MemIf.h" /* 用于获取WriteFctPtrType类型 */
```

```
const WriteFctPtrType WriteFctPtr[2] {v*} {Ea_Write, Fee_Write};
```

结果：

所需的代码和运行时与函数指针表位于NVRAM管理器内部时相同。内存抽象接口不会产生任何开销。

Interfaces: Interaction of Layers – Example “Memory”

结论

结论:

抽象层可以实现非常高效的实现

抽象层可以进行扩展

内存抽象接口简化了NVRAM管理器对一个或多个EEPROM和闪存设备的访问。

Interfaces: Interaction of Layers – Example “Communication”

PDU流经分层架构

术语解释：

山东大学

SDU是“服务数据单元”的缩写。它是上层传递的数据，带有传输该数据的请求。同样，它也是下层接收后提取并传递给上层的数据。

SDU是PDU的一部分。

PCI

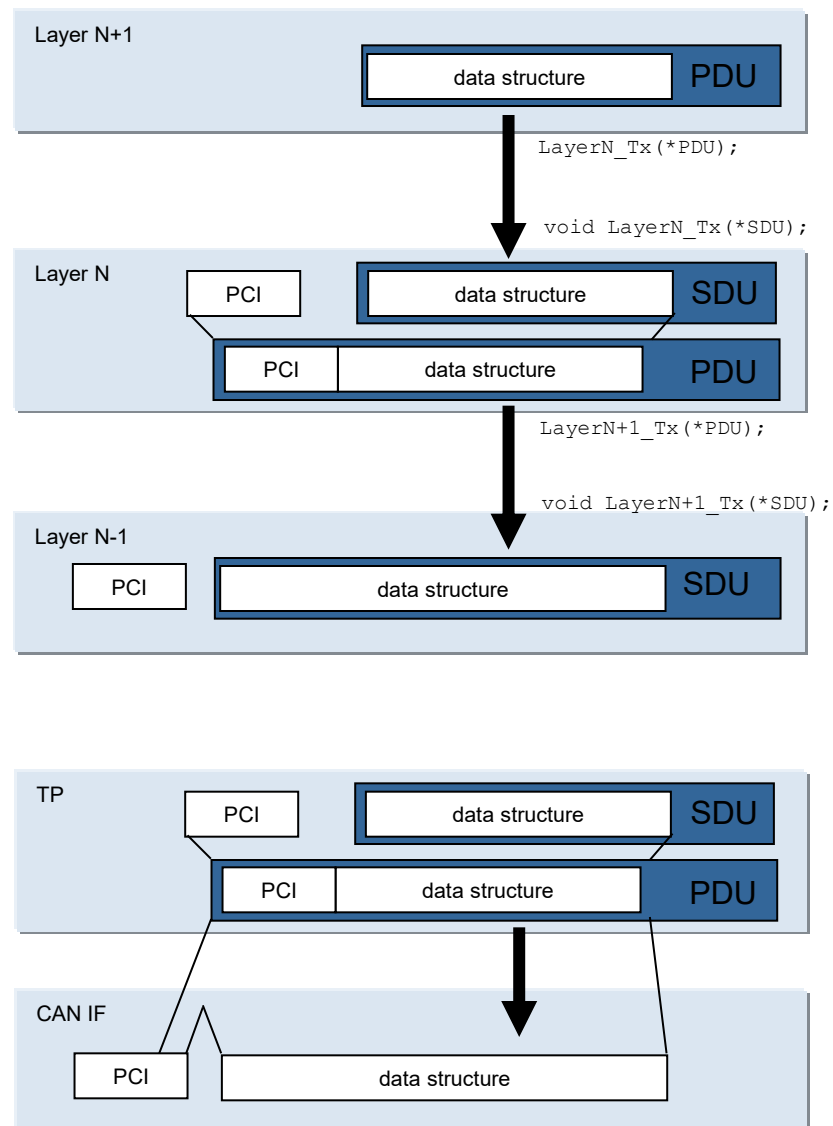
PCI是“协议控制信息”的缩写。此信息用于将SDU从一个特定协议层的实例传递到另一个实例。例如，它包含源和目标信息。

PCI由传输端的协议层添加，并在接收端再次移除。

协议数据单元

PDU是“协议数据单元”的缩写。PDU包含SDU和PCI。

在传输侧，PDU从上层传递到下层，下层将此PDU解释为其SDU。

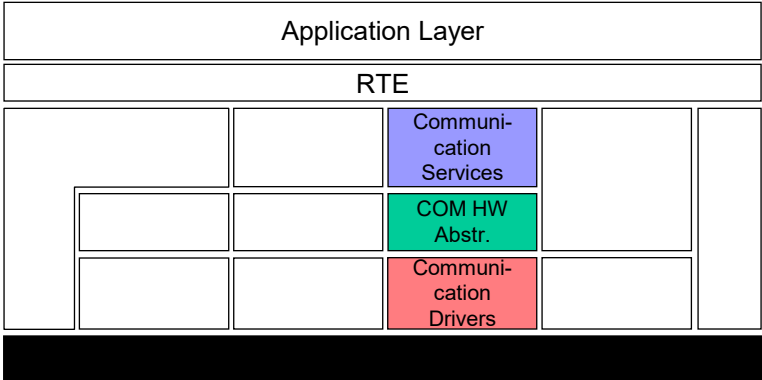


Interfaces: Interaction of Layers

示例“沟通”(1)

SDU与PDU命名规范

PDU和SDU的命名遵循以下规则： 对PDU: <bus prefix>
<layer prefix> - PDU 对SDU: <bus prefix> <layer prefix> - SDU
U bus prefix和layer prefix在下表中描述:



ISO Layer	Layer Prefix	AUTOSAR Modules	PDU Name	CAN prefix	LIN prefix	FlexRay prefix
Layer 6: Presentation (Interaction)	I	COM, DCM	I-PDU	N/A		
	I	PDU router, PDU multiplexer	I-PDU	N/A		
Layer 3: Network Layer	N	TP Layer	N-PDU	CAN SF CAN FF CAN CF CAN FC	LIN SF LIN FF LIN CF LIN FC	FR SF FR FF FR CF FR FC
Layer 2: Data Link Layer	L	Driver, Interface	L-PDU	CAN	LIN	FR

SF: Single 框架
FF: First 框架
CF: Cons执行 Frame
FC: Flow有关帧

请参见适用于CAN、LIN和FlexRay的AUTOSAR传输协议规范

示例: I-PDU 或 I-SDU CAN FF
N-PDU 或 FR CF N-SDU LIN L-P
DU 或 FR L-SDU



组件

PDU路由器：

- 在不同抽象通信控制器与上层之间提供PDU的路由
- 路由器的规模取决于ECU（例如，如果仅存在一个通信控制器，则可缩减至零尺寸）
- 提供动态TP路由。在完整TP数据缓冲完成前即开始传输TP数据

COM： ■ 在不同I-PDU之间提供独立信号或信号组的路由

NM协调器： ■ 通过NM协调器处理的网络管理，同步连接到ECU的不同通信通道的网络状态

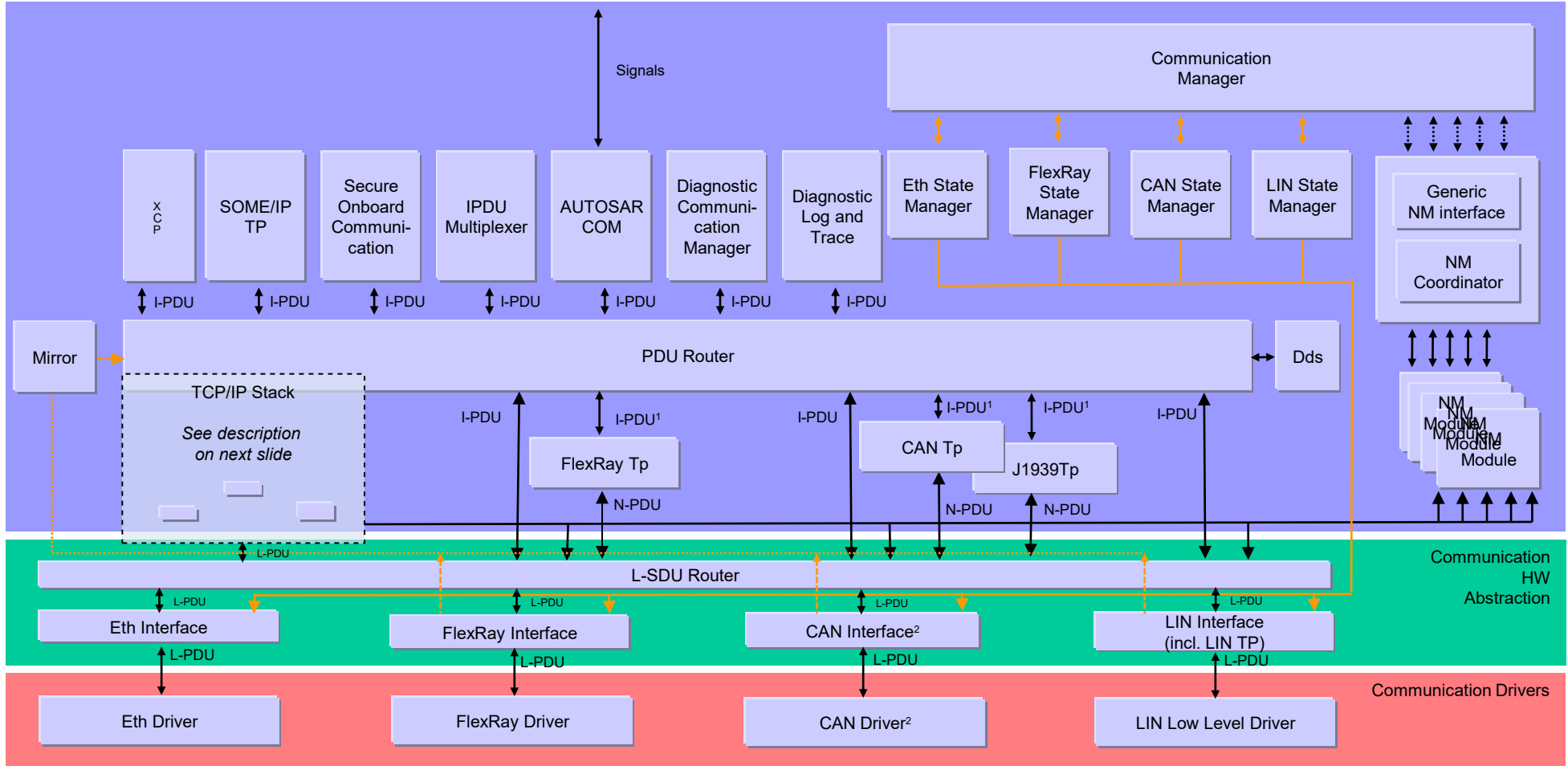
通信状态管理器： ■ 通过接口启动和关闭通信系统的硬件单元

■ 控制PDU组

Interfaces: Interaction of Layers

示例“沟通” (3)

RTE



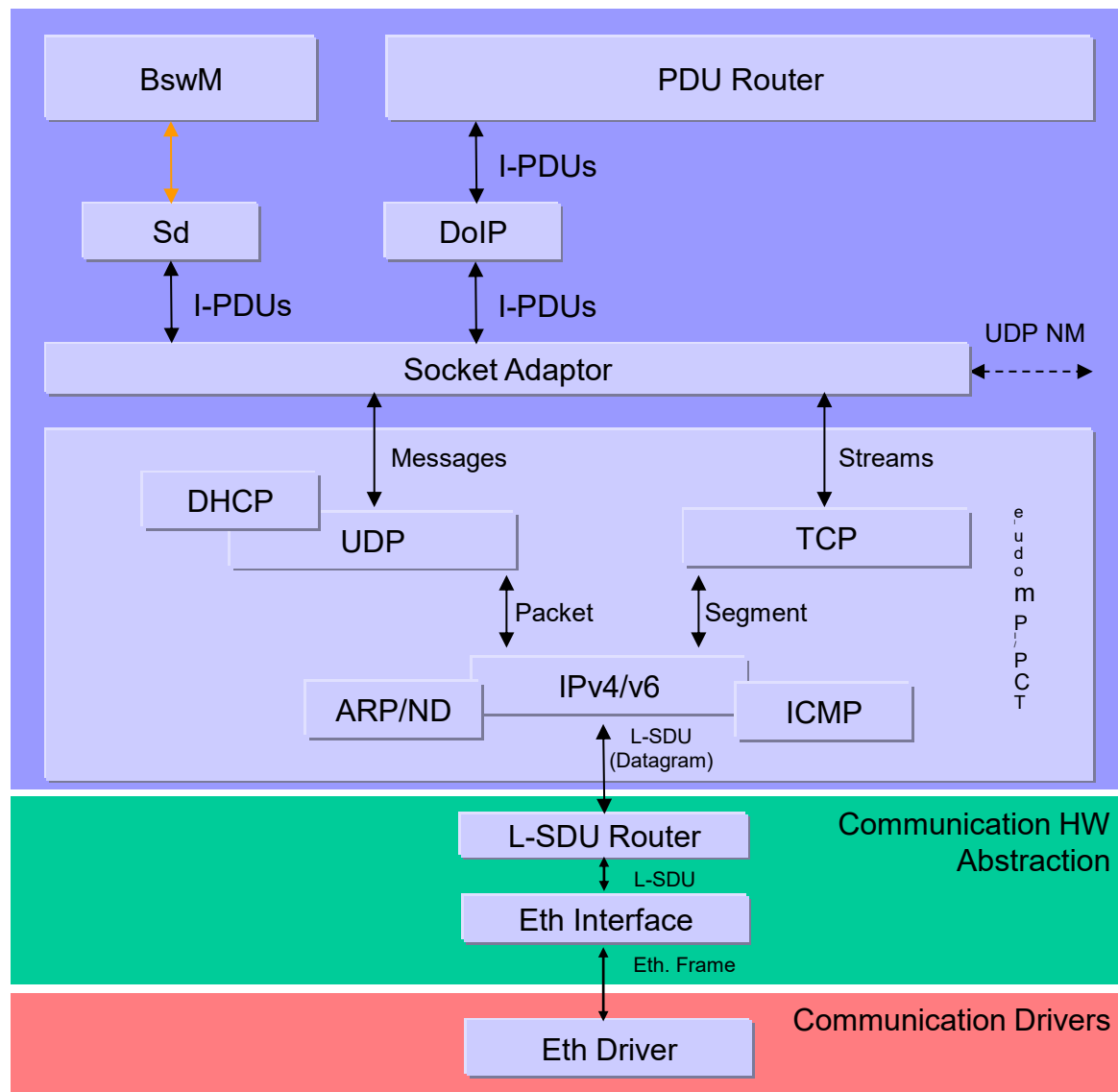
Note: This image is not complete with respect to all internal communication paths.

¹ The Interface between PduR and Tp differs significantly compared to the interface between PduR and the Ifs. In case of TP involvement a handshake mechanism is implemented allowing the transmission of I-PDUs > Frame size.

Interfaces: Interaction of Layers

示例“通信”（4）——以太网协议栈

该图展示了以太网栈内部和之间的交互。

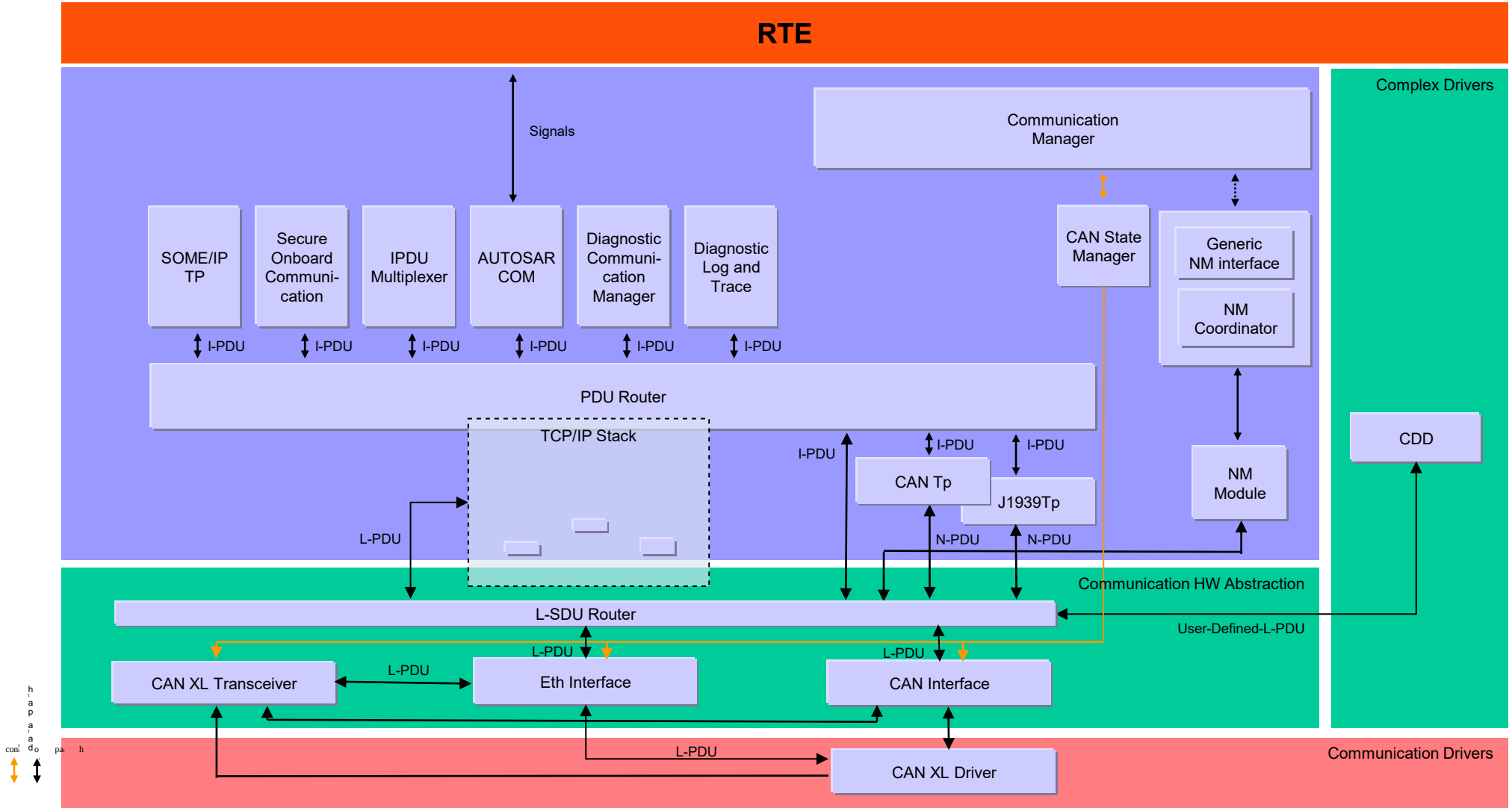


Interfaces: Interaction of Layers

示例“Communi 通信”(5) - 使用以太网和CAN通信

g CAN XL

8
d xmd w



注意：此图像并未完整展示所有内部通信路径。

Interfaces: Interaction of Layers

示例“数据转换”（1）—— 引言

以下页面解释了与数据转换的通信：

软件各层之间如何交互？

软件界面看起来是怎样的？

Interfaces: Interaction of Layers

示例“数据转换”

(2) ——示例与初步了解

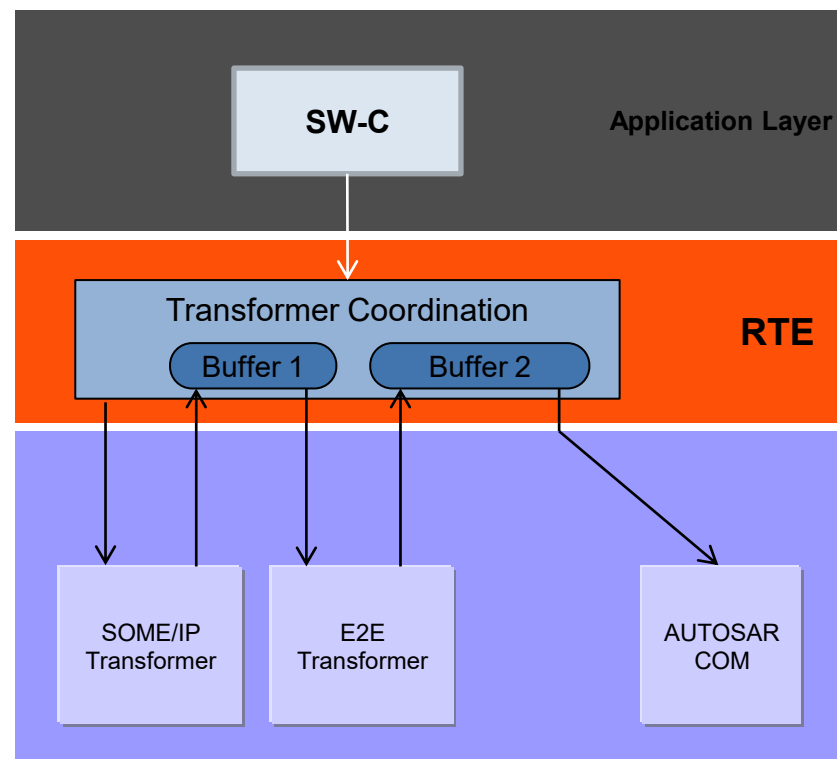
此示例展示了在ECU间通信中使用数据转换时的数据流。

一个软件组件（SW-C）发送配置为传输至远程ECU并需进行数据转换的数据。此数据转换不使用就地缓冲区处理。

功能性

RTE将SOME/IP转换器作为链中的首个转换器调用，并从SW-C传输数据。SOME/IP转换器执行转换并

将输出（字节数组）写入由RTE提供的缓冲区中。随后，RTE执行位于转换器链中第二位的安全转换器。安全转换器的输入是SOME/IP转换器的输出。安全转换器对数据进行保护，并将输出写入RTE提供的另一个缓冲区。由于未使用原地缓冲区处理，因此需要新的缓冲区。RTE将最终输出数据作为字节数组传输至COM模块。



Interfaces: Interaction of Layers

示例“数据转换”（3）——深入探讨接口

架构描述

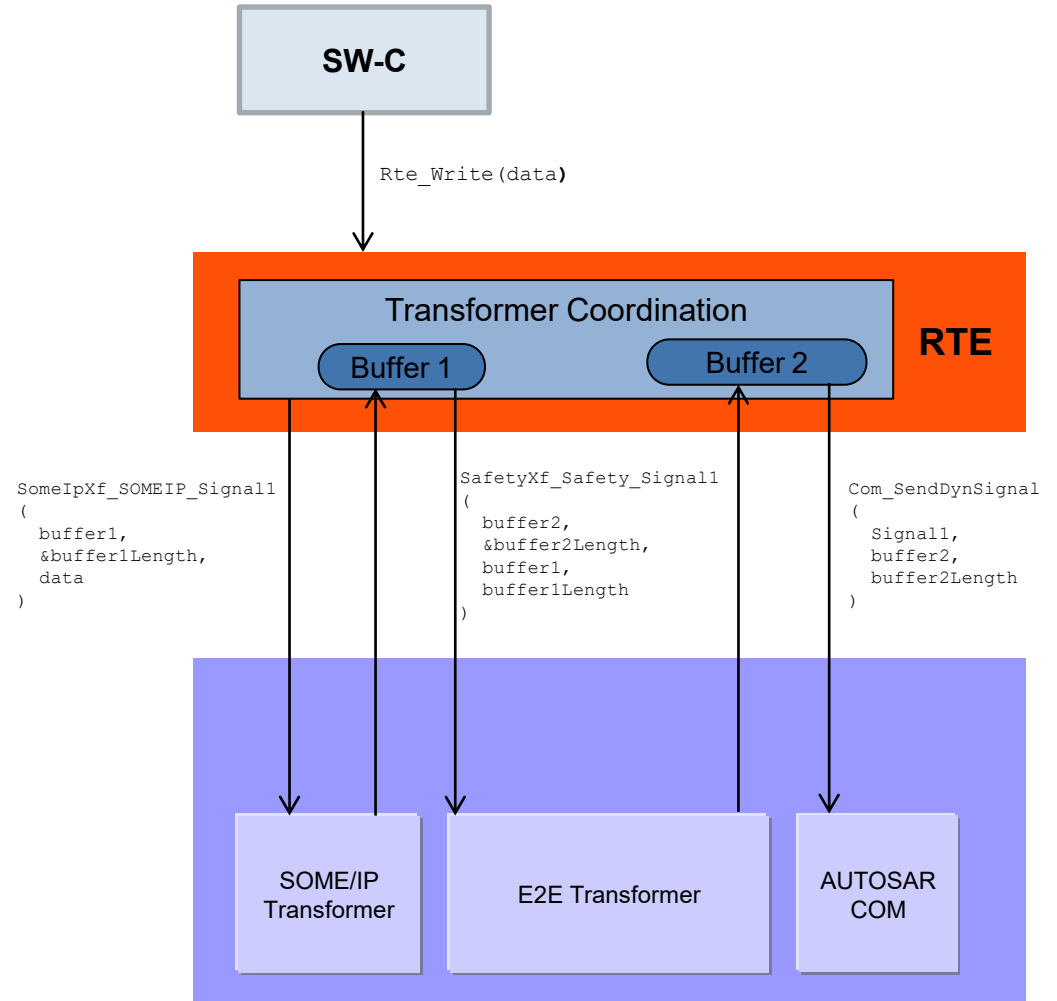
TRTE使用位于系统服务层中的变压器。

接口描述

本例中的变压器具有以下接口：

```
SomeIpXf_SOMEIP_Signal1 ( uint8  *buffer1, uint16 *buffer1Length, <type> data )
```

```
SafetyXf_安全信号1  
( uint8  *buffer2, uint16 *buffer2Length, uint8 *buffer1, uint16 buffer1Length )
```



Interfaces: Interaction of Layers

示例“数据转换”（4）——基于COM的转换

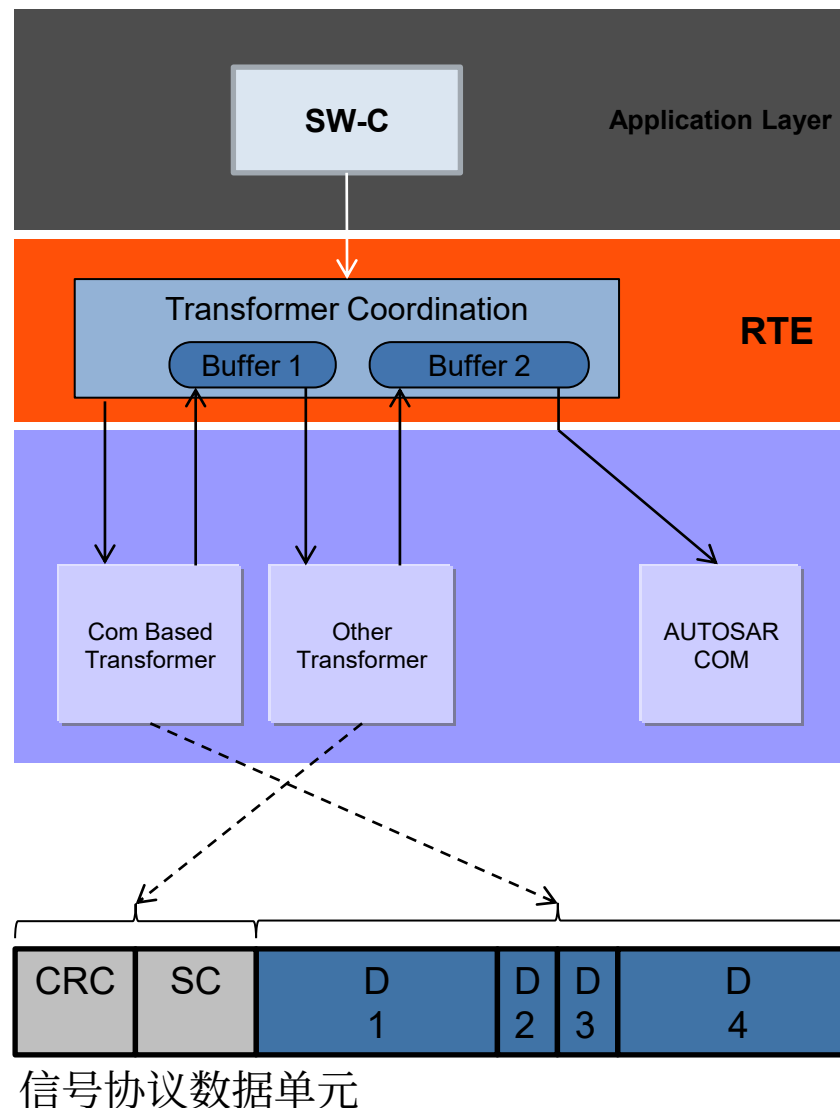
目标

COM Based Transformer 基于固定的通信矩阵为变压器链提供序列化功能。

固定的通信矩阵允许将信号优化地放置到PDU中（例如，布尔数据可配置为仅占用PDU中的一位）。这使得在Can或Lin等低负载网络中能够使用转换器链。

功能性

COM Based Transformer 是第一个转换器（序列化器），通过RTE从应用程序获取数据。基于COM配置（通信矩阵），数据以与COM模块完全相同的方式进行序列化（字节序、符号扩展）。其他转换器可增强有效载荷以包含CRC和序列计数器（SC）。转换器有效载荷通过Com_SendSignalGroupArray API以字节数组形式传递给COM模块。COM模块可配置为基于通信矩阵定义执行传输模式选择。



Interfaces: Interaction of Layers

信号-服务-翻译 (1)

目标

自适应平台将通信限制在面向服务的通信，然而车辆的其他部分仍使用基于信号的通信方式——因此，为了允许经典平台与自适应平台之间的交互，必须对这两种通信方式进行转换。

功能性

经典平台信号服务转换的定义和实现应在应用软件组件内部完成，即所谓的转换软件组件。

翻译软件组件定义了端口，并且使用端口接口描述有效载荷。

- 信号到服务：用于传入信号的端口和用于传出事件的端口
- 服务到信号：用于传入事件的端口和用于传出信号的端口



Interfaces: Interaction of Layers

信号-服务-翻译 (2)

功能性

对于基于信号的部分，经典平台COM-Stack的全部功能均可使用，并可配置为使得基于信号的ISignalIPdu可源自多种来源（Can、Lin、FlexRay），且ISignalIPdu可受到安全与防护保护。对于面向服务的部分，必须确保所定义的SOME/IP服务实际上与自适应平台兼容。这适用于有效载荷部分（例如，SOME/IP序列化器必须能够

以及使用BswM和ServiceDiscovery的控制路径。翻译软件组件本身的行为部分定义了如何将来自基于信号侧的数据传输到面向服务侧，反之亦然。

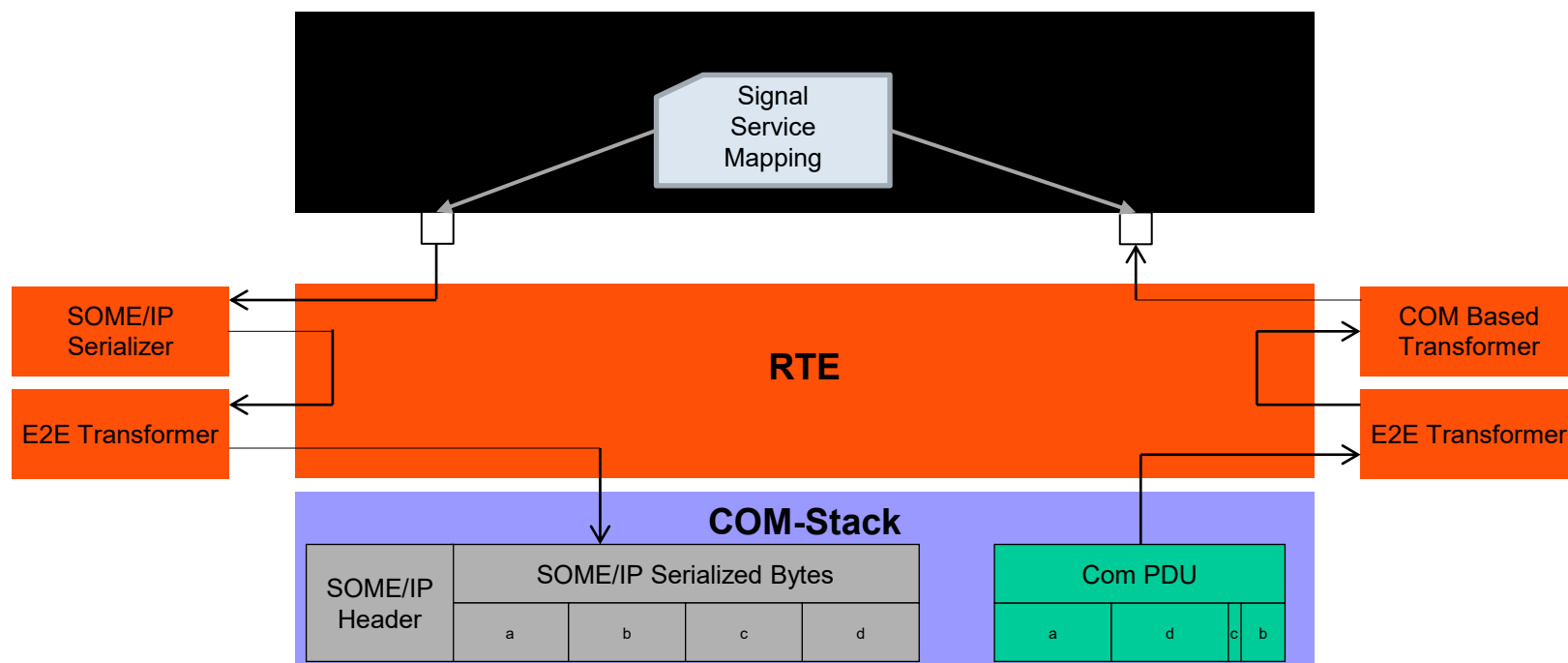


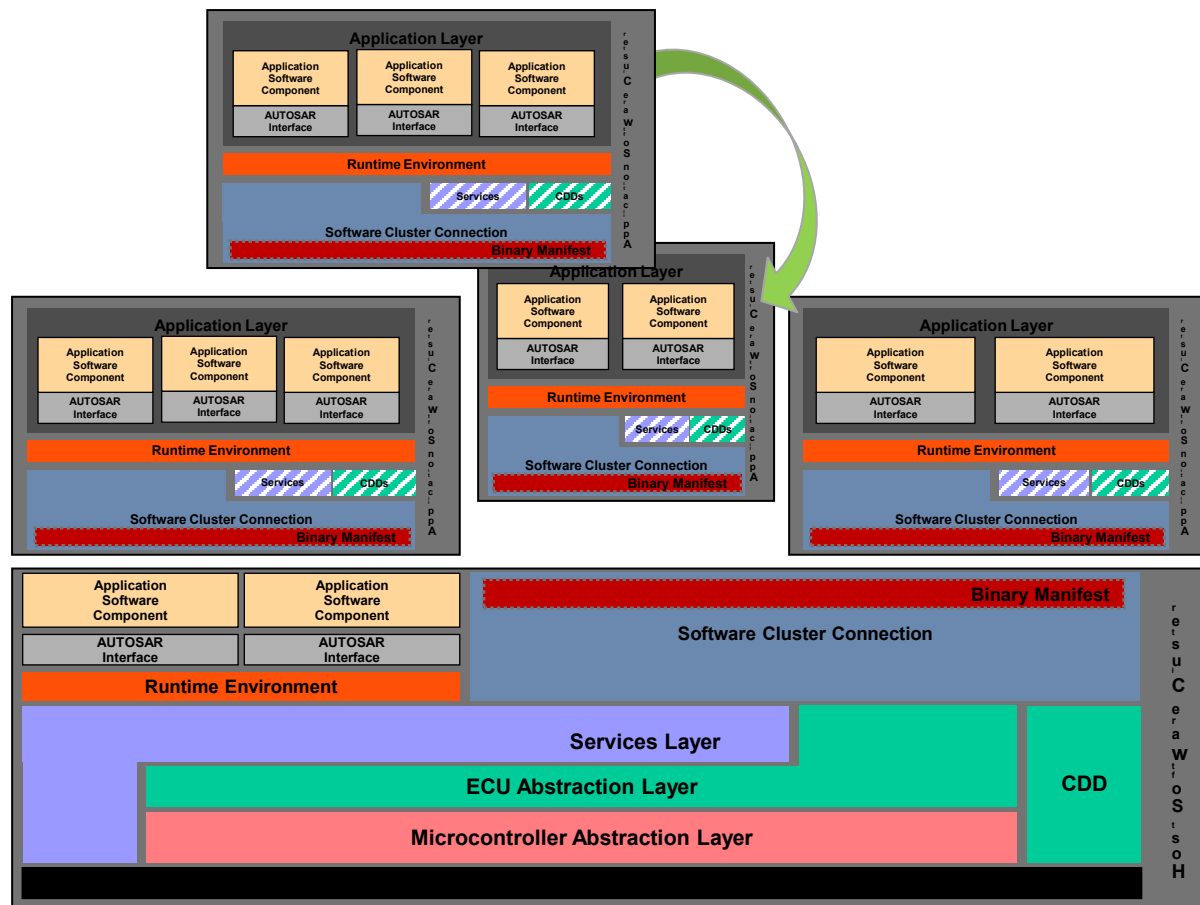
Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置 3. 集成与运行时方面

The approach in a nutshell



软件集群能够将单一的传统平台架构拆分为更小的单元

每个CP软件集群均可独立构建

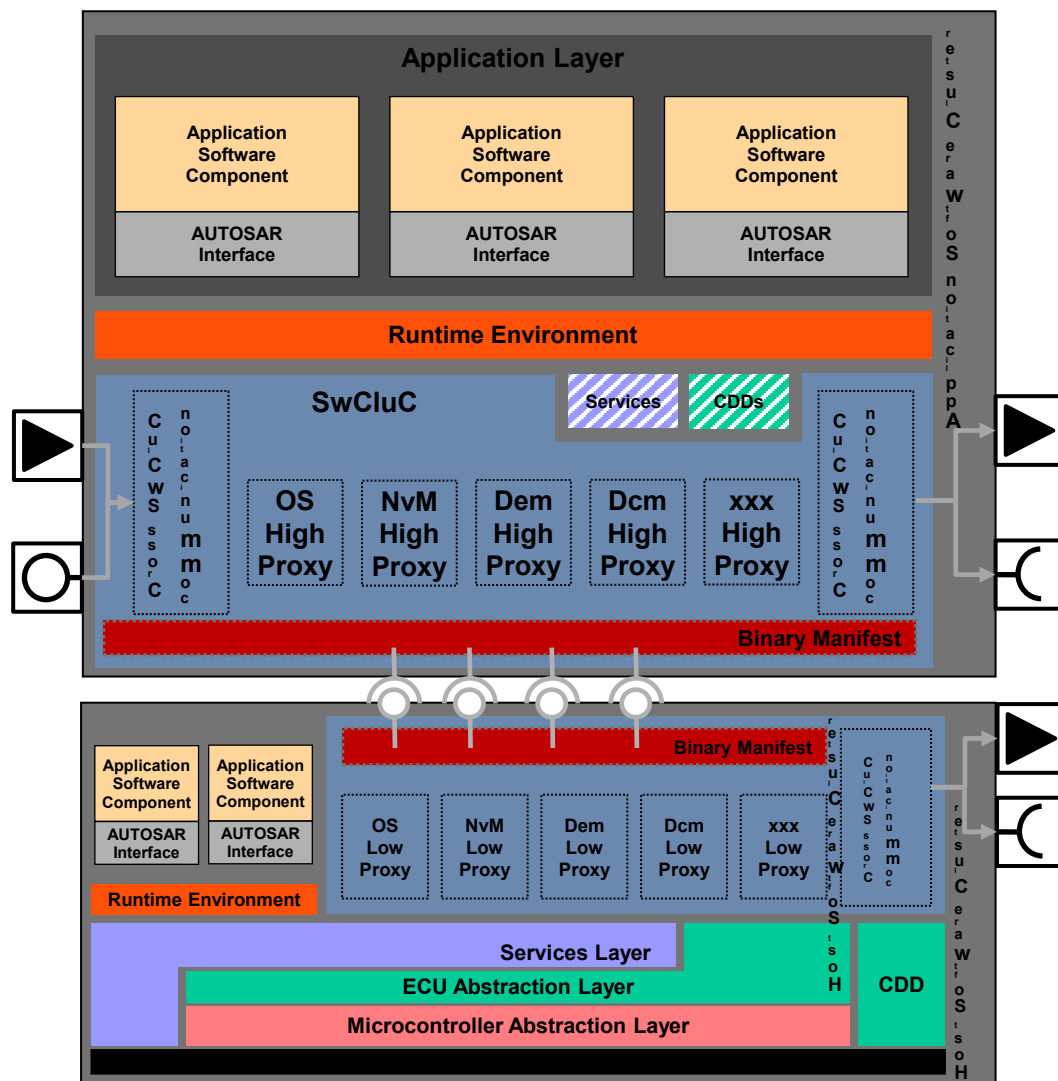
软件集群可独立更新

软件集群之间的连接基于二进制对象及二进制清单中所包含的信息而建立。

考虑到当前微控制器架构的局限性，例如缺乏地址虚拟化功能

在 *Application Software Cluster* 中，应用软件组件和基础软件模块（有限制地）可被集成

Host Software Cluster 包含了BSW堆栈的主要部分，特别是依赖于微控制器的模块，包括操作系统。



软件集群连接模块 (SwCluC) 包含3个部分:

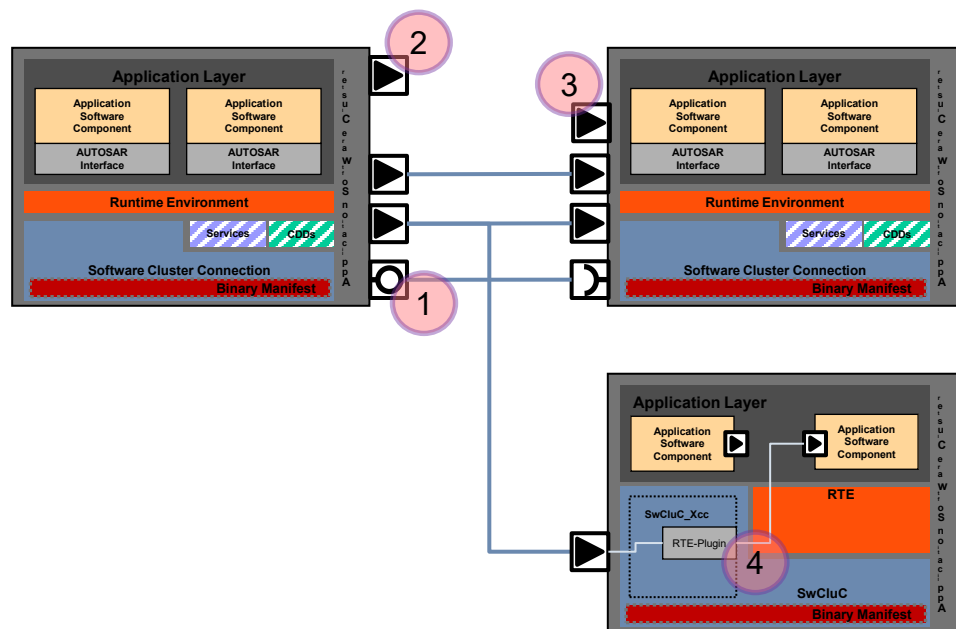
跨软件集群通信 (SwCluC_Xcc) 提供经典平台中的功能

- 实现基于二进制清单■的软件集群连接，以支持软件集群间的跨交互与通信。

在相应的代理模块中对非软件集群本地BSW模块及其API进行抽象

- 高代理替代非本地基础软件 (BSW) 并提供相应的API ■，低代理模块则连接到主机软件集群的常规BSW模块。

二进制清单 (BManif) 为接口提供二进制元信息，以便能够连接软件集群。



软件集群连接 (SwCluC) 实现了接口的灵活处理

- 接口将通过链接过程进行连接，该过程基于二进制清单以及所需条目与提供条目之间的匹配■。若找到匹配项，则建立连接■。若未找到请求方，接口将保持开放状态■。若未找到提供方，接口将保持开放，并提供默认值■。这使得软件集群能够随接口变更而更新

跨软件集群通信

(SwCluC_Xcc) 实现了通信模式以及与RTE的接口

- RTE接口：RIPS-插件

Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4. 混合关键系统中的软件层内容 5. 模块概述 6. 接口 1. 概述 2. 层间交互（示例）

2. 配置

3. 集成与运行时方面

Configuration Overview

AUTOSAR基础软件支持以下配置类别：

1. 预编译时间

- 预处理器指令 ■ 代码生成（选择或合成）

2. 链接时间

- 模块外部的常量数据；该数据可以在模块编译后进行配置

3. 构建后时间

- 模块外可加载的常量数据。与[2]非常相似，但数据位于允许重新加载的特定内存段中（例如在ECU生产线上重新刷写）

独立于配置类别，可以通过变点提供单个或多个配置集。若提供了多个配置集，当变点在运行时绑定时，实际使用的配置集需在运行时选择。

在许多情况下，一个模块的配置参数会属于不同的配置类别。例如：一个提供构建后时间配置参数的模块，仍可能包含一些支持预编译时间配置参数。

注意：在AUTOSAR 4.1.x版本之前，多个配置集被建模为构建后配置类的一个子类。

Configuration

Pre-compile time (1)

使用案例

预编译时配置将被选择用于

启用/禁用可选功能 这允许排除不需要的源代码部分 优化性能和代码大小 使用 `#defines` 在大多数情况下能生成比访问常量（甚至通过指针访问常量）更高效的代码。生成的代码避免了代码和运行时开销。

限制条件

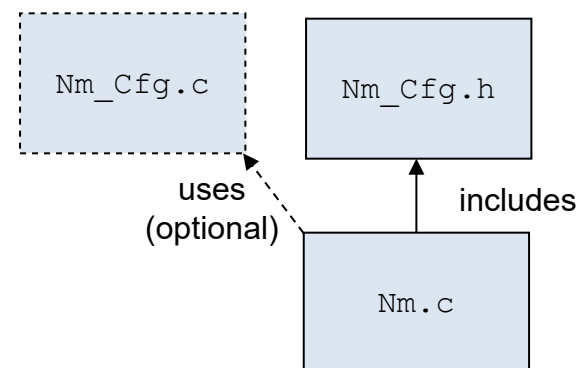
模块必须提供源代码形式。

配置是静态的，它可能包含一个或多个通过变点（variation points）标识的配置集。要更新任何配置集（例如更改某些参数的值），必须重新编译该模块。

所需实现

预编译时配置应通过模块的两个配置文件(*_Cfg.h, *_Cfg.c)和/或代码生成来完成：

- *_Cfg.h 存储例如宏和/或 `#defines`
- *_Cfg.c 存储例如常量



Configuration

Pre-compile time (2)

示例1: 启用/禁用功能 文件 Spi_Cfg.h: #define
SPI_DEV_ERROR_DETECT ON

文件 Spi_Cfg.c:
const uint8 我的常量 = 1U;

文件 Spi.c (提供源代码) : #include "Spi_Cfg.h" /* 用于导入配置参数 */

```
extern const uint8 myconstant;
```

```
#if (SPI_DEV_ERROR_DETECT == ON) Det_ReportError(Spi_ModuleId, 0U, 3U, SPI_E_PARAM_LENGTH); /* 仅有一个实例可用  
*/ #endif
```

Note: The Memory Abstraction (as specified by AUTOSAR) is not used to keep the example simple.

Configuration

Pre-compile time (3)

示例2: 报告给Dem的事件ID

NVRAM管理器的XML配置文件:

指定其需要事件符号NVM_E_REQ_FAILED用于生产错误报告。

文件 Dem_Cfg.h (由 Dem 配置工具生成) :

```
typedef uint8 Dem_EventIdType; /* 事件总数 = 46 => uint8 足够 */
```

```
#define DemConf_DemEventParameter_NVM_E_REQ_FAILED    5U#define DemC  
onf_DemEventParameter_CANSM_E_BUS_OFF    6U...
```

文件 Dem.h:

```
#include "Dem_Cfg.h" /* 用于提供对事件符号的访问 */
```

文件 NvM.c (提供源代码形式) : #include "Dem.h" /* 用于报告生产错误 */

```
Dem_SetEventStatus (DemConf_DemEventParameter_NVM_E_REQ_FAILED, DEM_EVENT_STATUS_PASSED);
```

Configuration

Link time (1)

使用案例

链接时配置将被选择用于

仅以目标代码形式提供的模块配置（例如，出于知识产权保护或保修原因）

编译后但链接前创建配置。

所需实现

1. 一个配置集，无运行时选择 配置数据应通过外部常量获取。这些外部常量位于独立的文件中。模块可直接访问这些外部常量。
2. 2..n 配置集，支持运行时选择 配置数据应在外部常量结构体中捕获。模块在初始化时获取指向其中一个结构体的指针。每次初始化时均可选择不同的结构体。

Configuration

Link time (2)

示例1：报告给演示的实体ID仅作为对象代码可用，例如“Foo”模块，这是一个多次实例化的模块。

Foo模块的XML配置文件：指定其需要事件符号FOO_E_WRITE_FAILED用于生产错误报告。

文件 Dem_Cfg.h（由 Dem 配置工具生成）：

```
typedef uint16 Dem_EventIdType; /* 事件总数 = 380 => uint16 必需 */
```

```
#define DemConf_DemEventParameter_FOO_E_ERASE_FAILED_0 1U#define DemConf_DemEventParameter_FOO_E_ERASE_FAILED_1 2U#define DemConf_DemEventParameter_FOO_E_WRITE_FAILED_0 3U#define DemConf_DemEventParameter_FOO_E_WRITE_FAILED_1 4U#define DemConf_DemEventParameter_NVM_EREQ_FAILED 5U#define DemConf_DemEventParameter_CANSME_BUS_OFF 6U
```

...

文件 Foo_Lcfg.c：

```
#include "Dem_Cfg.h" /* 用于提供对事件符号的访问 */
```

```
const Dem_EventIdType Foo_WriteFailed[2] = {DemConf_DemEventParameter_FOO_E_WRITE_FAILED_1, DemConf_DemEventParameter_FOO_E_WRITE_FAILED_2};
```

文件 Foo.c（以目标代码形式提供）：

```
#include "Dem.h" /* 用于报告生产错误 */ /*  
extern const Dem_EventIdType Foo_WriteFailed[];
```

```
Dem_SetEventStatus(Foo_WriteFailed[instance], DEM_EVENT_STATUS_FAILED);
```

注意：为了保持示例简洁，此处未展示包含所有前向声明的完整头文件结构。

Configuration

链接时间 (3)

示例2：由仅以目标代码形式提供的模块（Flash Driver）报告给Dem的事件ID

问题

Dem_EventIdType 也根据该ECU上事件ID的总数生成。在此示例中，它表示为 uint16。Flash Driver 使用此类型，但仅以目标代码形式提供。

解决方案

在在ECU开发的合同阶段，必须为每个ECU确定并分配一系列变量类型（包括Dem_EventIdType）。目标代码供应商必须使用这些类型进行编译，并交付使用正确类型的对象代码。

Configuration

Post-build time (1)

使用案例

构建后时间配置将被选择用于

数据配置，其中仅定义了结构，但在ECU构建时内容未知

可能在ECU构建后需要更改或调整的数据配置（例如，生产线末端、测试与标定期间）

ECU在不同车型版本间的可重用性（相同应用，不同配置），例如低成本车型中的ECU在总线上传输的信号可能比豪华车型中的同一ECU更少。

限制条件

实现需要在可擦写区域存储所有可能相关的配置项，并在访问配置时进行指针解引用。该实现方式排除了代码生成，这对性能、代码及数据大小均有影响。

所需实现

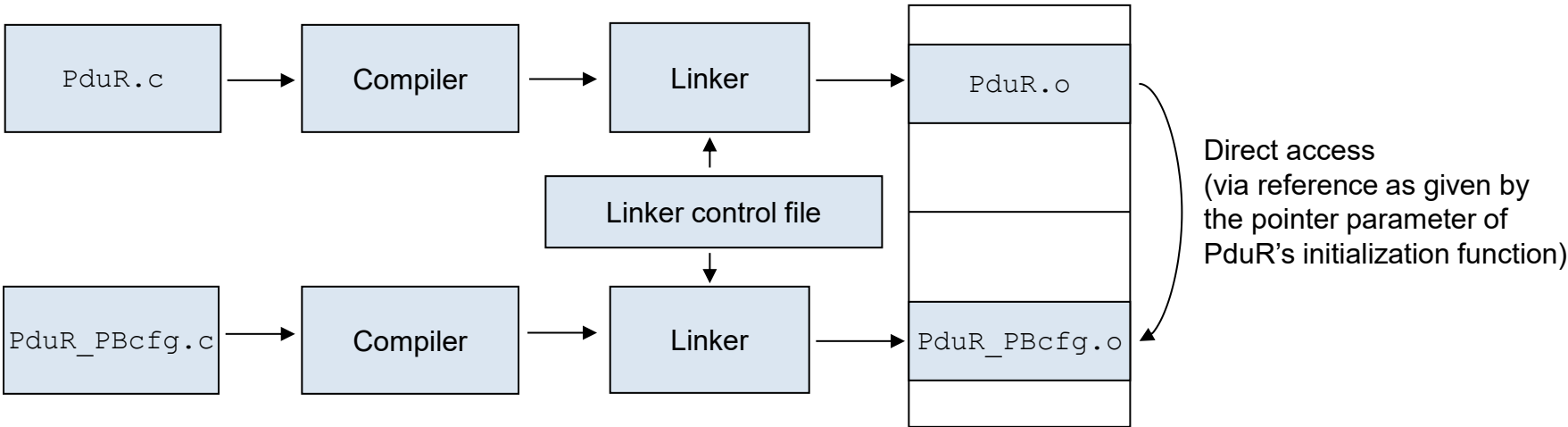
1. 一个配置集，无运行时选择 配置数据应存储在外部常量结构体中。这些外部结构体位于可单独重载的独立内存段中。模块在初始化时获取指向基础结构体的指针。
2. 2..n 配置集，支持运行时选择 配置数据应在外部常量结构体中捕获。这些外部结构体位于独立的内存段中，可单独重新加载。模块在初始化时获取指向多个基础结构体之一的指针。每次初始化时均可选择不同的结构体。

Configuration

构建后时间 (2)

示例 1

如果配置数据在内存大小和位置上固定，模块可以直接访问这些外部结构。



Configuration

构建后时间 (3)

必需实现2: 仅以目标代码形式提供的CAN驱动程序的配置；在初始化期间可以从多个配置集中选择一个配置集。

File Can_PBcfg.c:

```
#include "Can.h" /* for getting Can_ConfigType */
const Can_ConfigType MySimpleCanConfig [2] =
{
    {
        Can_BitTiming      = 0xDF,
        Can_AcceptanceMask1 = 0xFFFFFFFF,
        Can_AcceptanceMask2 = 0xFFFFFFFF,
        Can_AcceptanceMask3 = 0x00034DFF,
        Can_AcceptanceMask4 = 0x00FF0000
    },
    { ... }
};
```

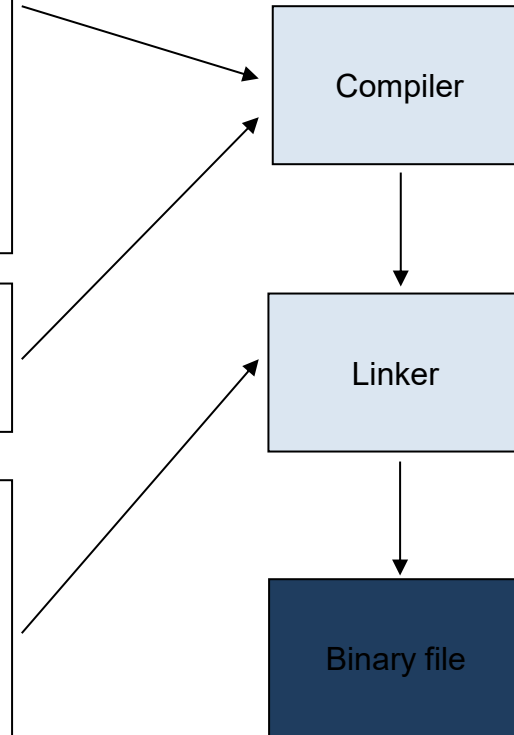
File EcuM.c:

```
#include "Can.h" /* for initializing the CAN Driver */
Can_Init(&MySimpleCanConfig[0]);
```

File Can.c (available as object code):

```
#include "Can.h" /* for getting Can_ConfigType */

void Can_Init(Can_ConfigType* Config)
{
    /* write the init data to the CAN HW */
};
```



Configuration

变体

不同的使用场景需要不同种类的可配置性。因此，以下配置变体是

提供：
变体-预编译 此变体中仅允许使用具有“预编译时”配置参数。 变体-链接时 此变体中仅允许使用具有“预编译时”和“链接时”的参数。 变体-构建后 此变体中允许使用具有“预编译时”、“链接时”和“构建后时”的参数。

示例用例：

网关中可重新编程的PDU路由表（需支持构建后配置的PDU路由器） 静态配置的PDU路由，无额外开销（需支持预编译时配置的PDU路由器）

为了在每个BSW模块中实现这些不同的用例，最多可以指定3种变体：

变体是指将模块的配置参数专门分配给配置类的一种特定分配方式。在一个变体中，一个配置参数只能分配给一个配置类。在一个变体中，不同配置参数的配置类可以不同（例如：开发错误检测采用预编译配置，而可重编程PDU路由表采用构建后配置）。某些特定配置参数在所有变体中均可分配至相同的配置类，这是可能且有意设计的（例如：开发错误检测通常可在预编译时配置）。

Configuration

内存布局示例：构建后配置

EcuM 定义了索引：

0x8000	&index (=0x8000)
0x8000	&xx_configuration = 0x4710
0x8002	&yy_configuration = 0x4720
0x8004	&zz_configuration = 0x4730
...	

Xx定义了模块的配置数据：

0x4710	&the_real_xx_configuration
0x4710	lower = 2
0x4712	upper =7
0x4714	more_data
...	

Yy定义了模块的配置数据：

0x4720	&the_real_yy_configuration
0x4720	Xx_data1=0815
0x4722	Yy_data2=4711
0x4724	more_data
...	

描述在哪里找到总体协议的内容：

1. EcuM需要知道包括索引在内的所有地址。
2. 模块（xx、yy、zz）需要知道各自的起始地址：在本例中为0x4710、0x4720……
3. 起始地址可能是动态的，即会随新配置而变化。
4. 初始化模块（例如xx、yy、zz）时，EcuM会将配置数据的基地址（例如0x4710、0x4720、0x4730）传递给模块，以适应配置数据大小的变化。

模块数据仅在本地（模块内）达成一致

1. 模块（xx,yy）知晓自身的起始地址（以便实现者能够分配数据段）
2. 仅模块（xx,yy）知晓其自身配置的内部细节

0x8002 &yy配置 = 0x4720 0x8004 &zz配置 = 0x4730 ... 0x8008
&xx配置 = 0x5000 0x800a &yy配置 = 0x5400 0x800c &zz配置
= 0x5200 ... 0x8010 &xx配置 = ... 0x8012 &yy配置 = ... 0x801
4 &zz配置 = FL FRRL

1. 索引中包含多个描述 (FL、FR等) 组成的数组 (此处约定每个数组元素的大小为8)
2. 存在一个约定变量, 用于存储描述选择器 = CheckPinCombination() 的位置
3. 不直接传递指针, 而是通过一次间接引用: (struct Ec uM_ConfigType *) &index[selector];
4. 其余机制与传统单一配置情况相同。

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射 2. 分区管理 3. 调度机制 4. 模式管理 5. 错误处理、报告与诊断 6. 测量与标定 7. 安全机制 8. 能源管理 9. 全局时间同步

Integration and Runtime Aspects

可运行对象的映射

可运行实体是软件组件的活动部分。它们可通过映射到不同任务实现并发执行。图示进一步展示了在此映射过程中需考虑的其他实体，如操作系统应用、分区、微控制器内核及基础软件资源。

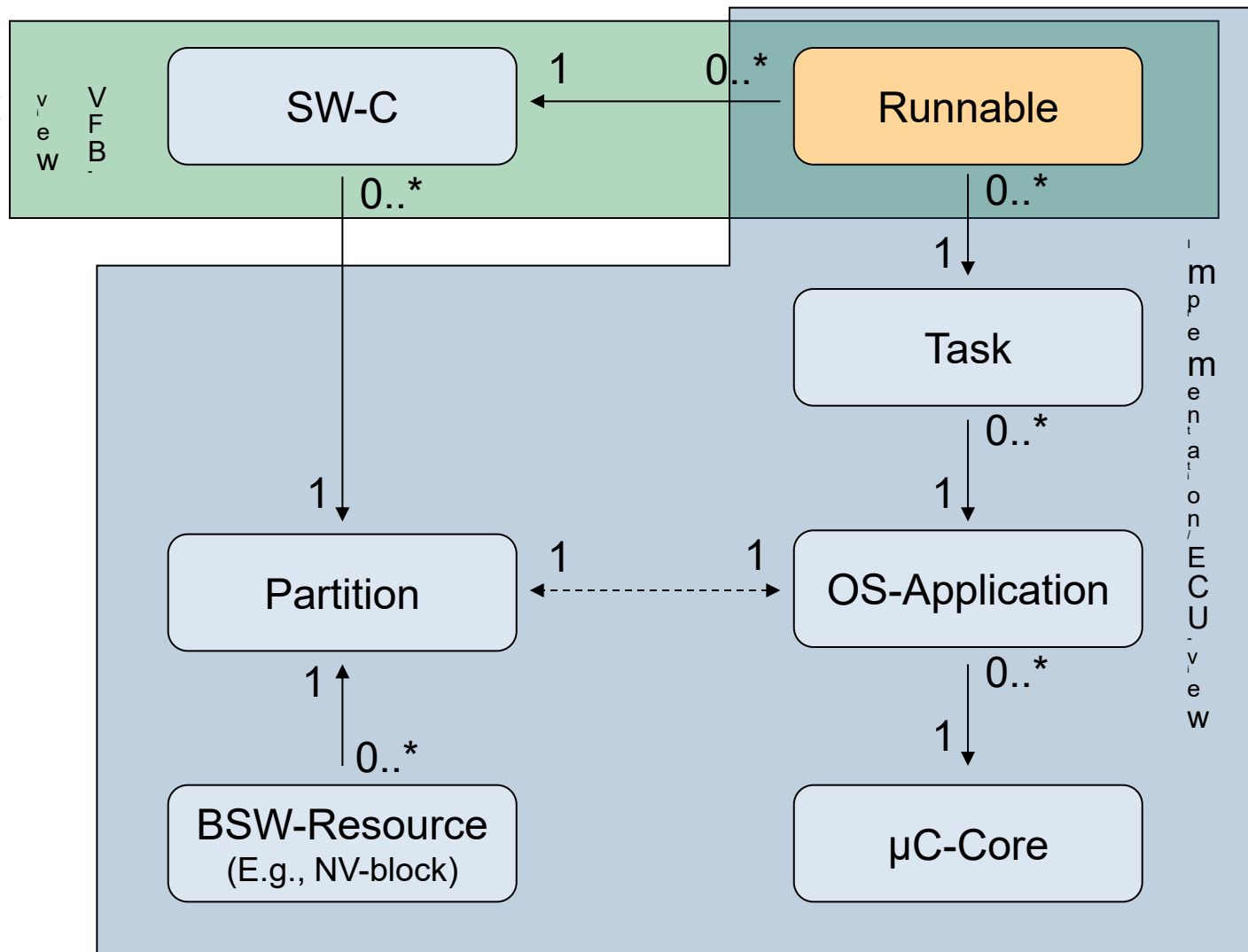


Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

分区是通过在操作系统内使用OS-Applications来实现的。

操作系统应用程序被用作错误隔离区域： ■ 允许对软件组件和资源进行逻辑分组 ■ 为每个操作系统应用程序单独定义恢复策略

OS-应用一致性由系统/平台确保，例如针对： ■ 内存访问违规 ■ 时间预算违规

OS-应用程序在运行时可能因检测到错误而被终止：

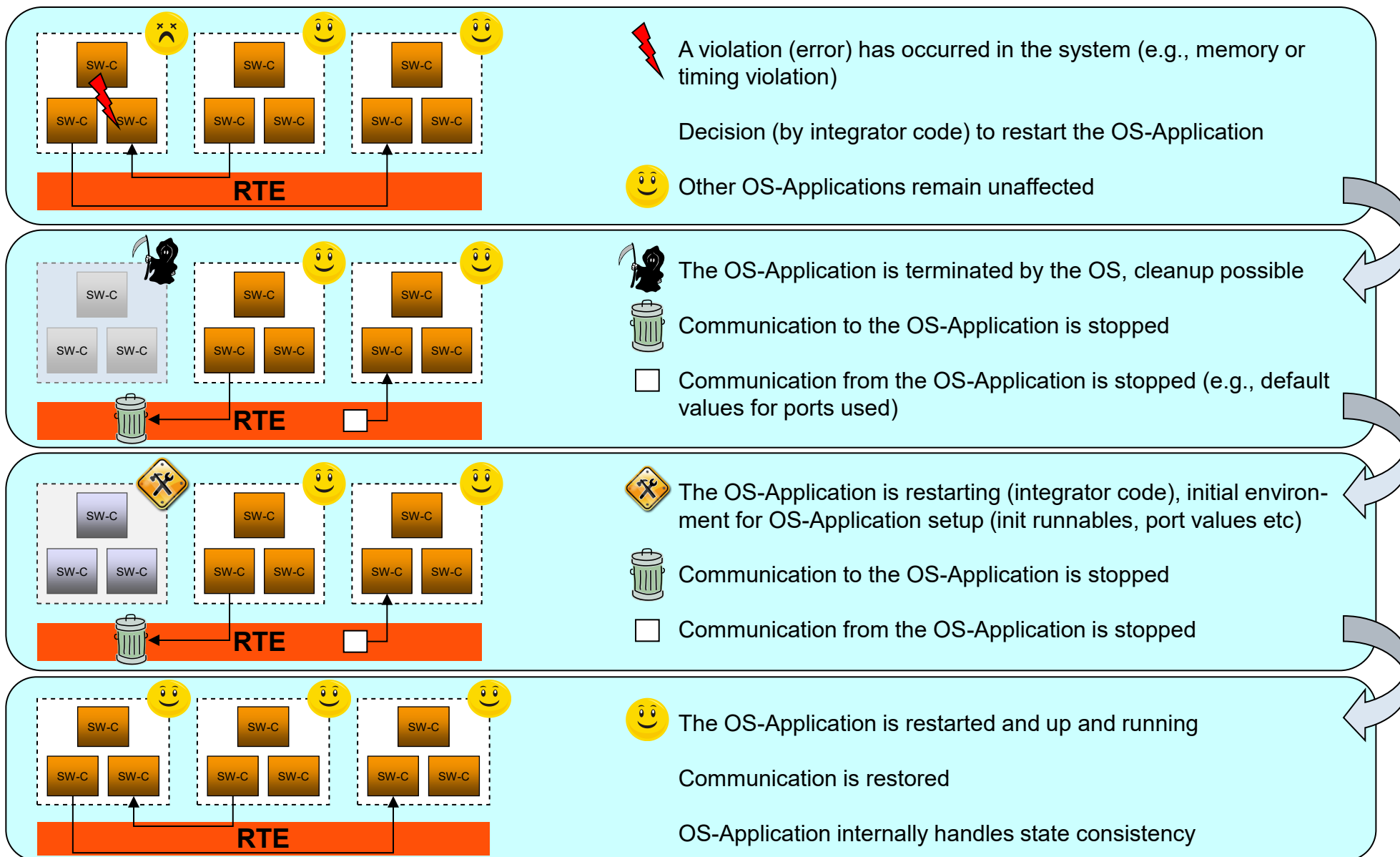
- 所有BSW模块都放置在特权操作系统应用程序■中，这些操作系统应用程序不应被终止。

OS-应用程序在ECU配置中配置： ■ 个SW-C被映射到OS-应用程序（结果：限制了可运行实体到任务的映射）

跨操作系统-应用程序边界的通信通过IOC实现

Integration and Runtime Aspects - Partitioning

Example of restarting OS-Application



Integration and Runtime Aspects - Partitioning

Involved components

保护钩

- 在保护违规（内存或时序）时执行
- 决定采取何种行动（终止、重启、关机、不操作）
- 由集成商提供
- 操作系统通过检查返回值来执行决策

OsRestartTask

- 由操作系统启动，以防保护钩返回重启
- 由集成商提供
- 在操作系统应用程序的上下文中运行，并启动必要的清理和重启活动，例如：
 - 停止通信（ComM）
 - 更新NvM
 - 通知看门狗、CDD等。

实时评估

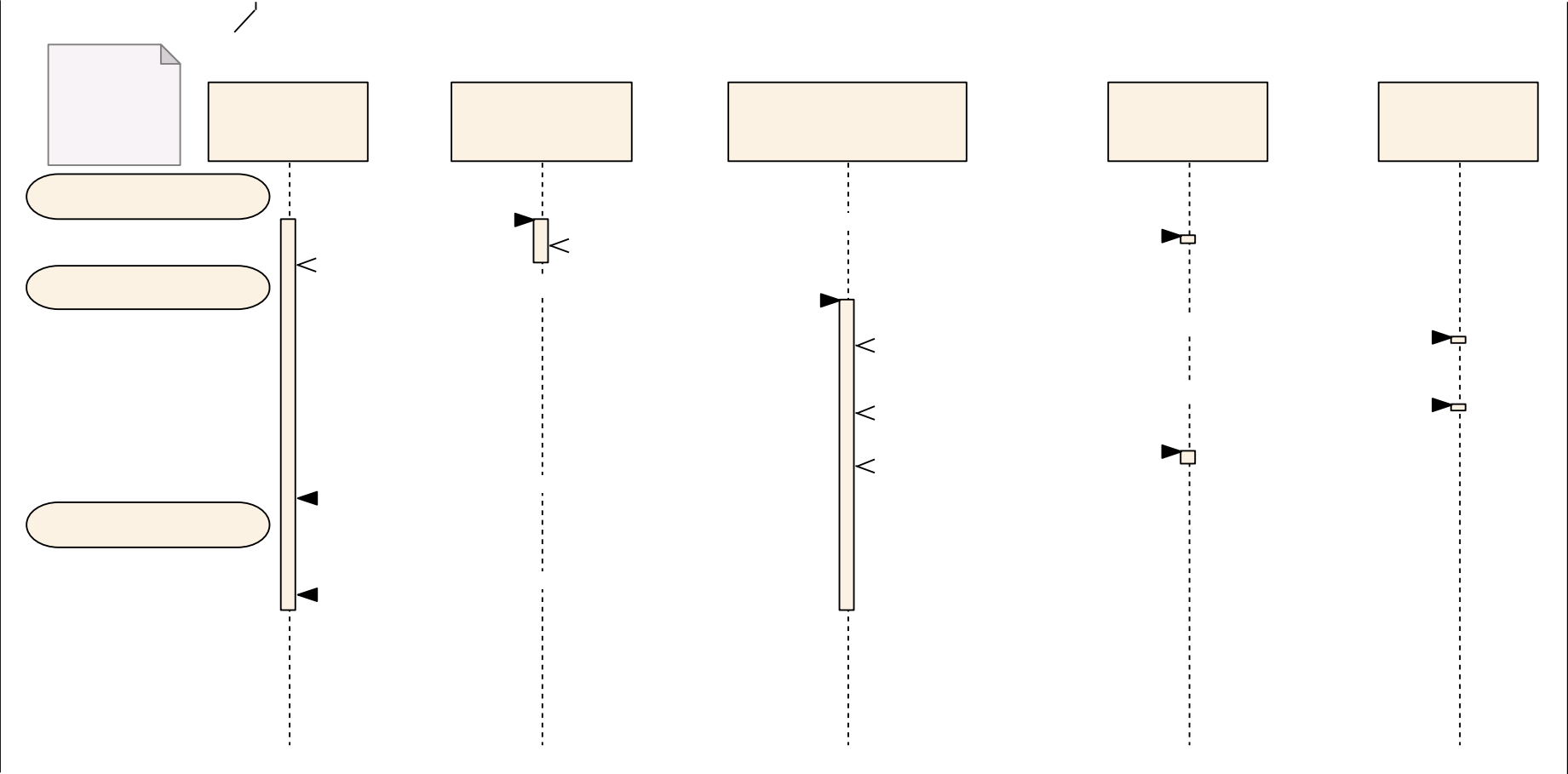
- 用于执行OS-Application中RTE清理和重启的函数
- 为重启的OS-Application触发初始化可运行实体
- 处理重启/终止的OS-Applications的通信一致性

操作系统

- OS-应用程序具有状态（可访问、重启中、已终止）
- OS提供API以终止其他OS-应用程序（适用于内存/时序之外的其他错误）

Integration and Runtime Aspects - Partitioning

Restart example



终止

- 操作系统应用程序可以直接终止 ■。同样，在终止时可能需要进行一些清理工作，且该过程应与重启操作系统应用程序时采用相同方式执行。

应用程序中的错误检测

- SW-C可能因内存或时序违规之外的其他原因需要重启 ■ 终止/重启可通过SW-C调用操作系统服务TerminateApplication()来触发 ■ 示例：分布式应用需要在多个ECU上重启

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射 2. 分区管理 3. 调度机制 4. 模式管理 5. 错误处理、报告与诊断 6. 测量与标定 7. 安全机制 8. 能源管理 9. 全局时间同步

基础软件调度器与RTE一同生成。这使得 ■ 同一操作系统任务能够调度基础软件主功能与软件组件的可运行实体 ■ 以优化资源消耗 ■ 配置可运行实体与基础软件主功能的交错执行序列。

- 一种影响BSW模块和应用软件组件的模式协调切换
- 由同一外部触发发生事件同步触发可运行实体和BSW主函数。

Integration and Runtime Aspects - Scheduling

Basic Scheduling Concepts of the BSW

BSW调度应

确保BSW的正确时序行为，即所有BSW模块在时间上的正确交互

数据一致性机制

应用的数据一致性机制应由ECU/BSW集成商根据配置的调度策略进行配置。

单个BSW模块不了解

ECU范围内的时序依赖关系 调度影响 实现数

据一致性的最高效方式

将BSW调度集中在由ECU/BSW集成商配置并由RTE生成器与RTE共同生成的BSW调度器中。

简化集成任务 支持对可调度对象 ■ 应用不同的调度策略（抢占式、非抢占式等）

支持应用不同的数据一致性机制 支持减少资源占用（例如最小化任务数量） 支持可
运行实体与基础软件主函数的交错执行序列

限制操作系统功能的使用

只有BSW调度器和RTE可以使用OS对象或OS服务（例外情况：EcuM、复杂驱动程序以及OS的GetCounterValue和GetElapsed
CounterValue服务；MCAL模块可以启用/禁用中断） 设计依据：

- BSW的调度应对系统（集成商）透明 ■ 能够减少操作系统资源（任务、资源等）的使用 ■ 支持在不同环境中复用模块

Integration and Runtime Aspects - Scheduling

调度对象、触发器与模式禁用依赖关系

BSW Scheduling objects

- Main functions
 - n per module
 - located in all layers

BSW Events

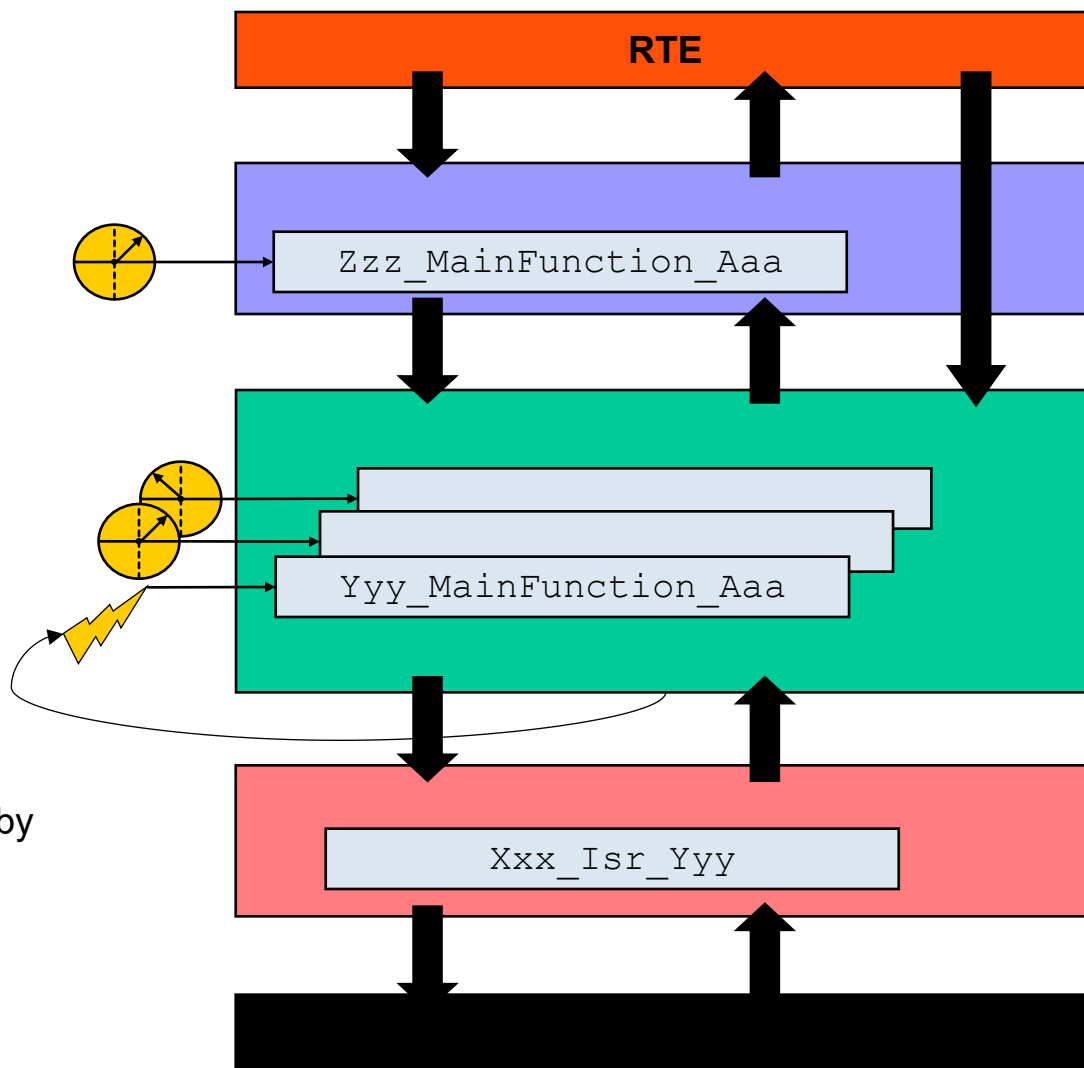
- BswTimingEvent
- BswBackgroundEvent
- BswModeSwitchEvent
- BswModeSwitchedAckEvent
- BswInternalTriggerOccuredEvent
- BswExternalTriggerOccuredEvent
- BswOperationInvokedEvent

Triggers

- Main functions can be triggered in all layers by the listed BSW Events

Mode Disabling Dependencies

- The scheduling of Main functions can be disabled in particular modes.



Integration and Runtime Aspects - Scheduling Transformation Process

Logical Architecture (Model)

- Ideal concurrency
- Unrestricted resources
- Only real data dependencies



- Scheduling objects
- Trigger
 - BSW Events
- Sequences of scheduling objects
- Scheduling Conditions
- ...

Technical Architecture (Implementation)

- Restricted concurrency
- Restricted resources
- Real data dependencies
- Dependencies given by restrictions



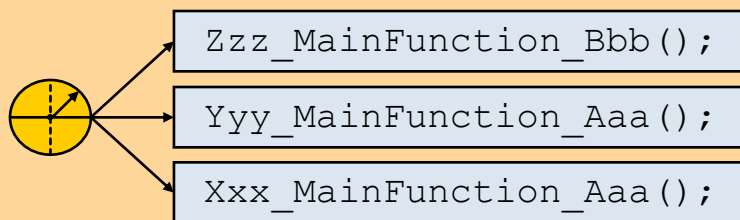
- OS objects
 - Tasks
 - ISRs
 - Alarms
 - Resources
 - OS services
- Sequences of scheduling objects within tasks
- Sequences of tasks
- ...

Transformation

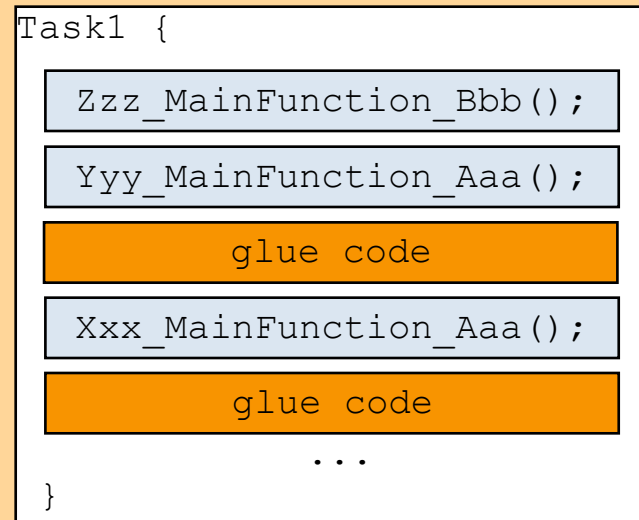
- Mapping of scheduling objects to OS Tasks
- Specification of sequences of scheduling objects within tasks
 - Specification of task sequences
 - Specification of a scheduling strategy
- ...

Integration and Runtime Aspects - Scheduling Transformation Process – Example 1

Logical Architecture (Model)



Technical Architecture (Schedule Module)

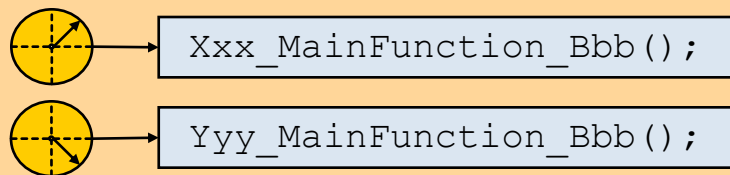


Transformation

- Mapping of scheduling objects to OS Tasks
- Specification of sequences of scheduling objects within tasks

Integration and Runtime Aspects - Scheduling Transformation Process – Example 2

Logical Architecture (Model)



Technical Architecture (Schedule Module)

```
Task2 {  
    ...  
    Xxx_MainFunction_Bbb();  
    ...  
}  
  
Task3 {  
    ...  
    Yyy_MainFunction_Bbb();  
    ...  
}
```

Transformation

- Mapping of scheduling objects to OS Tasks

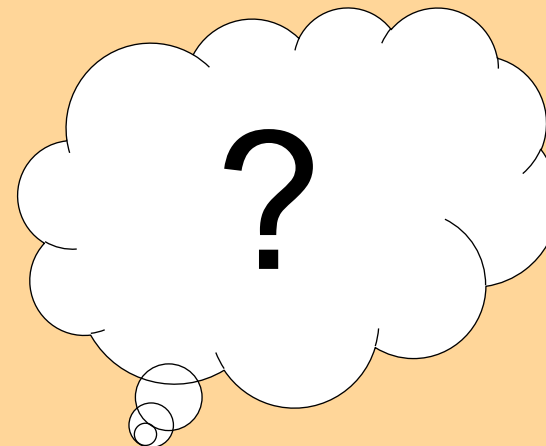
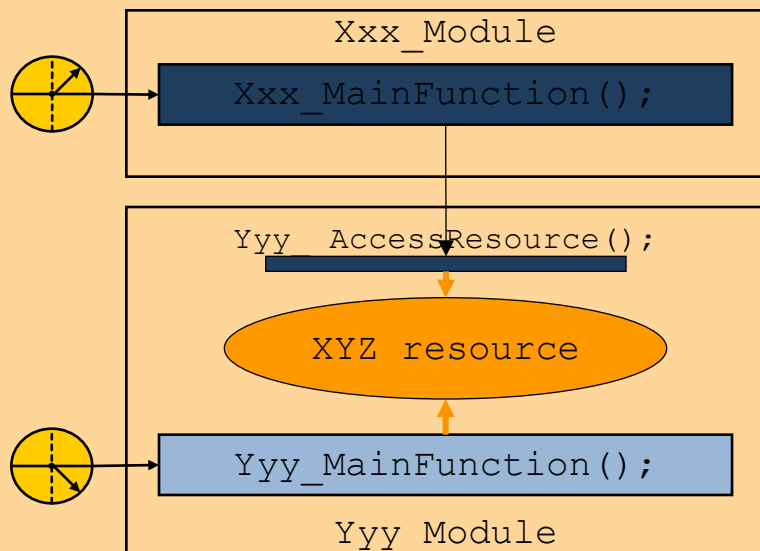
Integration and Runtime Aspects - Scheduling

Data Consistency – Motivation

Logical Architecture (Model)

Technical Architecture (Schedule Module)

- Access to resources by different and concurrent entities of the implemented technical architecture (e.g., main functions and/or other functions of the same module out of different task contexts)



Transform

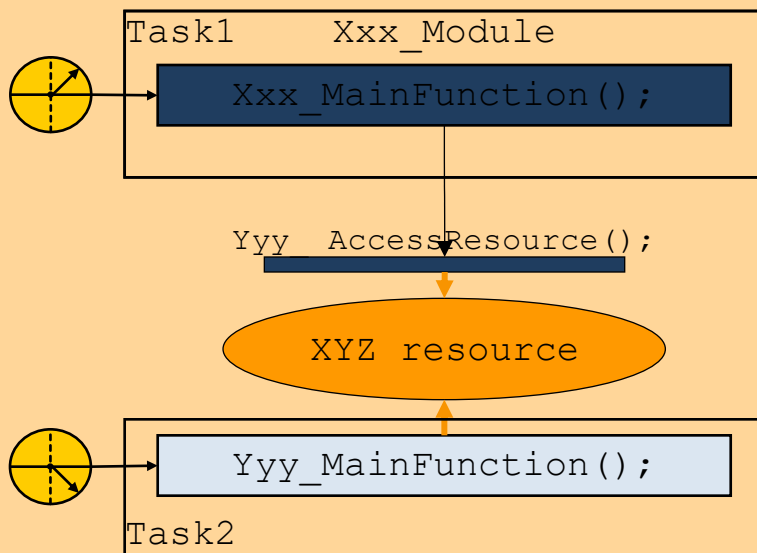
Data consistency strategy to be used:

- Sequence, Interrupt blocking, Cooperative Behavior, Semaphores (OSEK Resources), Copies of ...

Integration and Runtime Aspects - Scheduling

Data Consistency – Example 1 – “Critical Sections” Approach

Logical Architecture (Model)/ Technical Architecture (Schedule Module)



Implementation of Schedule Module

```

#define SchM_Enter_<mod>_<name> \
    DisableAllInterrupts
#define SchM_Exit_<mod>_<name> \
    EnableAllInterrupts

Yyy_AccessResource() {
    ...
    SchM_Enter_Xxx_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Xxx_XYZ();
    ...
}

Yyy_MainFunction() {
    ...
    SchM_Enter_Yyy_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Yyy_XYZ();
    ...
}
  
```

Transform

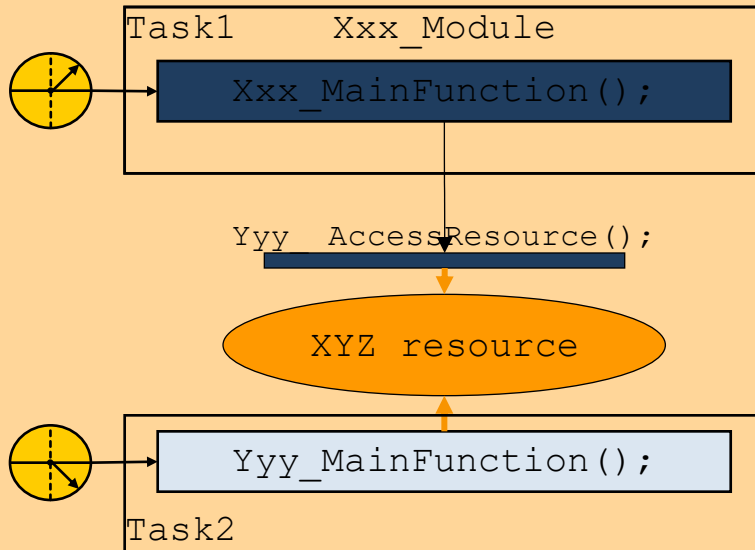
Data consistency is ensured by:

- Interrupt blocking

Integration and Runtime Aspects - Scheduling

Data Consistency – Example 1 – “Critical Sections” Approach

Logical Architecture (Model)/ Technical Architecture (Schedule Module)



Implementation of Schedule Module

```
#define SchM_Enter_<mod>_<name> \
/* nothing required */
#define SchM_Exit_<mod>_<name> \
/* nothing required */

Yyy_AccessResource() {
    ...
    SchM_Enter_Xxx_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Xxx_XYZ();
    ...
}

Yyy_MainFunction() {
    ...
    SchM_Enter_Yyy_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Yyy_XYZ();
    ...
}
```

Transform

Data consistency is ensured by:

- Sequence

Integration and Runtime Aspects

模式通信 / 模式相关调度

BSW模块的模式依赖调度与软件组件可运行实体的模式依赖调度相同。模式管理器在其基础软件模块描述中定义提供模式声明组原型，BSW调度器提供API以向BSW调度器传递模式切换请求。模式用户在其基础软件模块描述中定义所需模式声明组原型。BSW调度器按需提供API以读取当前活动模式。若基础软件模块描述定义了模式禁用依赖关系，BSW调度器将在特定模式下抑制BSW主函数的调度。

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

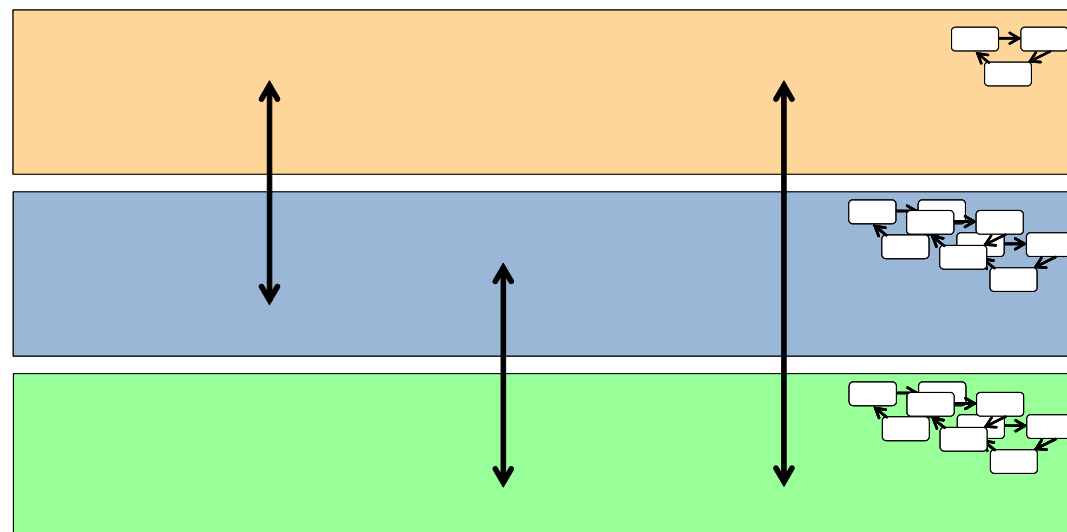
Integration and Runtime Aspects

车辆与应用模式管理 (1)

模式关系：

每个系统都包含不同粒度级别的模式。如图所示，存在车辆模式、若干带模式的应用程序以及具有本地BSW模式的ECU。

所有这些层面的模式都相互影响。



因此：

根据车辆模式的不同，应用程序可能处于激活或非激活状态，从而处于不同的应用模式。反之，特定应用程序的运行状态也可能引发车辆模式的变化。依据车辆及应用模式，基础软件（BSW）模式可能发生变化，例如：应用程序的通信需求可能导致通信网络的BSW模式改变。反之，BSW模式也可能影响应用程序乃至整车的模式，例如：当通信网络不可用时，依赖该网络的应用程序可能切换至跛行回家模式。

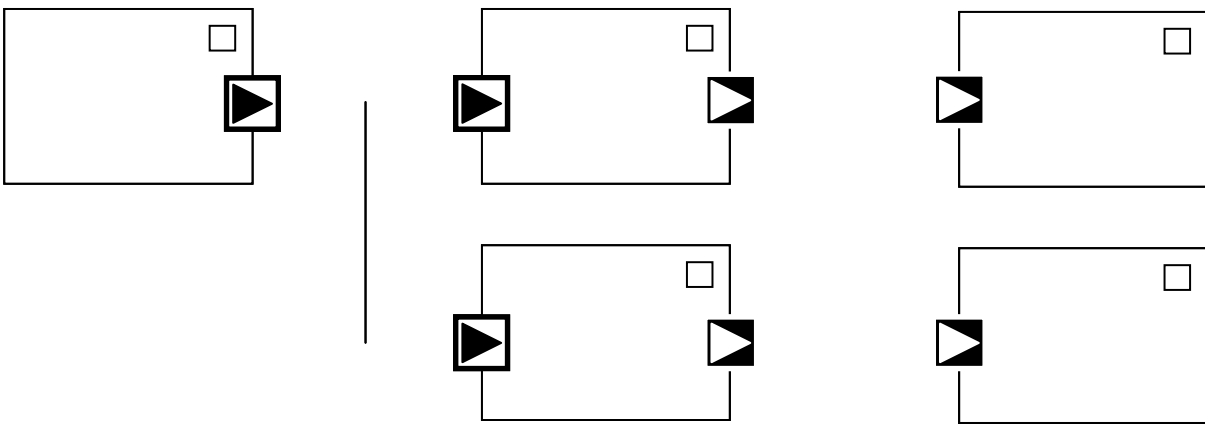
Integration and Runtime Aspects 车辆与应用模式管理 (2)

模式请求的处理

车辆模式管理的基本思想是分发和仲裁模式请求，并根据结果在本地控制BSW。

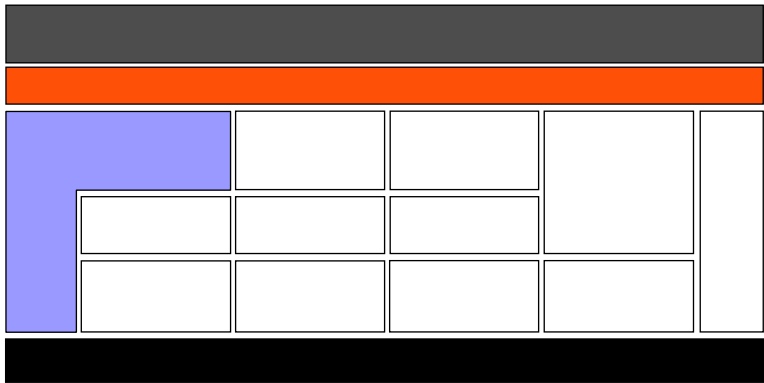
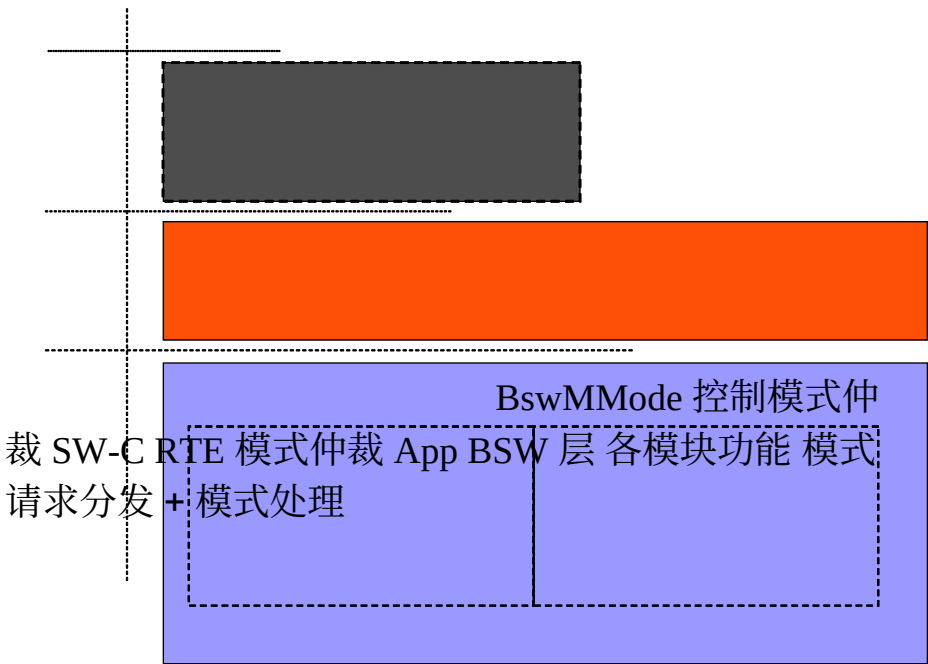
这意味着在每个OS-Application中，必须有一个模式管理器，用于为其本地模式用户切换模式并控制BSW。当然，也可以有多个模式管理器来切换不同的模式。

模式请求是一种“正常”的发送者/接收者通信（系统范围内），而模式切换始终是本地服务。



Integration and Runtime Aspects

车辆与应用模式管理 (3)



系统服务应用层

所需功能的主要部分被置于基础软件模式管理器（简称BswM）中。由于BswM位于基础软件层，它存在于每一个操作系统应用中，并且对模式用户以及受控的基础软件模块而言都是本地的。

模式请求的分布由RTE执行，且RTE也负责处理模式切换。例如，对于车辆模式，模式请求源自一个中央模式请求者SW-C，并需由多个ECU中的BswMs接收。这是SW-C仅能与本地BSW通信规则的一个例外。运行在不同OS-Applications中的BswMs可通过发送-接收通信（SchM_Send、SchM_Receive）传播模式请求。

Integration and Runtime Aspects

车辆与应用模式管理 (4)

Mode Processing Cycle

- The mode requester SW-C requests mode A through its sender port. The RTE distributes the request and the BswM receives it through its receiver port.
- The BswM evaluates its rules and if a rule triggers, it executes the corresponding action list.
- When executing the action list, the BswM may issue a (configurable optional) RTE call to the mode switch API as a last action to inform the mode users about the arbitration result, e.g. the resulting mode A'.
- Any SW-C, especially the mode requester can register to receive the mode switch indication.
- The mode requests can originate from local and remote ECUs or OS-Applications.
- Note that the mode requestor can only receive the mode switch indications from the local BswM, even if the requests are sent out to multiple OS-Applications.

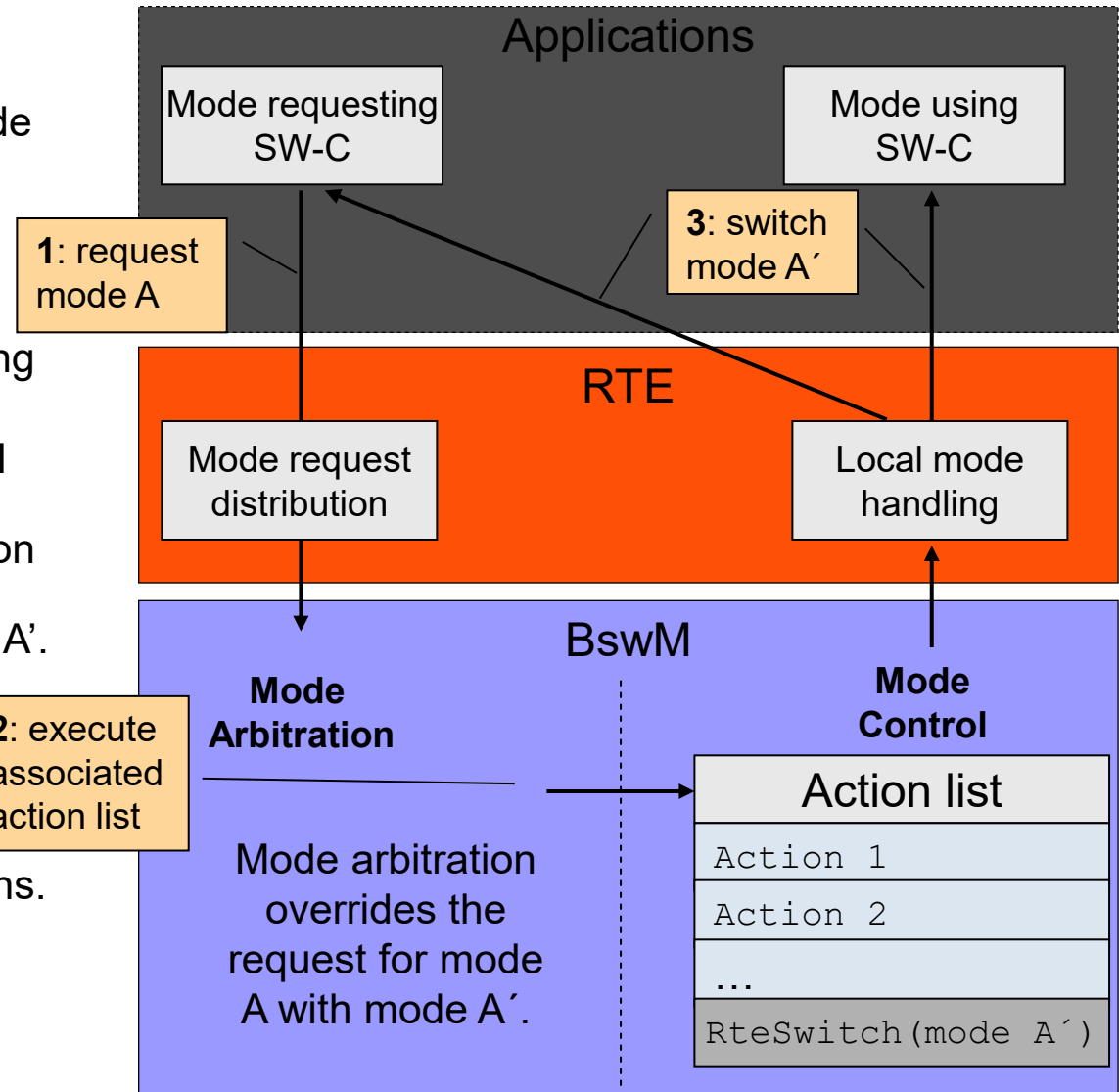


Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4. 模式管理5. 错误处理、报告与诊断6. 测量与标定7. 安全机制8. 能源管理9. 全局时间同步

错误分类 (1)**错误类型****硬件错误/故障**

- 根本原因：软件检测到损坏、故障或“数值超出范围” 示例 EEPROM 单元无法再写入
- 示例 传感器输出电压超出规定范围

软件错误

- 根本原因：错误的软件或系统设计，因为软件本身永远不会失效。 示例 错误的API参数（EEPROM目标地址超出范围）
- 示例 使用未初始化的数据

系统错误

- 示例1：CAN接收缓冲区溢出
- 示例 接收消息超时

报告错误有几种替代方案（详见以下幻灯片）：

通过API

通知调用者操作的成功/失败情况。

通过静态可定义的回调函数（通知）向调用者报告操作失败

通过中央错误钩子（默认错误追踪器，Det）用于在产品开发期间记录和追踪错误。可在生产代码中关闭此功能。

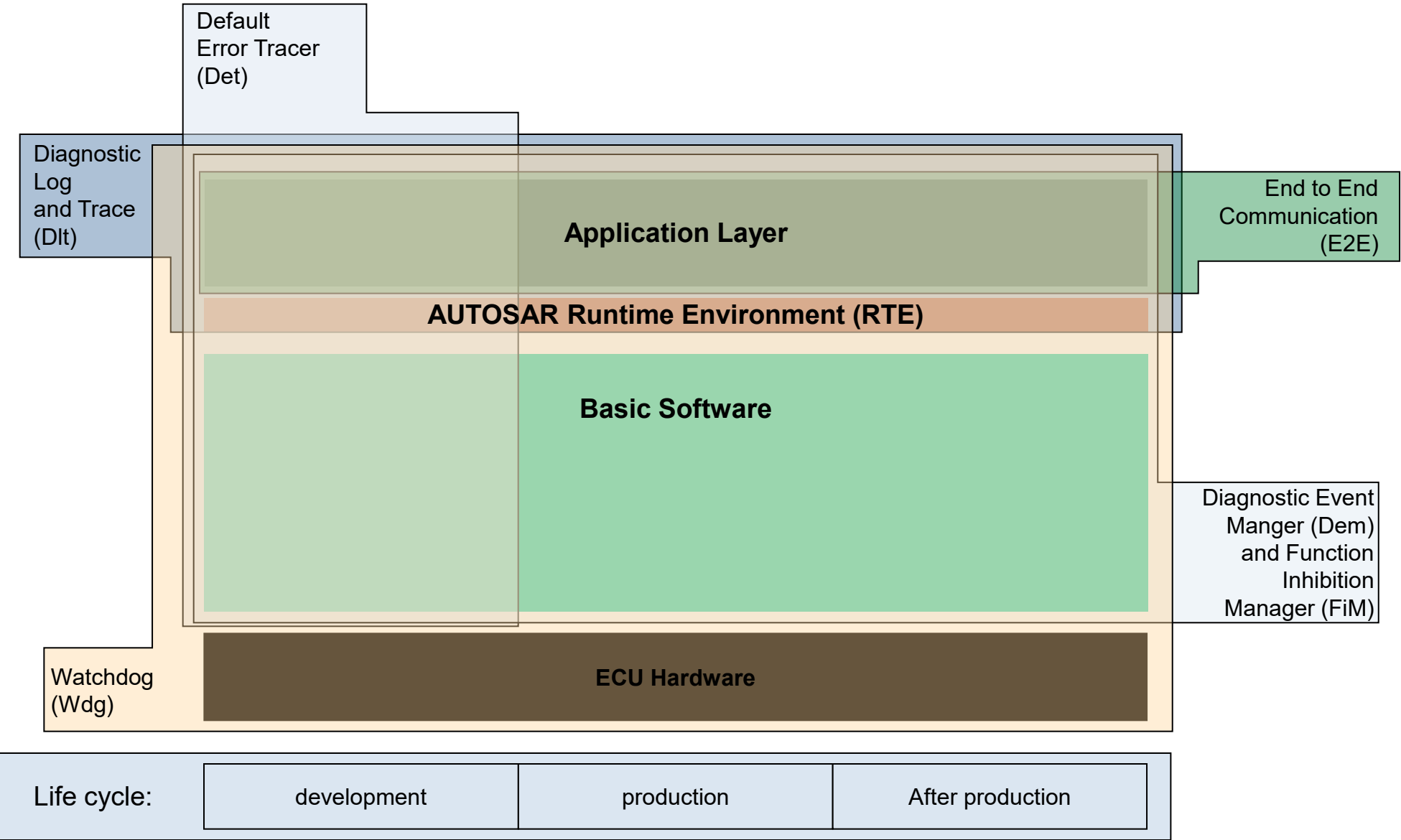
通过中央标注（默认错误追踪器，Det）用于处理产品生命周期中的错误。

通过中央错误函数（AUTOSAR诊断事件管理器）用于系列中的错误反应和记录（生产代码）

每个应用软件组件（SW-C）都可以向诊断事件管理器（Dem）报告错误。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

与AUTOSAR层级及系统生命周期相关的机制



错误分类 (2)**错误类别****开发错误**

开发错误属于软件错误。它们应像断言一样在开发阶段被检测并修复。对于生产代码，可按模块关闭仅应在开发期间出现的错误检测（通过静态配置，即预处理器开关）。相关API在AUTOSAR中已有规范，但具体功能可由开发者根据实际需求选择或实现。

运行时错误

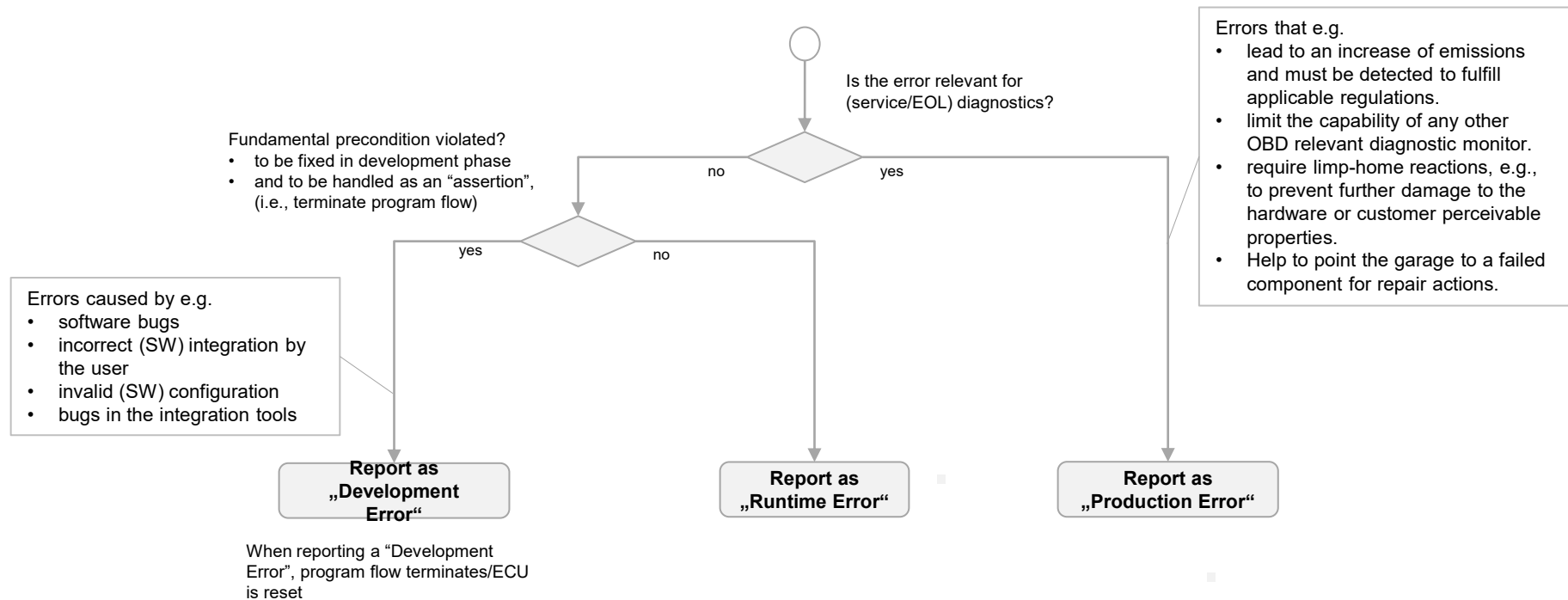
运行时错误是系统性的软件错误。它们指示了阻碍代码正确执行的严重异常。即使在已部署的系统中，监视器也可能停留在代码中。这些错误的同步处理可选择在集成器代码中完成。

生产错误

这些错误被存储在故障存储器中，以便在维修车间进行修复操作。它们的发生是可以预见的，并且在生产代码中无法避免。生产错误具有检测条件和恢复条件。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

错误分类 (3)



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

错误分类总结

Error Return Code	Inform API caller about current state of an error Leave error reaction to calling module, which has more context to decide on proper error reaction	During all life cycles	none	Event based (set condition only)	continues	To calling BSW module (synchronously via API return code or asynchronously via notification callback)
Development Error	Detect SW/integration issues early during development.	During „Development“ only	Module specific overall enabling/disabling of Detection + Reporting of development errors via pre-compile switch DevErrorDetect provides Callout for integration code in Det_ReportError	Event based (set condition only)	terminates	DET (via Det_ReportError; integration code in Callout allows some preparation before ECU is reset)
Runtime Error	Detect errors during normal operation, e.g. systematic software faults, timing faults or hardware issues	During all life cycles	Always active provides Callout for integration code in Det_ReportRuntimeError	Event based (set condition only)	continues	DET (via Det_ReportRuntimeError; integration code in Callout allows further reporting)
Production Error	Detect issues (and absense of issues) relevant e.g. for service/repair actions or system degradation	During all life cycles	<Ma>DemEventParameterRefs parameter allows to enable/disable the error	Maturing/healing (set/reset conditions)	continues	DEM (via Dem_SetEventStatus to the event memory)

通过API报告错误

通过API报告错误

通过返回错误状态来通知调用者操作失败。

基本返回类型

成功：E_OK（值：0）失败：

E_NOT_OK（值：1）

特定返回类型

如果生产代码需要区分不同的错误，必须定义自己的返回类型。只有当调用者确实能够处理这些错误时，才应使用不同的错误类型。具体的开发错误不应通过API返回。它们可以报告给默认错误跟踪器（Det）。

示例：ADC驱动服务 成功ADC_E_OK 一般错误（
服务未被接受）：ADC_E_NOT_OK ADC正忙于当前
转换：ADC_E_BUSY

错误报告 – 简介

通过诊断事件管理器 (Dem) 进行错误报告

用于报告生产/系列错误。

这些错误根据该ECU的配置有明确的反应，例如：

- 写入错误内存

- 禁用ECU功能（例如通过功能抑制管理器）

- SW-Cs通知

诊断事件管理器是一个标准的AUTOSAR模块，在生产代码中始终可用，其功能在AUTOSAR规范内定义。

通过默认错误追踪器 (Det) 进行错误报告

用于报告开发/运行时错误。

默认错误追踪器主要用于处理开发期间的错误，同时也用于处理生产代码中的系统性错误。在默认错误追踪器中可实现多种机制，例如：

- 计数错误

- 将错误信息写入RAM中的环形缓冲区

- 通过串行接口将错误信息发送至外部记录器

- 无限循环，断点

每个模块（预处理器开关或模块的不同对象代码构建）可以静态地开启/关闭向默认错误追踪器报告开发错误的功能，但运行时错误不在此列。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

诊断事件管理器 – 诊断错误报告

应用程序编程接口

诊断事件管理器具有以下API:

Dem_SetEventStatus(事件ID, 事件状态)

问题: 出于效率考虑, 通过此API传递的错误ID必须为ECU全局定义、静态定义, 且需占用紧凑的数值范围。原因: 诊断事件管理器将此ID作为索引来访问ROM数组。

错误编号概念: 基于XML的错误编号生成

Properties:

- 源与目标代码兼容
- 所有生产相关错误采用单一命名空间
- 需要工具支持
- 连续错误编号 → 错误管理器可轻松访问定义错误处理与应对措施的ROM数组

Process:

- 每个BS模块将其所需的所有生产代码相关错误变量声明为“extern”
- 每个BSW模块将其所需的所有错误变量存储在ECU配置描述中(例如CANSM_E_BUS_OFF)
- 诊断事件管理器的配置工具解析ECU配置描述, 并生成一个包含SW模块所需的全局常量变量的单一文件(例如const Dem_EventIdType DemConf_DemEventParameter_CANSM_E_BUS_OFF=7U; 或#define DemConf_DemEventParameter_CANSM_E_BUS_OFF ((Dem_EventIdType)7))
- 对错误的响应也在错误管理器配置工具中定义, 此配置是项目特定的。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

默认错误追踪器 – 示例：开发错误报告

应用程序编程接口

默认错误追踪器提供以下API用于报告开发错误（运行时错误使用相同但名称不同的API）：

Det_报告错误(uint16 模块标识, uint8 实例标识, uint8 接口标识, uint8 错误标识)

错误编号概念

模块ID (uint16)

模块ID包含来自CP SWS BSWGeneral的AUTOSAR模块ID。由于范围为16位，未来可扩展用于应用软件组件（SW-C）的开发错误报告。基础软件仅使用0..255的范围。

实例ID (uint8)

实例ID表示从0开始的基于索引的模块标识符。如果模块是单实例模块，则应传递0作为实例ID。

ApiId (uint8)

API-ID在BSW模块的软件规范中指定。它们可以是模块内定义的以0开头的#define或常量。

错误ID (uint8)

错误ID在BSW模块的软件规范中指定。它们可以是模块头文件中定义的#define指令。

如果某个特定软件模块检测到的错误未在AUTOSAR模块软件规范中明确规定，则必须在模块文档中予以记录。

所有错误ID必须在BSW描述中指定。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic Diagnostic Log and Trace (1)

诊断日志与追踪（Dlt）模块负责收集日志信息并将其转换为标准化格式。Dlt模块将数据转发至PduR，再由PduR将其发送至已配置的通信总线。

因此，Dlt 提供以下功能

： 日志记录

- 记录来自AUTOSAR SW-C的错误、警告和信息消息，提供标准化的AUTOSAR接口，
- 将所有AUTOSAR软件组件（SW-C）的日志和跟踪消息集中收集到基础软件（BSW）中的统一AUTOSAR服务组件（Dlt）内，
- 记录来自Dem的消息。

追踪

- RTE活动的

控制

- 单个日志和跟踪消息可以启用/禁用，且通过反向通道可独立控制
- 日志级别。

通用

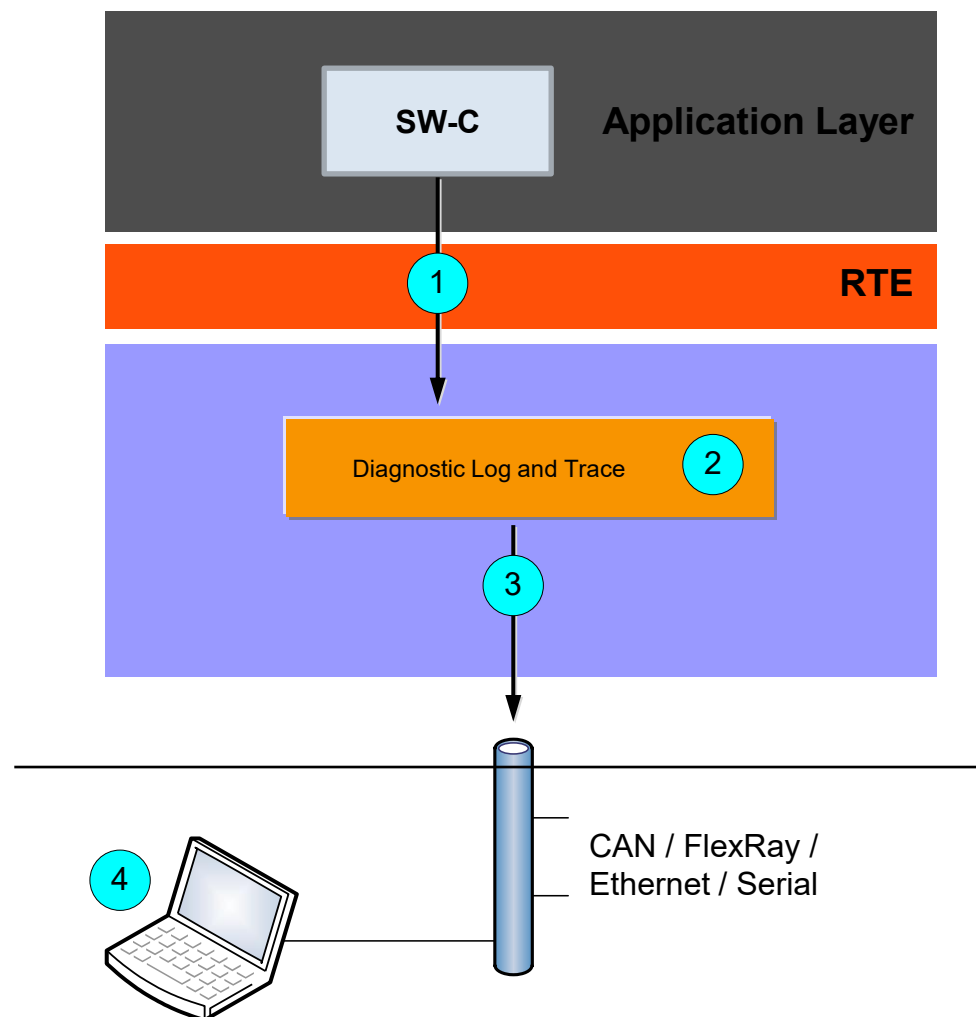
- Dlt在开发和生产阶段均可使用，
- 可通过标准诊断或平台特定测试接口进行访问，并且
- 提供了防止在生产阶段滥用的安全机制。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

诊断日志与追踪 (2)

Dlt通信模块由外部客户端启用。

(1) 一个SW-C正在生成日志信息。该日志信息通过调用Dlt提供的接口发送至Dlt (2) Dlt实现Dlt协议 (3) Dlt将编码后的日志信息发送至通信总线 (4) 外部Dlt客户端收集日志信息，并为后续分析提供支持



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

诊断日志与追踪（3）

应用程序编程接口

诊断日志与跟踪在语法上具有以下API:

Dlt_SendLogMessage(Dlt_SessionIDType 会话ID, Dlt_MessageLogInfoType 日志信息, uint8 *日志数据, uint16 日志数据长度)

日志消息识别:

会话ID

会话ID是日志或跟踪会话的识别号码。会话是日志或跟踪消息来源的逻辑实体。如果一个SW-C被多次实例化或向Dlt开放多个端口, 则每个实例都会使用一个具有新会话ID的新会话。此外, 如果SW-C向Dlt开放了多个端口, 它还可以拥有多个日志或跟踪会话。

log_info 包含:

应用ID / 上下文ID

应用ID是SW-C的简称, 用于在日志和追踪消息中标识SW-C。上下文ID是用户定义的ID, 用于对SW-C产生的日志和追踪消息进行分组以区分功能。每个应用ID可拥有多个上下文ID。上下文ID按应用ID分组。两者均由四个8位ASCII字符组成。

消息ID

消息ID是用于表征信息本身的标识符, 它随消息一同传输。该ID可用于识别消息的源代码来源, 并应用于描述消息的有效载荷。消息ID在开发或配置阶段静态固定。

日志数据

包含log或跟踪数据本身。所提供缓冲区的内容和结构协议。

由 Dlt 传输指定

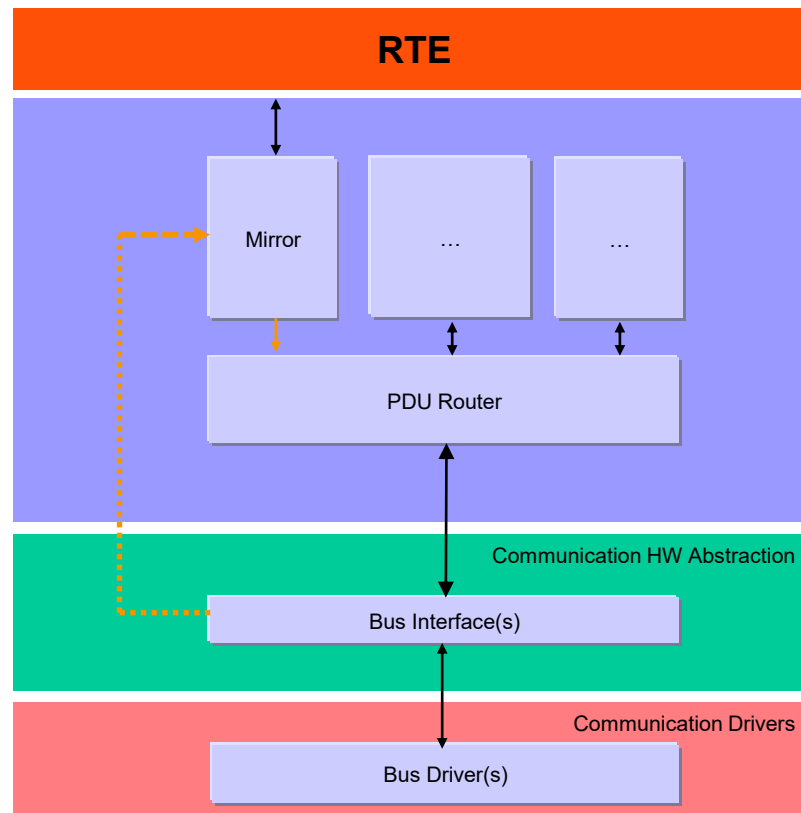
描述文件

通常, 日志数据仅包含非固定变量或信息的内容(例如, 不传输静态字符串)。此外, 应提供一个描述文件。在该文件中, 会发布与消息ID关联的日志消息的相同信息。这些信息说明了如何解释日志数据缓冲区以及哪些固定条目属于日志消息。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

镜像总线通信

- 总线镜像模块（Mirror）从多条总线收集帧，并将这些帧逐一或汇总成流后转发至一个共同的目标总线。
- 镜像模块定义了镜像帧流内部使用的协议。
- 镜像模块应仅允许单向转发，以避免内部网络因外部链接（需配置为镜像目标）而暴露于安全漏洞。



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

运行期间的数据收集：VDP

VDP是AUTOSAR的车辆数据协议，由VdpCmR模块实现。

VDP是一种AUTOSAR标准，旨在实现量产车辆中ECU数据的采集。

AUTOSAR中的VDP提供以下功能

- 远程ECU中数据采集（采样）与数据传输的系统性解耦。
- 动态配置应收集哪些数据。
- 数据点的激活与停用以稳定数据收集的临时暂停。

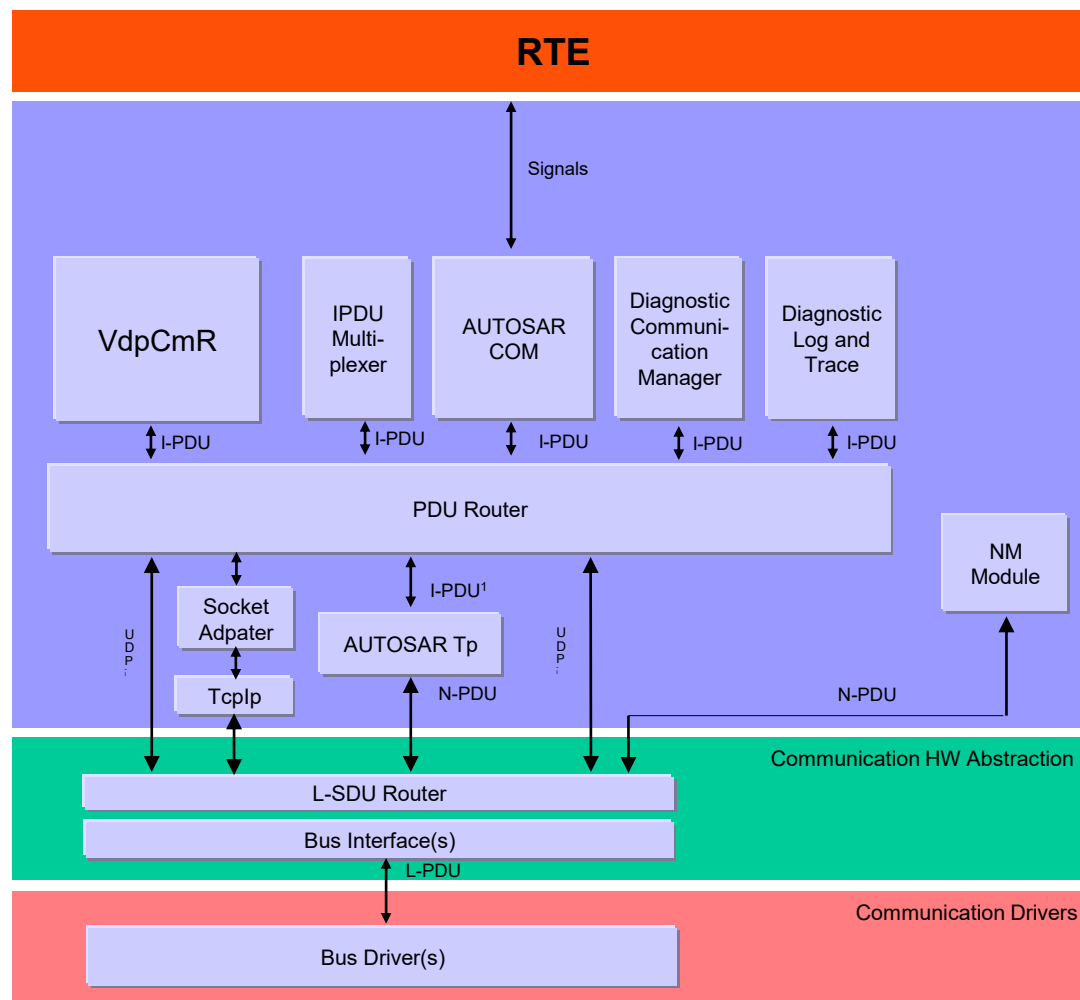


Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

Integration and Runtime Aspects - Measurement and Calibration

XCP

XCP是用于ECU校准的ASAM标准。

AUTOSAR中的XCP提供以下基本功能：

同步数据采集

同步数据刺激

在线内存校准（读/写访问）

校准数据页初始化和切换

用于ECU开发目的的Flash编程

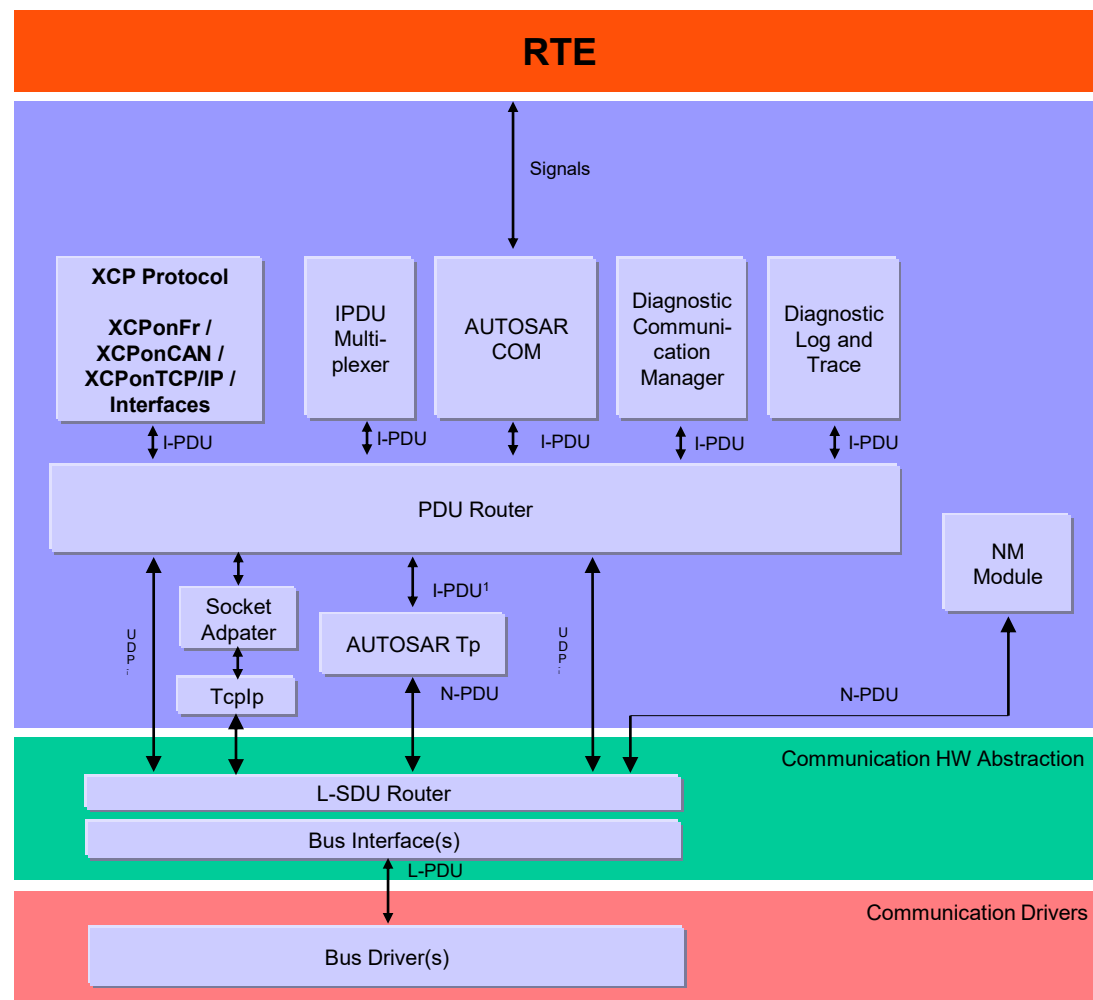


Table of contents

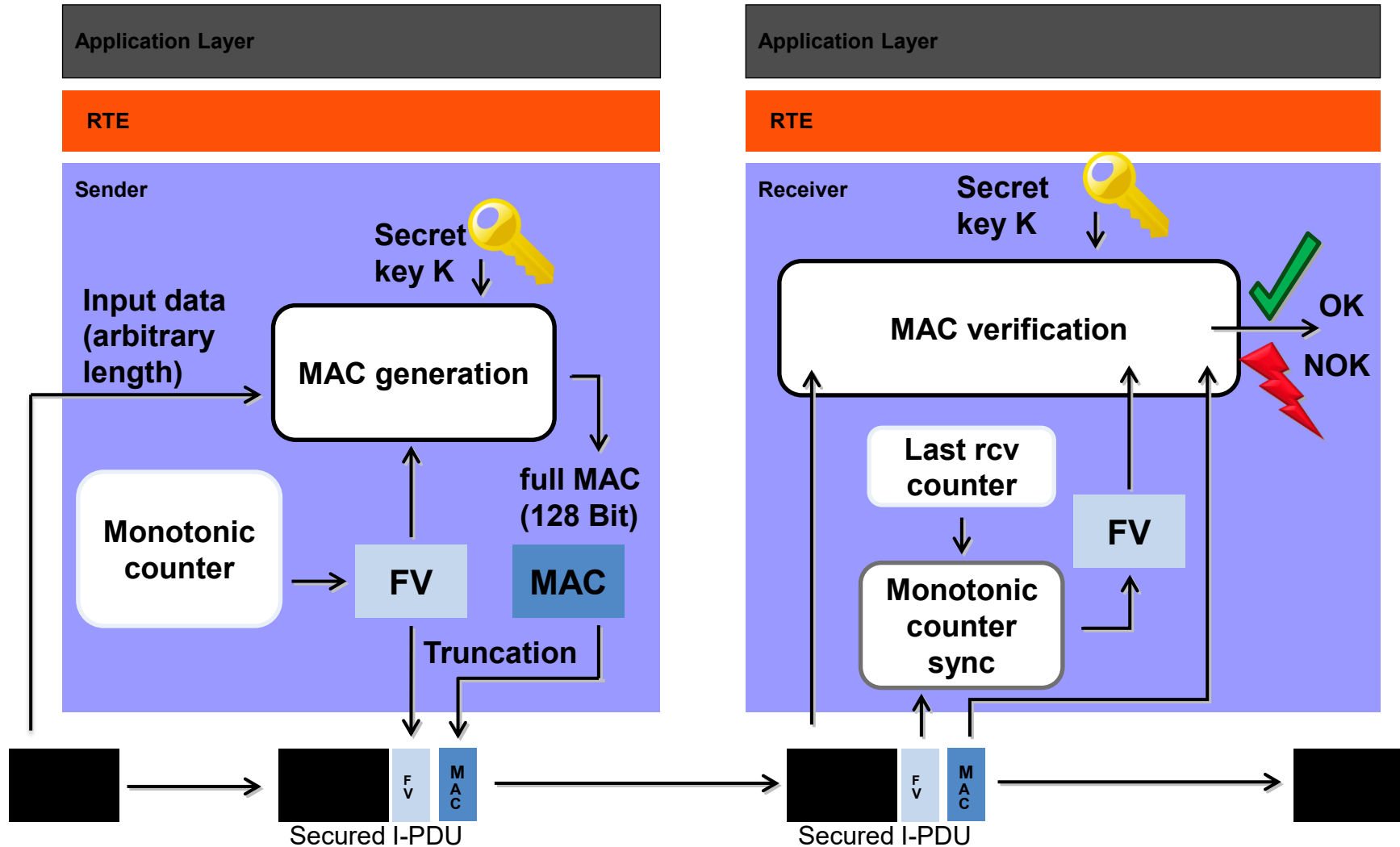
1. 架构 2. 配置

3. 集成与运行时方面

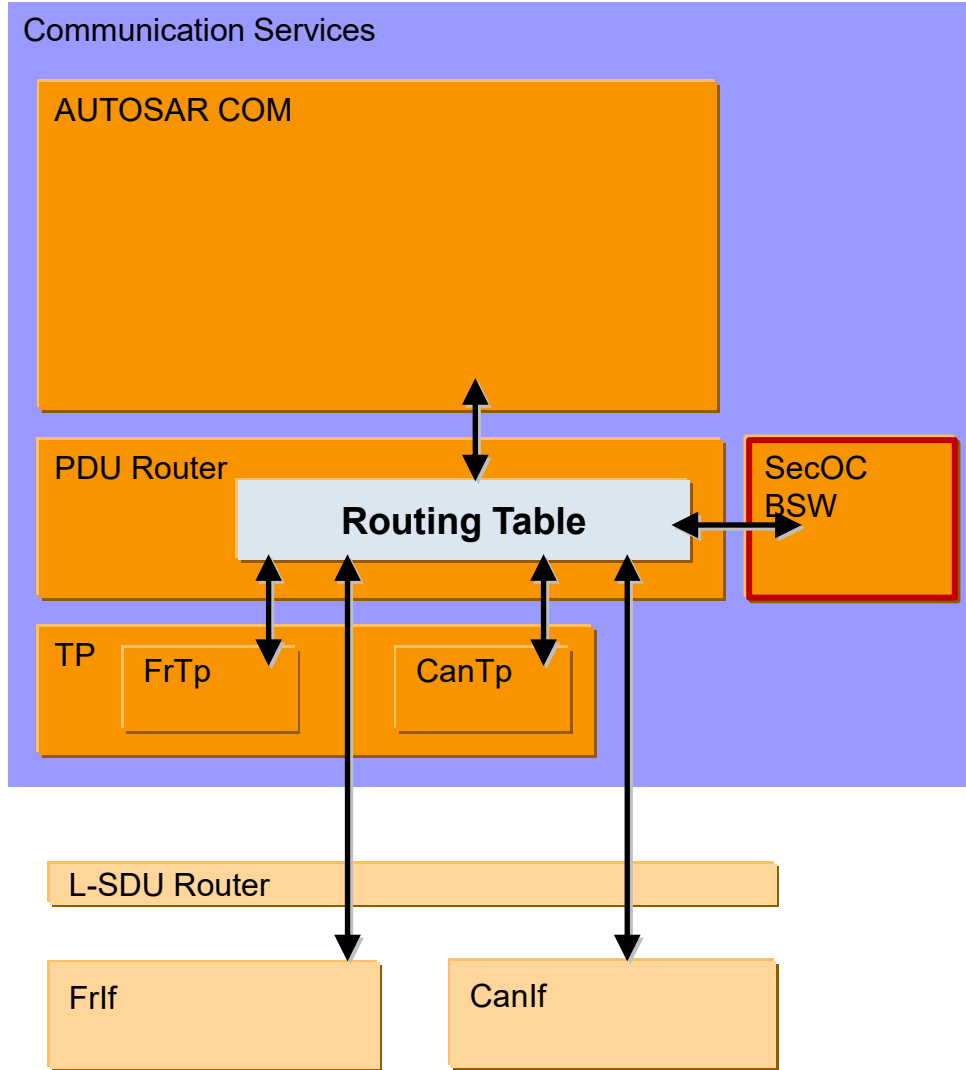
1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

Integration and Runtime Aspects – Secure Onboard Communication

概述 - 消息认证与新鲜性验证

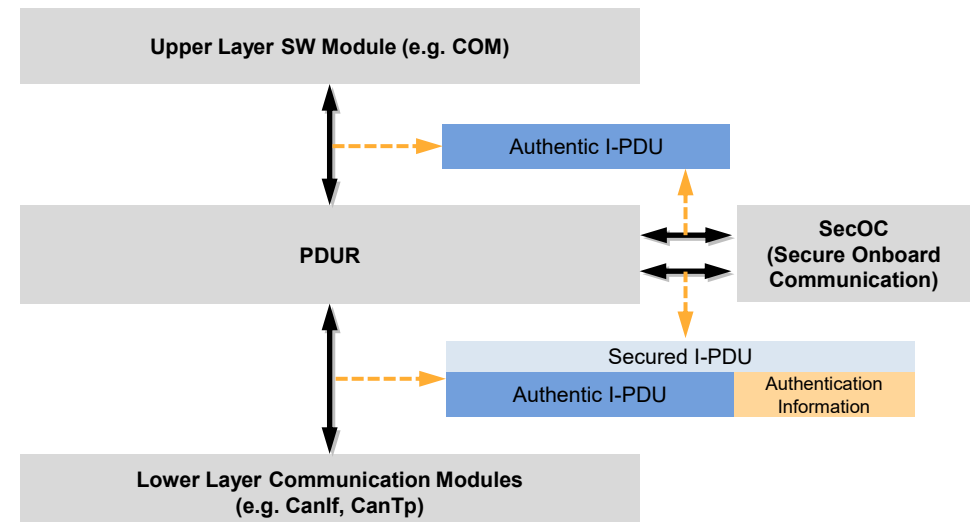


MAC: 消息认证码
FV: Freshness Counter Value



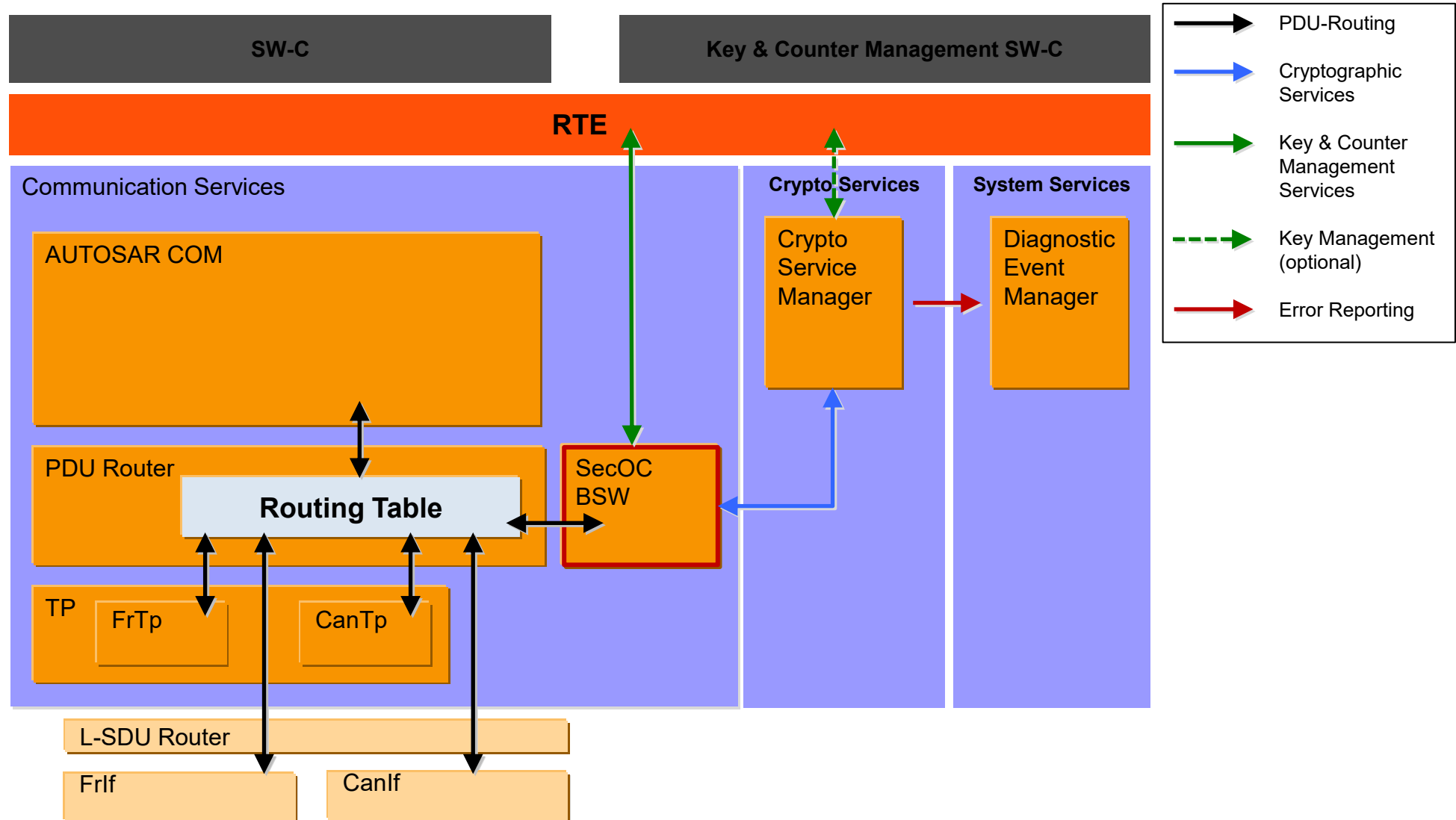
SecOC BSW:

- adds/verifies authentication information (for/from lower layer)
- realizes interface of upper and lower layer modules
- is addressed by PduR routing configuration
- maintains buffers to store and modify secured I-PDUs



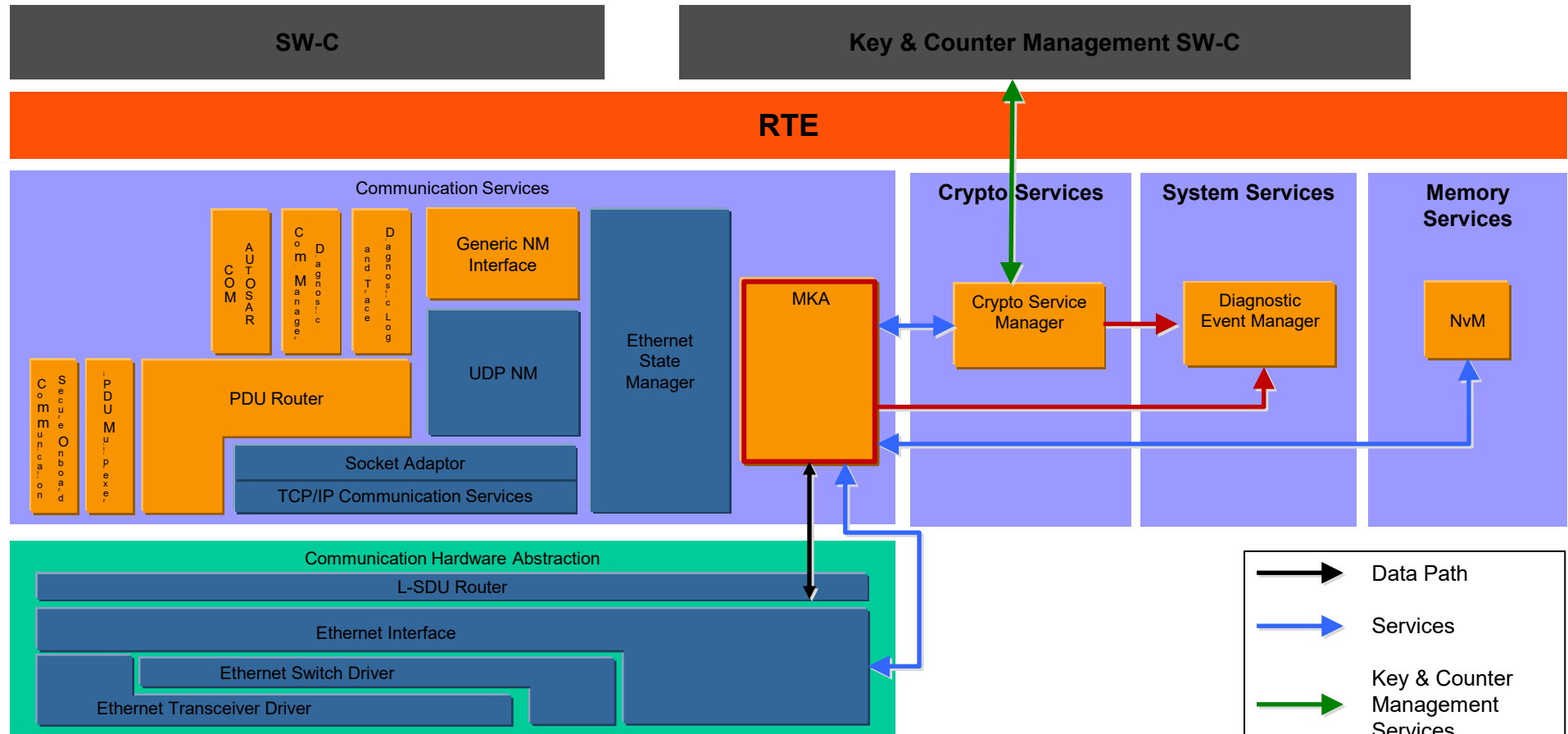
Integration and Runtime Aspects – Secure Onboard Communication

与其他服务的集成



Integration and Runtime Aspects – MACsec Key Agreement

与其他服务的集成



MACsec Key Agreement:

- Configures the MACsec entities to enable MACsec protected traffic
- Generates and processes MKPDUs

使用加密服务生成和验证MKPDU的完整性校验值 (ICV)

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

AUTOSAR中高效能源管理的目标是提供节能机制，尤其是在总线通信处于活动状态时（例如充电或15号端子激活）。AUTOSAR R3.2和R4.0.3仅支持部分网络功能。

部分网络

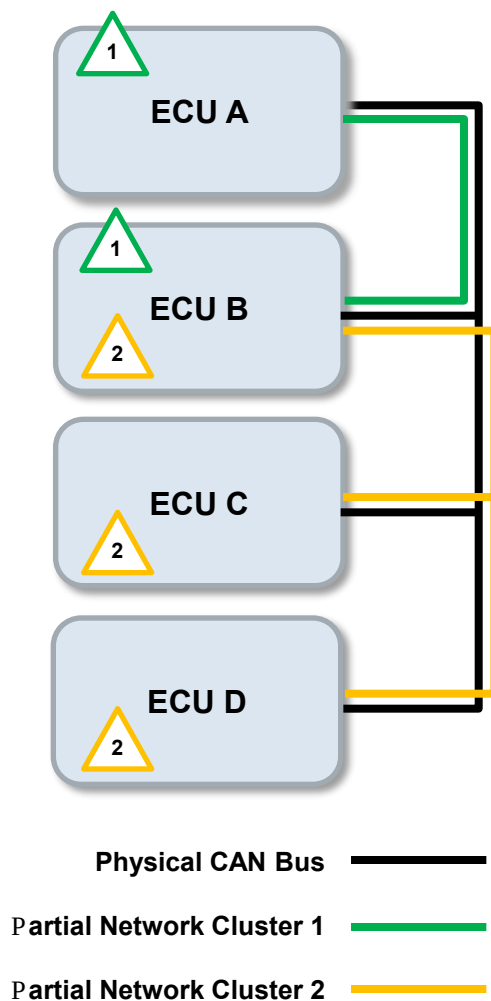
允许在特定条件下关闭多个ECU之间的网络通信，当它们提供的功能不需要时。其他ECU可以继续在同一总线通道上进行通信。使用NM消息在参与ECU之间传递部分网络集群的请求/释放信息。

ECU退化

允许切换外围设备。

Energy Management – Partial Networking

部分网络进入休眠的示例场景



初始情况:

ECU “A” 和 “B” 是部分网络集群（PNC）1的成员。

ECU “B”、“C” 和 “D” 是PNC 2的成员。

ECU的所有功能都组织在PNC 1或PNC 2中。

两个PNC均处于活动状态。

PNC 2 仅由 ECU “C” 请求。

需要ECU “C” 上PNC 2的功能已终止，因此ECU “C” 可以释放PNC 2。

情况是这样的:

ECU “C” 停止请求PNC 2激活。

ECU “C” 和 “D” 不再参与任何PNC，可以关闭。

ECU “B” 停止发送和接收与PNC 2相关的所有信号。

ECU “B” 仍参与PNC 1。这意味着它保持唤醒状态，并继续发送和接收与PNC 1相关的所有信号。

ECU “A” 完全不受影响。

能源管理的一个重要部分是关于模式处理。对于术语

- 车辆模式、■应用模式和
- 基础软件模式

请参阅本文档的第3.4章。

虚拟功能集群（VFC）：将实现一个或多个车辆功能所需的软件组件之间在端口层面的通信进行分组。这是逻辑视图，支持可复用且独立于总线/ECU的设计。

VFC控制器：一种特殊的软件组件，用于判断在特定时刻是否需要VFC的功能，并据此请求或释放通信。

部分网络集群（PNC）：是一组系统信号，这些信号对于支持一个或多个车辆功能是必需的，并且分布在车辆网络的多个ECU中。这代表了将一组总线映射到一个或多个VFC的系统视图。

部分网络（PN）目前支持CAN和FlexRay总线。LIN和CAN从属总线（即无网络管理的CAN总线）可通过PN激活*，但这些总线不支持唤醒或NM消息（包括PNC位向量）的通信。要唤醒PN ECU，需要符合ISO 11898-5规定的特殊收发器硬件。■ 先前AUTOSAR版本中已知的无特殊收发器硬件的标准唤醒方式仍受支持。一个VFC可映射到任意数量（包括零个）的PNC ■ PN概念将通过VFC映射至ComM中的内部通道类型来考虑仅包含ECU内部通信的VFC，因为其不涉及总线通信且无物理PNC。

CAN 的限制条件

- J1939与PN相互排斥，这是由于地址声明和J1939的启动行为所致。■ J1939在唤醒后需先在网络中注册其地址，才能开始通信。■ 然而，如上所述，不使用地址声明的J1939总线可通过PN作为CAN从属总线来激活。

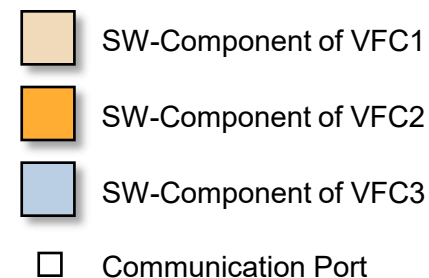
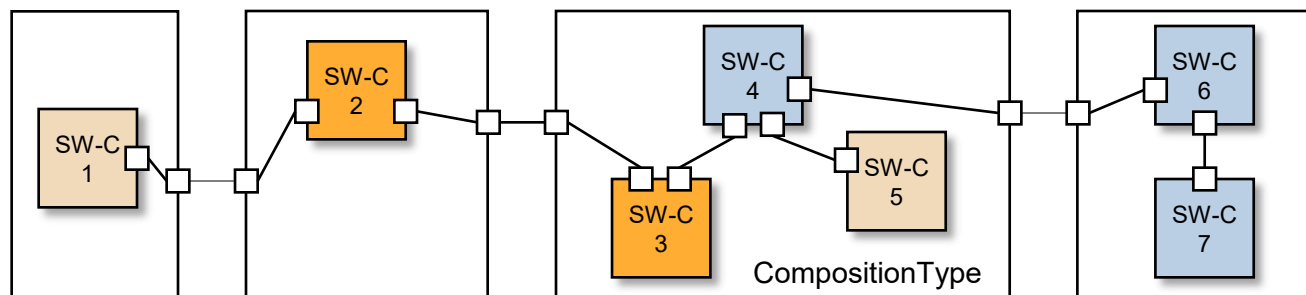
FlexRay的限制条件

- FlexRay仅支持请求和释放PNC。■ 由于没有支持PN的可用硬件，FlexRay节点无法被关闭。

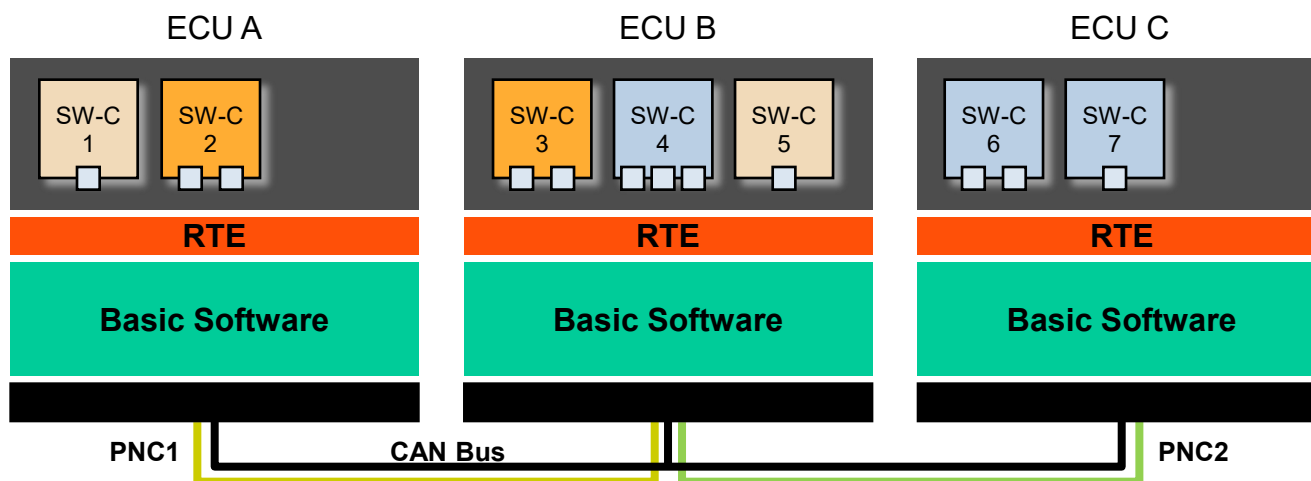
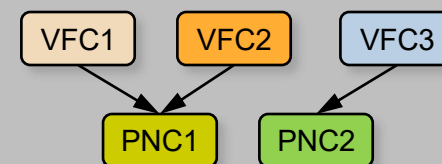
* 连接到从属总线的所有节点始终处于激活状态。不可能仅激活节点的子集。

Energy Management – Partial Networking

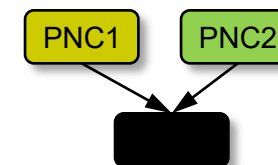
虚拟函数集群到部分网络集群的映射



Mapping of
VFC on PNC

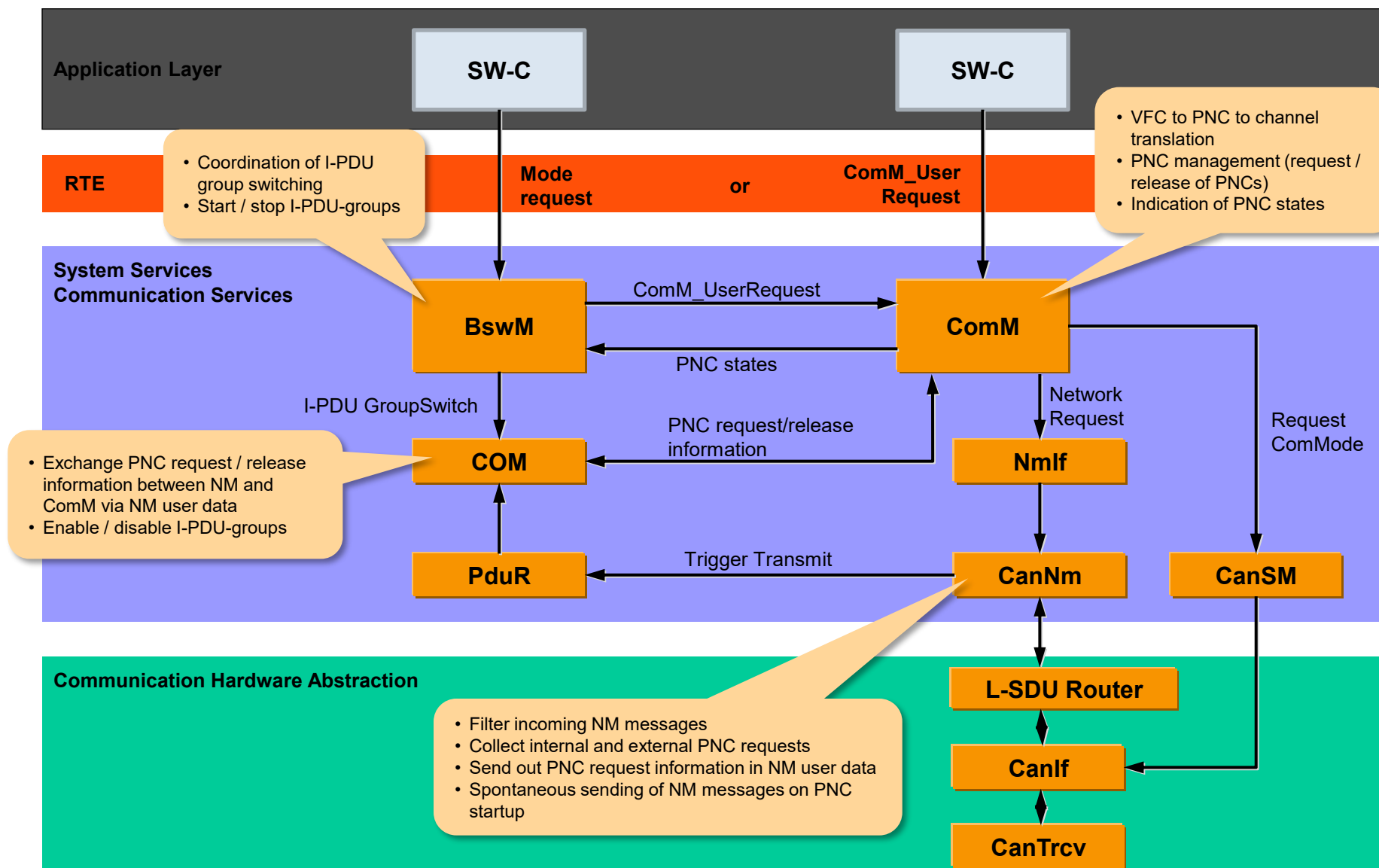


- Here both Partial Networks map to one CAN bus.
- One Partial Network can also span more than one bus.



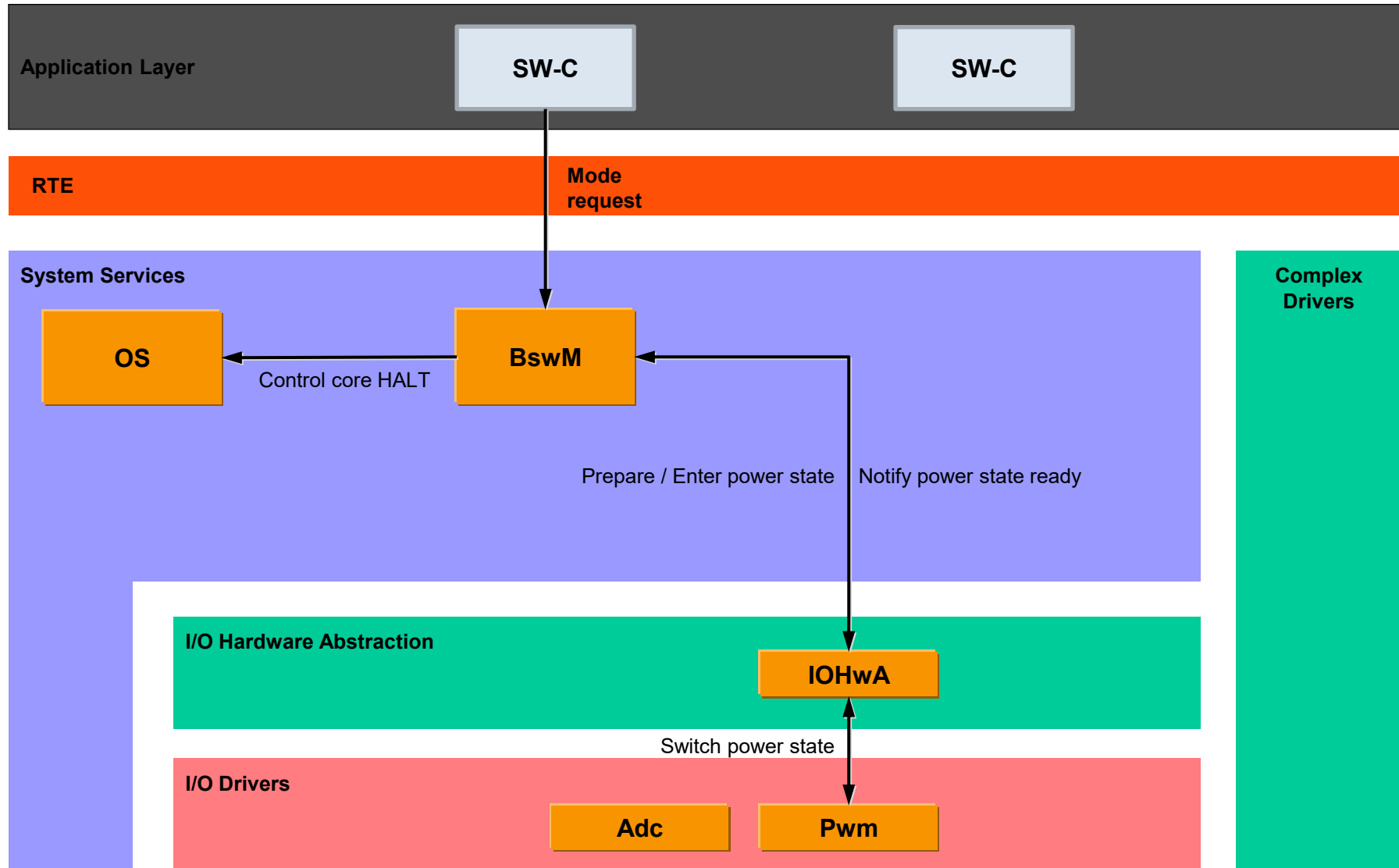
Energy Management – Partial Networking

涉及的模块 – CAN解决方案



Energy Management – ECU Degradation

涉及的模块 – I/O驱动器的解决方案



Energy Management – ECU Degradation

限制条件

ECU Degradation 目前仅在 MCAL 驱动 Pwm 和 Adc 上受支持。 Core HALT 与 ECU 睡眠被视为互斥模式。 通过修改时钟来降低功耗不在该概念的范围內（但仍可通过特定的 MCU 驱动配置实现）。

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射 2. 分区管理 3. 调度机制 4. 模式管理 5. 错误处理、报告与诊断 6. 测量与标定 7. 安全机制 8. 能源管理 9. 全局时间同步

Integration and Runtime Aspects – Global Time Synchronization

全局时间同步在多个车载网络间提供同步的时间基准。

StbM提供以下功能：

时间规定

时间基准状态

时间网关

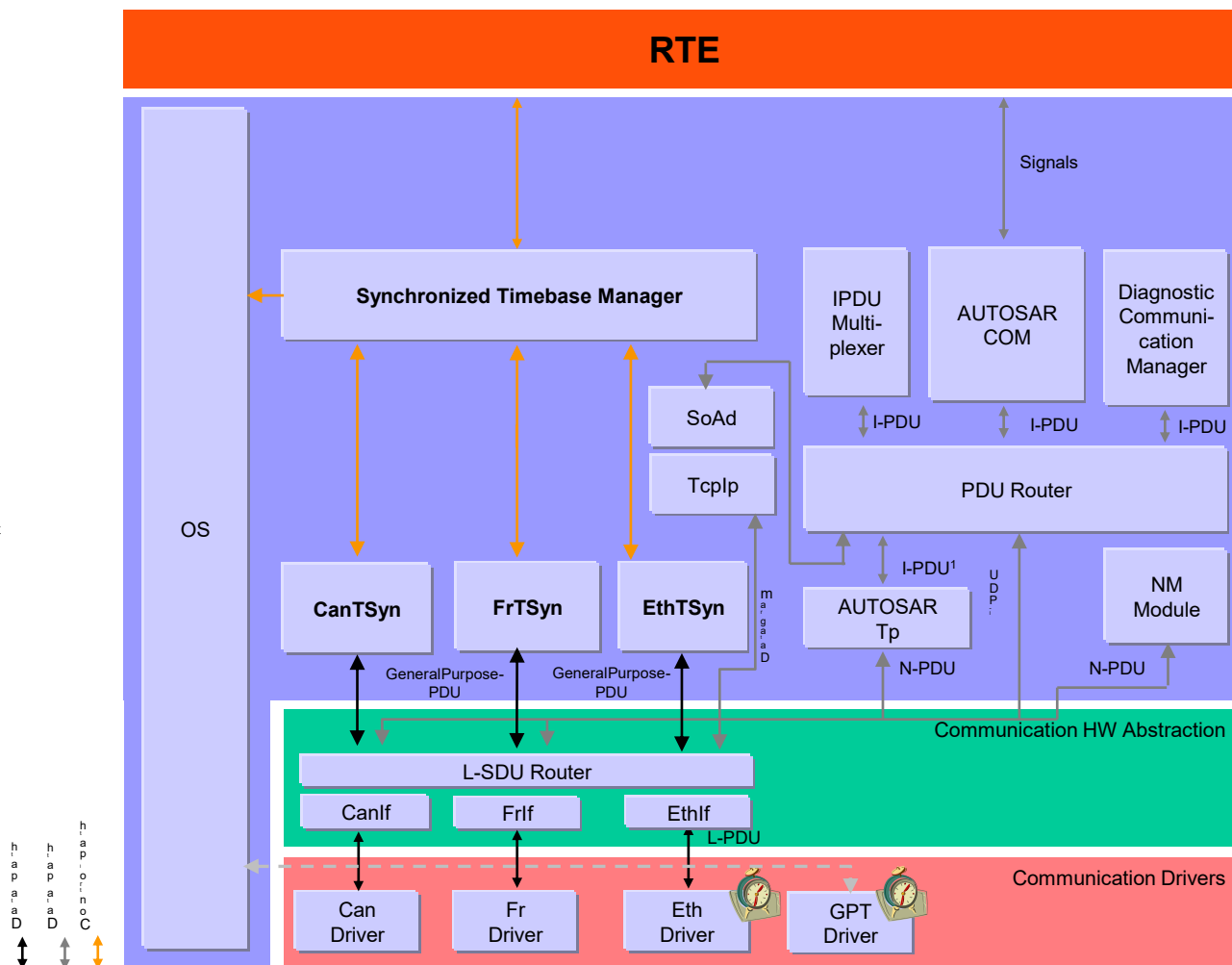
CanTSyn / FrTSyn / EthTSyn 提供网络特定的时间同步协议。

EthTSyn 还额外提供了速率校正和延迟计算功能。

用例示例：

传感器数据融合

跨ECU日志记录



Integration and Runtime Aspects – Secure Global Time Synchronization

安全全局时间同步确保车载网络内同步时间基准的完整性和真实性。

